



Universidad de Murcia

Facultad de Informática

Proyecto Fin de Carrera

QFTbx, herramienta de diseño QFT: especificación de requisitos y prototipado

Isaac Martínez Forte

48474854-P

isaac.martinez@um.es

Director:

Joaquín Cervera López

jcervera@um.es

5º Ingeniería Informática

Murcia - septiembre 2013

Resumen

En este trabajo se aborda la creación de un prototipo software que reúna las distintas investigaciones realizadas sobre la técnica de diseño de controladores robustos *QFT* en el Grupo de Investigación *Informática Industrial*, con la pretensión de que pueda incluir más adelante investigaciones de ámbitos más amplios, bajo una implementación libre y abierta. Actualmente hay software *CACSD* (Control Aided Control System Design) que facilita la realización de controladores con *QFT* todas ellas tienen algún factor que las hace no adecuadas para llevar a cabo la tarea propuesta, fundamentalmente el hecho de no ser software libre que permita su ampliación futura. Por ello este proyecto aborda la creación de un prototipo inicial que implementa las primeras fases de *QFT* hasta el cálculo de fronteras y teniendo como vías futuras su crecimiento aunando las distintas investigaciones realizadas y por realizar sobre el tema.

Este proyecto se plantea como una necesidad del grupo de investigación de *Informática Industrial* del departamento de *Informática y Sistema*, así pues, dicho grupo de investigación actuó como cliente del proyecto y se realizaron una serie de reuniones para la extracción de requisitos. En dichas reuniones se estudiaron todas las características que se podían incluir en el software a desarrollar, las cuales fueron muy amplias debido a la clara orientación hacia la investigación de dicho software.

Como paso previo a la implementación del software se realizó una especificación en *UML* de los requisitos obtenidos de los clientes, que permitió crear los “planos” del software antes de su implementación. A partir de estos “planos” se realizó un prototipo que implementa parcialmente las primeras fase del diseño de controladores mediante *QFT*. La realización de los “planos” es un paso fundamental para facilitar la futura ampliación del prototipo. Para la implementación se han utilizado las librerías de *Qt* sobre *C++* para la construcción de la *GUI* y los tipos de datos básicos, lo cual ha permitido que el prototipo sea multiplataforma.

En el presente proyecto se han implementado en *C++* los distintos algoritmos necesarios para las fases de *QFT* desarrolladas: un algoritmo para el cálculo de plantillas por fuerza bruta, el algoritmo ϵ - *hull* para el cálculo de las envolventes de las plantillas y un algoritmo de cálculo de fronteras multivaluadas.

Como parte final del proyecto se ha realizado un ejemplos ilustrativo donde se puede el correcto funcionamiento del prototipo mediante la comparación de los resultados con una herramienta clásica, de fiabilidad más que contrastada, para el mismo propósito.

Abstract

In this paper we deal with the creation of a prototype software package that collects the various investigations about the technique of robust control design *QFT* under free implementation. Currently, there are software packages *CACSD* (Control Aided Control System Design) which ease the design of *QFT* controllers in the research group on *Industrial Computer* with the claim that they may include further investigations of broader areas, but they all have some factor that makes those packages inappropriate to grow as free software, from proprietary code applications to others in which unfree languages are used. Due to these restrictions, the aim of this project is to create an initial prototype that implements the first stages of *QFT* up to the calculation of boundaries as the basis for a future development, combining previous and forthcoming investigations on this topic.

This project is presented as a need of the research group on *Industrial Computer Engineering and Systems* Department. That research group operated as a client, holding meetings to establish the requirements of the project. In those meetings, every characteristic that could be included in the software was discussed; characteristics which were wide indeed due to clear to the research orientation of such software.

The first step prior to the implementation of the software was an *UML* specification of the client's requirements was performed, which permitted the drawing up of the basic 'plans' of the software. From these 'plans' a prototype partially implements the first stage of the controller design by *QFT*. The realization of the 'plans' is a fundamental step to facilitate the future expansion of the prototype. For the implementation, *Qt* libraries using *C++* for the construction of the *GUI* and basic data types were used, which allowed the prototype to be multiplatform.

In this project we have implemented the various algorithms required for the *QFT* phases developed: algorithms for the calculation of brute force templates, ϵ - *hull* algorithms for calculation of the enveloping templates, and algorithms for calculating multi-valued boundaries.

As final part of the project, illustrative example have been carried out, in which the appropriate operation of the prototype can be checked out by comparación by the results with a conventional tool, proven reliability than, for the same purpose.

Agradecimientos

En primer lugar quiero agradecer al profesor *Joaquín Cervera López* por el tiempo que le ha dedicado a este proyecto. Han sido muchos meses de duro trabajo en los cuales siempre ha estado disponible para resolver mis dudas incluso en periodo de vacaciones. También agradecerle los 3 años como alumno interno donde he podido descubrir mi vocación por la informática industrial y por el control.

En segundo lugar agradecer al profesor *Rafael García Valencia* por aceptar como prácticas de la asignatura *Ingeniería de requisitos* la especificación de requisitos recogida en este documento, a él le supuso un esfuerzo adicional y por ello estoy muy agradecido. Agradecer al resto de profesores a los que en algún momento hemos tenido que hacer consultas puntuales y que siempre nos han ayudado de buen grado.

Una vez acabados los agradecimientos “académicos”, uno empieza a pensar en todas aquellas personas que le han ayudado directa e indirectamente en la conclusión de este proyecto, personas que me han animado y soportado durante este tiempo. Mi compañero de piso *Pedro*, estudiante de matemáticas, el cual siempre estuvo dispuesto a echarme una mano con alguna definición que se me atascaba, pero sobre todo, que junto a mi compañera de piso *Cati* venían a rescatarme con sus risas cuando necesitaba huir de las líneas de código del proyecto.

Por supuesto agradecer a mi familia, a mi prima *Sofía*, a mi hermana *Irene* y a mis padres, *Fini* y *José Rafael*, que junto con mis amigos siempre me dieron ánimo y mostraron interés por mi proyecto, aunque la mayoría de ellos se arrepintieran de preguntarme “¿De qué va tu proyecto?”, siempre intentaron aparentar que “algo” habían entendido.

Y para terminar un agradecimiento muy especial a *Elena*, ya que los dos hemos vivido juntos la experiencia de tener que realizar un proyecto de este tipo, de facultades diferentes, pero de esfuerzo similar. El ánimo, comprensión y apoyo mutuo ha sido fundamental para que las ganas de continuar hacia adelante no decayeran y concluyéramos con éxito nuestros respectivos proyectos.

Gracias a todos.

Isaac Martínez Forte

Notaciones

Notaciones usadas en QFT

- k_{hf} : ganancia de alta frecuencia un lazo $L(s)$,

$$k_{hf} = \lim_{s \rightarrow \infty} s^{n_{pe}} L(s) \quad (1)$$

donde n_{pe} es el exceso de polos sobre ceros, es decir, el número de polos menos el número de ceros.

- *Diagrama de Nichols* o *plano de Nichols*: forma de representación del plano complejo, basada en la representación polar de los números complejos. En el eje de abscisas se representa la fase en grados y en el de ordenadas la magnitud en decibelios. Algunos autores lo llaman también plano complejo logarítmico.
- **Fronteras multivaluadas**: Son fronteras que representadas en el *plano de Nichols* para una fase pueden tener más de solución. En la figura (1) se puede observar una frontera multivaluada con tres valores para una misma fase.

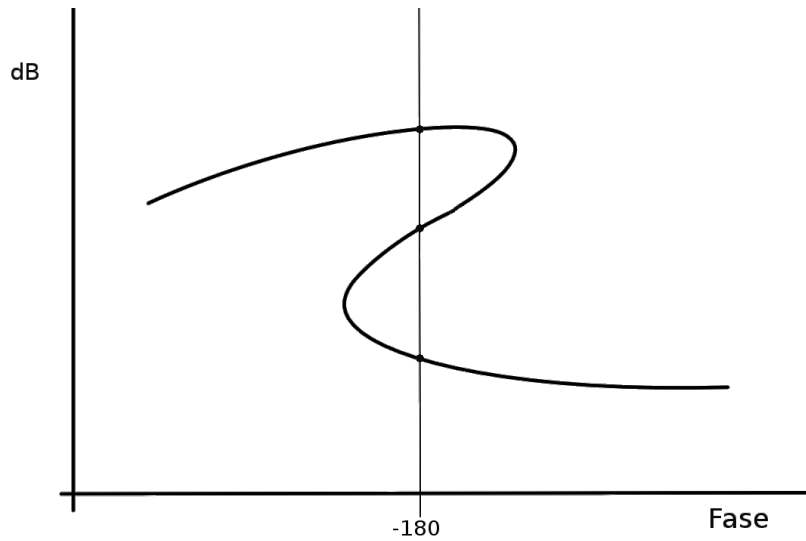


Figura 1: Explicación frontera multivaluada.

- *Círculo-M*: en el plano de Nichols, lugar de los puntos p que cumplen

$$\left| \frac{p}{1+p} \right| = \lambda$$

para un cierto λ . Se utilizan típicamente para definir fronteras de estabilidad robusta.

- Ω : conjunto de *frecuencias de diseño*, frecuencias a las que, dado un problema de control a resolver con QFT, se calcularán las fronteras dadas por las especificaciones y se comprobarán la no violación de dichas fronteras.

- *Función de transferencia*: Modelo matemático que a través de un cociente de polinomios en la variable compleja s relaciona la respuesta de un sistema a una señal de entrada o excitación. En la teoría de control, a menudo se usan las funciones de transferencia para caracterizar las relaciones de entrada y salida de componentes o de sistemas que se describen mediante ecuaciones diferenciales lineales e invariantes en el tiempo.
- *Diagrama de Bode* Es una representación gráfica que sirve para caracterizar la respuesta en frecuencia de un sistema. Normalmente consta de dos gráficas separadas, una que corresponde con la magnitud de dicha función y otra que corresponde con la fase.

Notaciones usadas en el modelo del dominio y la especificación de requisitos

- *Rol*: Comportamiento que muestra un objeto en relación al resto de objetos que participan con él en el sistema.
- *Actor*: Especifica un rol jugado por un usuario o cualquier otro sistema que interactúa con un sujeto. Los actores pueden representar roles jugados por usuarios humanos, hardware externo, u otros sujetos. Un actor no necesariamente representa una entidad física específica, sino simplemente una faceta particular (es decir, un “rol”) de alguna actividad que es relevante a la especificación de sus casos de uso asociados. Así, una única instancia física puede jugar el rol de muchos actores diferentes y, asimismo, un actor dado puede ser interpretado por múltiples instancias diferentes.
- *Caso de uso*: Es una descripción de los pasos o las actividades que deberán realizarse para llevar a cabo la conclusión de algún proceso. Los *personajes* o *entidades* que participan en un caso de uso se denominan actores. En el contexto de *ingeniería del software*, un caso de uso es una secuencia de interacciones que se desarrollan entre un sistema y sus actores en respuesta a un evento que inicia un actor principal sobre el propio sistema.
- *Diagrama estático*: Los Diagramas estáticos o también llamados estructurales se encargan de definir que elementos (entidades, objetos, áreas, clases, departamentos, componentes, etc.) deben de estar definidos dentro del sistema u organización del cual se quiere desarrollar el correspondiente modelado. También se encarga de especificar cómo deben de estar estructurados estos elementos. En cambio los modelos dinámicos se encargan de definir el comportamiento de estos.
- *Proyecto*: Un proyecto es una planificación que consiste en un conjunto de actividades que se encuentran interrelacionadas y coordinadas entre sí.
- *Patrón de diseño*: Son guías de soluciones a problemas comunes en el desarrollo de software, son ampliamente aceptados por la comunidad de desarrolladores, los principales patrones fueron publicados en el libro [GAMMA ET AL. [1995]], pero hay muchos más para problemas específicos y lenguajes concretos.

- *DAO: Data Access Object (DAO)* es un patrón de diseño que suministra una interfaz común entre la aplicación y uno o más dispositivos de almacenamiento de datos, tales como una base de datos o un archivo.

Otras notaciones usadas

- *Diagrama de caja*: Es una presentación visual de un conjunto de datos que describe varias características importantes, tales como la dispersión y simetría. Para su realización se representan los cuartiles primero, segundo y tercero, y los valores mínimo y máximo de los datos, sobre un rectángulo, alineado horizontal o verticalmente.



Figura 2: Explicación diagrama de caja.

- $X_{\min}$: valor mínimo de los datos.
 - Q_1 : primer cuartil.
 - Q_2 : segundo cuartil, coincide con la mediana.
 - Q_3 : tercer cuartil.
 - $X_{\max}$ valor máximo de los datos.
- *GUI*: Interfaz gráfica de usuario, acrónimo en inglés.
 - *IDE*: Entorno de desarrollo integrado, acrónimo en inglés.

Índice

Resumen	i
Abstract	ii
Agradecimientos	iii
Notaciones	iv
Notaciones usadas en QFT	iv
Notaciones usadas en el modelo del dominio y la especificación de requisitos . . .	v
Otras notaciones usadas	vi
Índice	viii
Índice de figuras	x
1 Introducción	1
1.1 Motivación	1
1.2 Introducción a <i>QFT</i>	2
1.2.1 Modelado de la planta con su incertidumbre.	4
1.2.2 Elección de las frecuencias de diseño.	5
1.2.3 Cálculo de plantillas	5
1.2.4 Definición de especificaciones	6
1.2.5 Cálculo de fronteras	7
1.2.6 Síntesis del controlador	8
1.2.7 Síntesis del pre-filtro (opcional)	8
1.2.8 Validación del diseño	9
1.3 Estado del Arte	9
2 Metodología	12
2.1 Proceso Unificado de Desarrollo de Software	12
2.2 Introducción a UML	13
3 Especificación de requisitos	17
3.1 Introducción	17
3.2 Diagrama conceptual	19
3.3 Diagrama de actividad	20
3.4 Glosario	21
3.5 Diagrama de casos de uso	22
3.6 Requisitos	25
3.7 Definir el problema	26
3.8 Introducir frecuencias de diseño	33
3.9 Cálculo de plantillas	36
3.10 Introducir especificaciones	43
3.11 Introducir nuevo algoritmo	46
3.12 Cálculo de boundaries	49
3.13 Diagrama de clases	53

4	Diseño del prototipo	54
4.1	Estudio de alternativas	54
4.1.1	Análisis con benchmark	56
4.1.2	Comparación basada en el “white paper” [DALHEIMER. [2002]]	58
4.1.3	Conclusiones	59
4.2	Introducción a <i>Qt</i>	59
4.3	Sistema de guardado	61
4.4	Trabajo colaborativo	63
4.5	Patrones de diseño utilizados	64
4.5.1	<i>Modelo-Vista-Controlador</i>	65
4.5.2	<i>Observer</i>	65
4.5.3	<i>DAO</i>	67
4.6	Librerías utilizadas	68
4.6.1	Librería <i>Contour</i>	69
4.6.2	Librería <i>QCustomPlot</i>	70
4.6.3	Función <i>pnpoly</i>	71
4.7	Algoritmos implementados en el prototipo	71
4.7.1	Algoritmo de fuerza bruta para el cálculo de las plantillas	71
4.7.2	Algoritmo e-hull para el cálculo del contorno de las plantillas	71
4.7.3	Algoritmo para el cálculo de fronteras	74
4.8	Documentación generada con <i>Doxygen</i>	76
5	Manual de usuario y batería de pruebas	77
5.1	Pantalla principal	77
5.2	Introducir la planta	77
5.3	Introducir las frecuencias de diseño (ω)	78
5.4	<i>Diagrama de Bode</i>	79
5.5	Cálculo de plantillas	80
5.6	Cálculo de fronteras	82
6	Conclusiones y vías futuras	84
6.1	Conclusiones	84
6.2	Vías futuras	85
A	Anejo 1: GNU GENERAL PUBLIC LICENSE	88
B	Anejo 2: Código de los algoritmos implementados	100
B.1	Algoritmo de cálculo de plantillas	100
B.2	Algoritmo <i>e – hull</i> para el cálculo de contornos de plantillas	101
B.3	Algoritmo de cálculo de fronteras	106
C	Anejo 3: Ejemplo de planta guardada en <i>XML</i>	110
	Bibliografía	114

Índice de figuras

1	Explicación frontera multivaluada.	iv
2	Explicación diagrama de caja.	vi
3	Sistema de control de <i>lazo abierto</i> .	3
4	Sistema de control de <i>lazo cerrado</i> .	3
5	Plantillas de la planta (4).	6
6	Fronteras sin agrupar para la planta (4).	8
7	Fronteras agrupados para la planta (4).	9
8	Gráfica de síntesis del controlador basándose en la ecuación (11).	10
9	Historia de UML.	14
10	Diagrama conceptual.	19
11	Diagrama de actividad.	20
12	Diagrama de casos de uso.	22
13	Casos de Uso: Esquema típico de control realimentado.	23
14	Diagrama de secuencia del sistema: Definir el problema (camino principal).	28
15	Diagrama de secuencia del sistema: Definir el problema usando datos experimentales.	29
16	Colaboraciones: Crear Proyecto.	30
17	Colaboraciones: Introducir Planta.	31
18	Colaboraciones: Modificar diagrama de Bode.	32
19	Diagrama de secuencia del sistema: introducir frecuencias de diseño.	34
20	Colaboraciones: Introducir las frecuencias de diseño en el sistema.	35
21	Diagrama de secuencia del sistema: cálculo de plantillas.	38
22	Colaboraciones: Elegir Algoritmo para el cálculo de plantillas.	39
23	Colaboraciones: Ejecutar algoritmo para calcular las plantillas.	40
24	Colaboraciones: Elegir algoritmo para el cálculo de la envolvente de la nube.	41
25	Colaboraciones: Ejecutar el algoritmo de cálculo de la envolvente de la nube	42
26	Diagrama de secuencia del sistema: introducir especificaciones del controlador.	44
27	Colaboraciones: Introducir especificaciones del controlador.	45
28	Diagrama de secuencia del sistema: introducir nuevo algoritmo.	47
29	Colaboraciones: Introducir nuevo algoritmo.	48
30	Diagrama de secuencia del sistema: Cálculo de fronteras.	50
31	Colaboraciones: Elegir algoritmo para el Cálculo de fronteras.	51
32	Colaboraciones: Ejecutar el algoritmo para calcular los boundaries.	52
33	Diagrama de clases.	53
34	Tabla con los lenguajes de programación más usados.	55
35	Gráfico general de tiempos de ejecución de los benchmark.	57
36	Diagrama de clases del patrón Modelo-Vista-Controlador.	65
37	Diagrama de secuencia del patrón Modelo-Vista-Controlador.	66
38	Diagrama de secuencia del patrón Modelo-Vista-Controlador sustituyendo la <i>GUI</i> por un navegador.	66
39	Diagrama de clases del patrón Observer	67
40	Diagrama de clases del patrón <i>DAO</i> implementado en el sistema.	68
41	Diagrama de secuencia del patrón <i>DAO</i> .	69

42	Imagen que representa la búsqueda del segundo punto en el algoritmo de $\epsilon - hull$	72
43	Imagen que representa la búsqueda del siguiente punto en el algoritmo de $\epsilon - hull$	73
44	Pantalla principal del prototipo.	77
45	Pantalla de introducción de la planta en el prototipo prototipo.	78
46	Pantalla de introducción de la incertidumbre de la planta en el prototipo.	78
47	Pantalla de introducción de las frecuencias de diseño en el prototipo.	79
48	Pantalla del <i>diagrama de Bode</i> en el prototipo.	79
49	Pantalla de introducir datos para el algoritmo de cálculo de plantillas en el prototipo.	80
50	<i>Diagrama de Nyquist</i> de las plantillas calculado por el prototipo.	81
51	<i>Diagrama de Nichols</i> de las plantillas calculado por el prototipo.	81
52	<i>Diagrama de Nichols</i> del contorno de las plantillas calculado por el prototipo.	82
53	Pantalla de introducir datos para el algoritmo de cálculo de fronteras en el prototipo.	83
54	<i>Diagrama de Nichols</i> con las fronteras calculadas por el prototipo.	83

1 Introducción

1.1 Motivación

La motivación por la cual se propuso este proyecto era muy clara, el estado actual de la disciplina de *QFT* (*Quantitative feedback theory*) es que está en constante investigación, se están desarrollando nuevos y mejores algoritmos para las fases principales del proceso de diseño *QFT*, que serán descritas en la sección (1.2). De hecho para la fase *ajuste del lazo* no se puede decir que su informatización haya sido completada ni para los tipos más sencillos. Con lo cual es una disciplina en constante evolución y cambio, esta evolución se ha dado de manera heterogénea, con algoritmos individuales para cuestiones muy concretas desarrollados por diferentes investigadores. *QFT* es tan amplio que cada nueva investigación realiza avances en un parte específica del proceso completo, con la desventaja que conlleva para la unificación de todos estos resultados. En la sección (1.3) se verá como está actualmente el estado del arte, con esta segunda parte se podrá comprender completamente la necesidad de este proyecto.

El grupo de investigación de Informática Industrial del departamento de Informática y Sistemas, tenía la necesidad de unificar los esfuerzos investigadores realizados durante años, en un software orientado a investigación y futuro desarrollo. Pero que fuera construido de manera que su uso fuera asequible para personas no expertas, de modo que conjugara las dos vertientes (investigación y uso sencillo).

La idea de aglutinar los distintos proyectos e investigaciones llevadas a cabo en el pasado junto con construir el sistema pensando en su ampliación futura, son dos de los objetivos principales del proyecto. El tercero de los objetivos se centraba en el rendimiento y la usabilidad.

Actualmente los programas existentes están casi todos basados en matlab, lenguaje que tiene sus puntos fuertes en la sencillez y en el manejo matricial, pero su rendimiento general no es muy adecuado para este tipo de aplicaciones de cálculo intensivo. Es por ello que uno de los objetivos principales de este proyecto es buscar un sistema que tenga un mejor rendimiento y eficiencia, para con ello poder abrir las puertas a nuevas funcionalidades basadas en una ejecución rápida de los distintos algoritmos. Como por ejemplo, que con una variación de las especificaciones se inicie una actualización en cadena que recalculé las *fronteras* en tiempo real.

Con estos tres requisitos fundamentales se hizo un análisis de la mejor plataforma y lenguaje de programación para realizar el proyecto, dicho estudio se puede consultar en la sección (4.1).

Es por ello se decidió que lo mejor era usar un lenguaje orientado a objetos, dado que su modularidad absoluta permite ampliaciones de funcionalidad sin que sea necesario modificar el resto del código. Otro de los factores que se tuvo en cuenta es que fuera multiplataforma, dado que el uso de *QFT* está muy extendido más allá de la *informática industrial* y es usado por todo tipo de ingenieros. Junto con la multiplataforma otra idea básica es la compatibilidad con paquetes matemáticos como pueden ser *Matlab*, *Octave* o *Scilab*. Se ha favorecido la exportación de datos a ficheros de texto que puedan ser importados de manera sencilla por otros sistemas.

También se estimó importante que el software fuera libre, para que en el futuro pudiera ser ampliamente utilizado y desarrollado más allá del contexto del grupo de investigación Informática Industrial y de los alumnos que colaboren con él. Es por ello que el software está bajo la licencia GPLv3 (ver [Anejo 1: GNU GENERAL PUBLIC LICENSE](#)), para que cualquier interesado pueda modificar el software con la obligación principal de liberar las modificaciones que se realicen, de esta manera se asegura que el futuro del proyecto pasa por ser software libre y sobre todo que se basa en la colaboración de los distintos interesados en usar y mejorar el software.

A continuación, en la sección 1.2, se da una breve introducción a *QFT*.

1.2 Introducción a *QFT*

El objetivo de un sistema de control es conseguir que un sistema dinámico, denominado planta, tenga un comportamiento deseado, dado por unas especificaciones de comportamiento. Esto se consigue diseñando uno o más sistemas dinámicos adicionales, denominados controladores, que se conectan a la planta para modificar su comportamiento. Los sistemas dinámicos pueden modelarse de diferentes formas, siendo las dos principales *lazo abierto* (figura 3) y *lazo cerrado* (figura 4).

En este contexto tenemos que la señal de entrada $u(t)$ y la señal de salida $y(t)$ se entienden como una función en el tiempo de una variable real, $u : t \rightarrow R$, y representa la evolución en el tiempo de una magnitud física continua y, si el sistema es *LTI*¹, o bien mediante aproximaciones, en el dominio de la frecuencia (transformadas de Fourier o Laplace de las señales temporales, obteniendo señales tipo $U(j\omega)$ o $U(s)$, respectivamente).

En los sistemas de *lazo abierto* no hay relación entre la señal $y(t)$ que se obtiene a la salida, con la señal $u(t)$ que se tiene en la entrada, es decir, no hay información de la salida del sistema en la entrada. Con lo cual, la planta debe ser conocida a la perfección y no existir incertidumbre sobre ella. La función de transferencia de *lazo abierto*, por definición es:

$$L(s) = C(s)P(s) \quad (2)$$

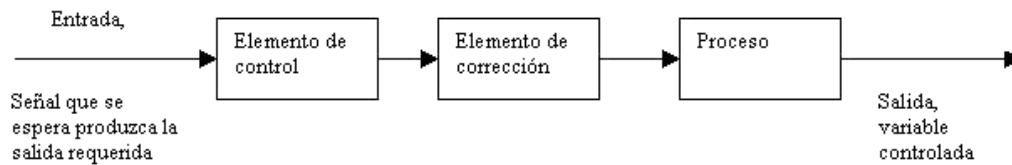
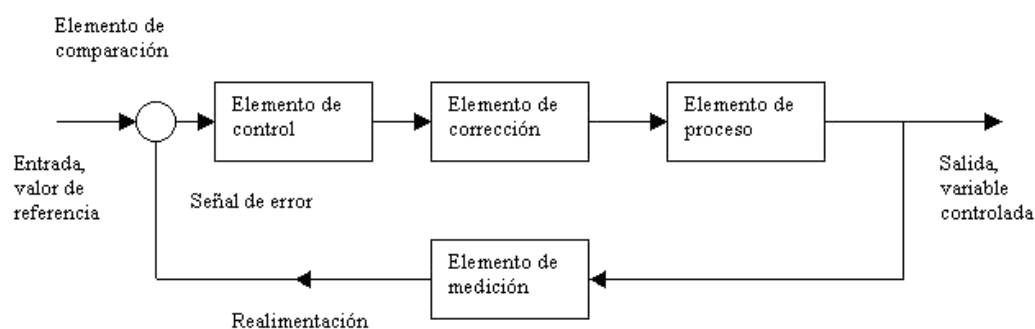
Donde $C(s)$ es el controlador, $P(s)$ es el sistema a controlar y $L(s)$ es el sistema controlador.

En cambio, en un sistema que sigue un modelo E/S (*lazo cerrado*), la señal $y(t)$ que se obtiene a la salida de un sistema y la señal de entrada $u(t)$ se relacionan entre sí, es decir, se tiene información de la salida que se puede utilizar para modificar la señal de entrada. La función de transferencia de *lazo cerrado*, por definición es:

$$T(s) = \frac{L(s)}{1 + F(s)L(s)} \quad (3)$$

Donde $F(s)$ es la realimentación en el sistema y $T(s)$ es el sistema controlador.

¹Lineal e invariante en el tiempo

Figura 3: Sistema de control de *lazo abierto*.Figura 4: Sistema de control de *lazo cerrado*.

Se habla de control robusto cuando el controlador es diseñado de modo que sea capaz de cumplir las especificaciones a pesar de la incertidumbre y las perturbaciones sobre la planta. *QFT* es una técnica de control robusto en el dominio de la frecuencia que se focaliza en cuantificar la relación entre incertidumbre y esfuerzo de control necesario para cumplir especificaciones a pesar de esa incertidumbre en la planta.

En el año 1945 Bode publicó el libro *Network Analysis and Feedback Amplifier Design* [BODE [1945]]. Donde introdujo los fundamentos de la realimentación, del análisis de sistemas y del diseño de compensadores en el dominio de la frecuencia.

En 1959 el científico Isaac M. Horowitz publicó un artículo titulado *Fundamental theory of automatic linear feedback control systems* [HOROWITZ [1959]] donde introdujo el concepto de cuantitativo a las ideas principales de Bode, introduciendo con ello la idea de que es necesario cuantificar el diseño del controlador respecto a las especificaciones requeridas y la incertidumbre de la planta. Este es el inicio del control cuantitativo en el dominio de la frecuencia. En 1963, Horowitz ampliaría estas ideas en su libro *Synthesis of Feedback Systems* [HOROWITZ [1963]], aportando novedosas ideas como el control basado en dos grados de libertad, entre otras.

QFT es un método de ingeniería de control en el cual se usa la realimentación cuantitativa para paliar los efectos de la incertidumbre en la planta. En general, la realimentación no es estrictamente necesaria a menos que exista incertidumbre en el problema de control

a resolver. En las técnicas de control robusto se incluye la incertidumbre como forma de asegurar que el sistema de control funcione correctamente para cualquier dinámica del proceso real, siempre que pertenezca al conjunto utilizado para modelar la planta, incluyendo la incertidumbre. De este modo, por el que se incorpora a priori un modelo de la incertidumbre al proceso de diseño, se garantiza que el diseño realizado satisface las especificaciones para cualquier planta del conjunto considerado. Obviamente, en la resolución de este problema existen restricciones, especialmente en términos del esfuerzo de control.

En *QFT*, la incertidumbre de la planta se representa mediante plantillas de incertidumbre dependientes de la frecuencia (*templates* en la literatura sobre el tema). Tanto la incertidumbre del modelo de la planta $P(j\omega)$ como sus especificaciones frecuenciales y temporales son trasladadas a un conjunto de curvas $B(j\omega_i)$ (para cada frecuencia de diseño ω_i) en el diagrama de Nichols, llamadas *fronteras*, *bounds* o *boundaries*, que representan las restricciones de diseño. El paso principal, denominado *ajuste del lazo* o *loop shaping*, consiste en buscar una función de transferencia nominal de lazo abierto $L_0(j\omega) = C(j\omega)P_0(j\omega)$ para dichas restricciones de modo que se obtenga un controlador lo más simple posible, que requiera el menor esfuerzo de control y que satisfaga el conjunto de especificaciones deseadas para cualquier posible planta dentro de la incertidumbre.

Las principales fases del método son las siguientes:

- Modelado de la planta con su incertidumbre.
- Elección del conjunto de frecuencias de diseño (ω).
- Generación de plantillas (*templates*) y elección de planta nominal $P_0(j\omega)$.
- Definición de especificaciones.
- Generación de fronteras $B(j\omega)$ (*boundaries*).
- Síntesis del controlador $C(j\omega)$.
- Síntesis del pre-filtro $F(j\omega)$ (opcional).
- Validación del diseño.

A continuación se detalla una breve explicación de todas las fases del método, para ejemplificar la explicación se ha usado el ejemplo número uno del *Toolbox de QFT de Matlab* [BORGHESANI ET AL. [2003]].

1.2.1 Modelado de la planta con su incertidumbre.

Esta fase es exclusiva de introducción de datos por parte del usuario, en ella el usuario deberá introducir el modelo de la planta que desea controlar, en este caso del ejemplo es el siguiente:

$$P(s) = \frac{k}{(s+a)(s+b)} \quad (4)$$

$$k \in [1, 10]$$

$$a \in [1, 5]$$

$$b \in [20, 30]$$

QFT permite la descripción del modelo dinámico de la planta a controlar desde muy diversas perspectivas, mediante datos de respuesta frecuencial, a partir de datos experimentales, mediante funciones de transferencia lineales, variantes (LTV) o invariantes (LTI) en el tiempo, mediante ecuaciones no lineales y mediante ecuaciones en espacio de estados, todas ellas con incertidumbre no paramétrica y/o paramétrica, la cuál a su vez puede ser, entre otras intervalar, afín y probabilística.

1.2.2 Elección de las frecuencias de diseño.

En esta fase se trata simplemente de elegir las frecuencias de diseño que el usuario estime oportunas, suelen ser pocas las frecuencias elegidas. Las formas más habituales de introducir las frecuencias de diseño son las siguientes.

- Espaciado lineal entre un inicio y un fin (rango) con un número de puntos determinado.
- Espaciado logarítmico entre un inicio y un fin (rango) con un número de puntos determinado.
- Frecuencias introducidas manualmente.

En el ejemplo utilizado en esta sección las frecuencias de diseño, introducidas de forma manual, son las siguientes:

$$\omega = 0.1, 5, 10, 100$$

1.2.3 Cálculo de plantillas

Los modelos de la planta son trasladados al dominio de la frecuencia, representándose en el diagrama de Nichols, para cada frecuencia de diseño un conjunto de puntos (números complejos con magnitud en decibelios y fase en grados). Las plantillas representan la información disponible de la planta, en *QFT*, las plantillas pueden responder a diferentes niveles de estructura en la incertidumbre. El nivel más alto es la incertidumbre paramétrica, siendo usualmente una buena práctica utilizar parámetros que tengan significado físico. El nivel más bajo es la incertidumbre no estructurada, en torno a un modelo nominal. Las variaciones respecto del modelo nominal no tienen información de fase, solo de magnitud (típicamente se expresan a partir de una norma). Representaciones más estructuradas se pueden desestructurar, obteniendo resultados más conservadores. Sin embargo, el paso inverso no es posible, debido a la pérdida de información en una representación no estructurada.

En el caso de incertidumbre paramétrica, el problema general de cálculo de las plantillas no está completamente resuelto, salvo por el método de fuerza bruta, de hecho este es un tema de investigación activo en la actualidad. Sin embargo, existen en la literatura una

serie de técnicas que permiten calcular, de forma eficiente, las plantillas para diferentes niveles de estructura en los parámetros, y que son adecuadas para la gran mayoría de los problemas prácticos.

Respecto al ejemplo usado, en la figura (5) se puede observar el resultados para cada frecuencia de diseño, estos han sido calculados implementando un algoritmo sencillo, propiciado por la conocimiento previo que tenían de la planta, utilizando varios bucles anidados calculan directamente el contorno de la nube de puntos.

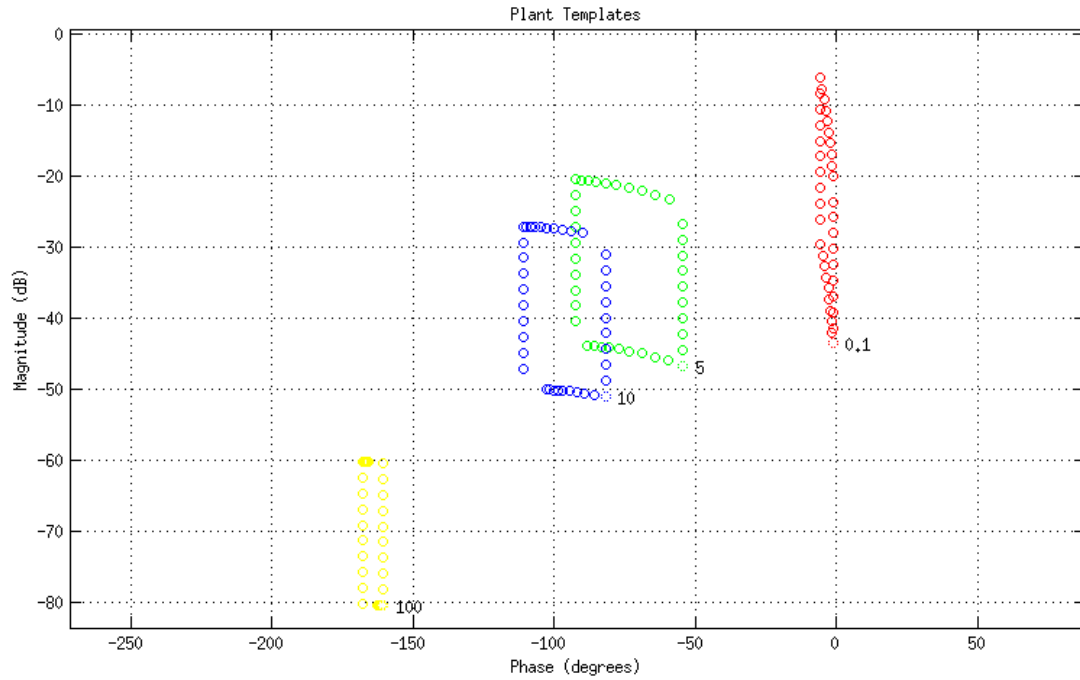


Figura 5: Plantillas de la planta (4).

1.2.4 Definición de especificaciones

En *QFT*, las especificaciones de lazo cerrado se dan en el dominio de la frecuencia, en términos de cotas admisibles sobre la respuesta en frecuencia de las funciones de transferencia de lazo cerrado.

Las restricciones que típicamente se han venido considerando en *QFT* son las siguientes:

(1) Seguimiento:

$$\alpha(\omega) \leq \left| \frac{L(j\omega)}{1 + L(j\omega)} F(j\omega) \right| \leq \beta(\omega) \quad \forall \omega > 0 \quad (5)$$

(2) Estabilidad:

$$\left| \frac{L(j\omega)}{1 + L(j\omega)} \right| \leq \lambda \quad \forall \omega > 0 \quad (6)$$

(3) Eliminación de ruido del sensor:

$$\left| \frac{L(j\omega)}{1 + L(j\omega)} \right| \leq \delta_n(\omega) \quad \forall \omega > 0 \quad (7)$$

(4) Eliminación de perturbaciones en la salida de la planta:

$$\left| \frac{1}{1 + L(j\omega)} \right| \leq \delta_{po}(\omega) \quad \forall \omega > 0 \quad (8)$$

(5) Eliminación de perturbaciones en la entrada de la planta:

$$\left| \frac{P(j\omega)}{1 + L(j\omega)} \right| \leq \delta_{pi}(\omega) \quad \forall \omega > 0 \quad (9)$$

(6) Esfuerzo de control:

$$\left| \frac{C(j\omega)}{1 + L(j\omega)} \right| \leq \delta_{ce}(\omega) \quad \forall \omega > 0 \quad (10)$$

Obsérvese que la especificación de estabilidad robusta se da en términos de una limitación sobre la magnitud de lazo cerrado, lo que conlleva una especificación simultánea, y por tanto ligada, sobre el margen de fase y el margen de ganancia. Esta limitación conjunta suele recibir el nombre, en el plano de Nichols, de círculo-M.

1.2.5 Cálculo de fronteras

Las especificaciones se combinan con la incertidumbre del sistema, dada en forma de plantillas (vistas en la sección (1.2.3)), para obtener restricciones, o fronteras en terminología *QFT*.

Este es uno de los pasos clave en *QFT* en el que se traducen de las especificaciones en el dominio de la frecuencia a regiones en el plano de Nichols donde deberá mantenerse la función $L_0(j\omega)$. Estas regiones, que pueden ser conexas o no, están definidas por sus fronteras. Un cálculo preciso de las fronteras es importante, dado que determinarán el coste de la realimentación en términos de ganancia y ancho de banda del controlador.

El cálculo de fronteras requiere del uso de herramientas informáticas y no es trivial en el caso general, en que pueden ser multivaluadas. Además las fronteras pueden ser abiertas o cerradas. En general, una frontera delimita una región (conexa) del plano de Nichols, y puede expresarse como una función $M = F(\phi)$, donde F es la realimentación, $F = |1 + L|$. Se entiende por frontera multivaluada aquella en la cual la función F es multivaluada. Las fronteras cerradas, típicamente las de estabilidad, son al menos bivaluadas. Las fronteras de seguimiento, que típicamente son abiertas, pueden ser multivaluada en muchos casos.

Una vez calculadas las fronteras y dibujadas sobre el diagrama de Nichols, para cada especificación y cada frecuencia de diseño (figura 6), son agrupados de modo que finalmente exista una sola curva para cada frecuencia (figura 7), la fusión de las restricciones impuestas para cada frontera.

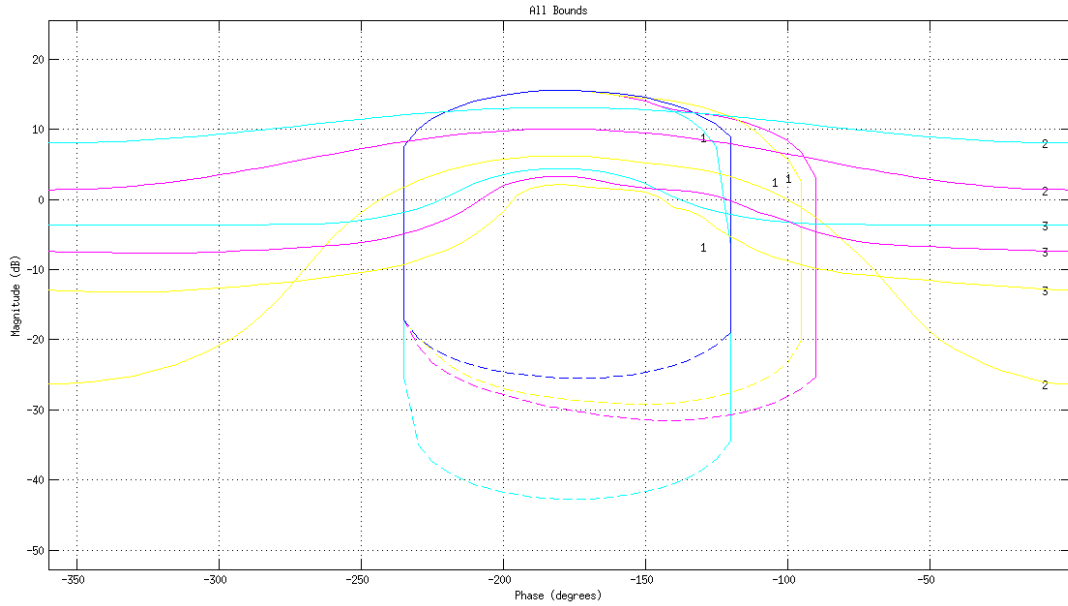


Figura 6: Fronteras sin agrupar para la planta (4).

1.2.6 Síntesis del controlador

Sin duda, la fase más difícil en *QFT*, y la que plantea más cuestiones abiertas, es el diseño de la función de ganancia del lazo nominal $L_0 = C(s)P_0(s)$. La síntesis consiste en el moldeo de la función $L_0(j\omega)$ en el *plano de Nichols*, de forma que se cumplan las especificaciones de diseño, lo que se traduce en que para cada frecuencia de diseño, el número complejo $L_0(j\omega)$ debe de estar fuera de la región prohibida definida por las fronteras calculadas previamente.

Una vez dibujados los contornos en el *diagrama de Nichols*, y superpuesta la función $L_0 = P_0C$ ($C = 1$, inicialmente), el controlador $C(j\omega)$ se diseña añadiendo polos y ceros al mismo hasta que la función L_0 se encuentre fuera de la región prohibida. El controlador óptimo (reducir al mínimo la ganancia de alta frecuencia) será, según demostraron Gera y Horowitz [GERA Y HOROWITZ [1980]], aquel que teniendo la mínima ganancia consiga que L_0 descansa sobre los contornos a cada frecuencia de interés.

Si nos centramos en el ejemplo que se ha usado a lo largo de esta sección, se puede observar el seguimiento de L_0 que para el controlador ejemplo (11) se obtiene la figura (8) como resultado.

$$C_2(s) = \frac{379(\frac{s}{42} + 1)}{\frac{s^2}{272^2} + \frac{s}{247} + 1} \quad (11)$$

1.2.7 Síntesis del pre-filtro (opcional)

En algunos sistemas de control se requieren especificaciones de seguimiento de referencia (tracking), en estos sistemas será necesario también el diseño de un pre-filtro $F(j\omega)$.

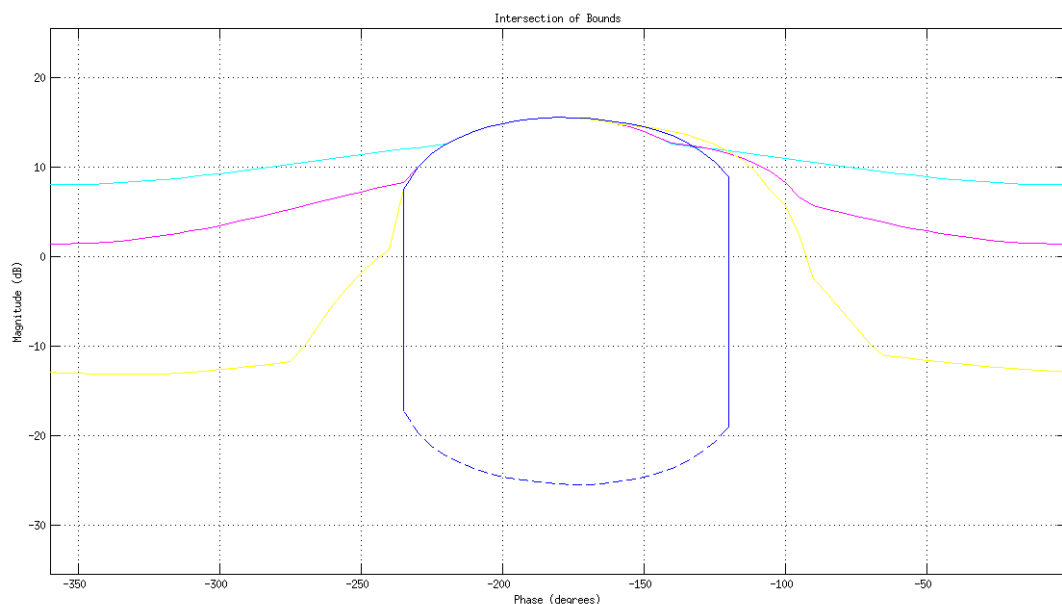


Figura 7: Fronteras agrupados para la planta (4).

Así mientras $C(j\omega)$ se encarga de reducir la incertidumbre y alcanzar los objetivos señalados anteriormente, el objetivo del pre-filtro es el cumplimiento de las especificaciones de tracking.

1.2.8 Validación del diseño

Una vez que se ha finalizado el diseño de $C(j\omega)$ es necesario analizar su comportamiento frente a las especificaciones deseadas, tanto en el dominio de la frecuencia como en el dominio del tiempo. No se ha podido demostrar que haya una relación directa entre el cumplimiento de las especificaciones en el dominio de la frecuencia y el cumplimiento de las especificaciones en el dominio del tiempo. Sin embargo, pocas veces aparece la paradoja de cumplirse las especificaciones frecuenciales asociadas y no las temporales. Por lo tanto, el principal objetivo de esta fase es validar el diseño realizado en un grupo amplio de especificaciones frecuenciales y de tiempo, por supuesto incluyendo los más desfavorables de la incertidumbre de la planta.

1.3 Estado del Arte

Como se mencionó en la sección de motivación (1.1) actualmente hay distintos software existentes que desempeñan una función similar al realizado en este proyecto. Se mencionan a continuación brevemente algunas de las herramientas de diseño *QFT* más importantes desarrolladas hasta la fecha:

- **QFT toolbox** [BORGHESANI ET AL. [2003]]: Se considera el toolbox clásico de *QFT*, fue distribuido durante algunos años por *The MathWorks Inc.*[®] como un toolbox de *Matlab*². Posteriormente, en el año 2000, fue distribuido por una empre-

²Página oficial: <http://www.mathworks.es/> Último acceso(5/09/2013)

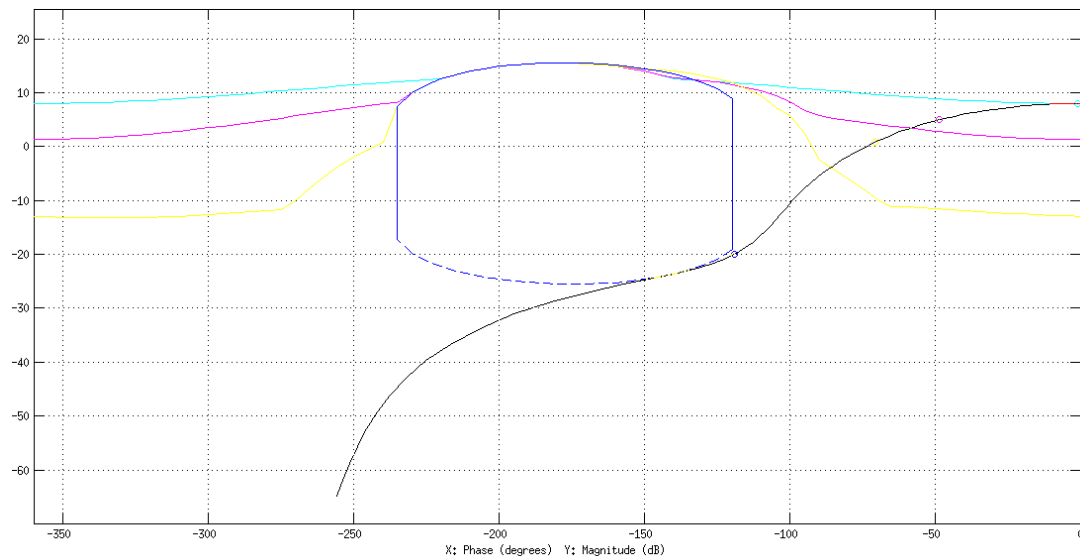


Figura 8: Gráfica de síntesis del controlador basándose en la ecuación (11).

sa independiente, *Terasoft Inc.*^{®3}, también como toolbox de Matlab. Finalmente, en 2013, fue puesto a libre disposición para descarga⁴, si bien no fue publicado como software libre. Los ficheros fuente están disponibles, de modo que es posible inspeccionar el funcionamiento de los algoritmos.

- **Qsyn** [GUTMAN [1996a]] [GUTMAN [1996b]]: También distribuido como toolbox de *QFT*, pero de forma independiente. Es software privativo y no está disponible para descarga gratuita. Introduce importantes mejoras en relación al *QFT toolbox*, como son las herramientas para cálculo de plantillas o la posibilidad de almacenar datos calculados (plantillas, fronteras, etc.) en ficheros. Está fuertemente basado en la introducción de la información en formato texto.
- **SISO-QFTIT** [DORMIDO ET AL. [2004]]: Una herramienta basada en el entorno de programación interactiva *Sysquake*⁵, de la empresa *Calerga*[®], que pone énfasis en la interactividad de cara a su uso como herramienta docente. Para aumentar la velocidad de respuesta de la herramienta, de cara a su propósito académico, se hacen algunas simplificaciones (conservativas) durante el proceso de diseño. La herramienta está disponible para descarga gratuita⁶, pero no es libre.
- **QFTCT** [GARCIA-SANZ ET AL. [2009]]: Disponible para descarga gratuita⁷ como toolbox de *Matlab*, si bien ni es software libre ni los ficheros fuente están disponibles. Añade, en relación al *QFT toolbox* original, características de interactividad muy interesantes, como es el hecho de tener diferentes informaciones gráficas a la vista de forma simultánea. También es interesante la adición de la introducción interactiva

³Página oficial: <http://www.terasoft.com/> Último acceso(5/09/2013)

⁴Página descarga: <http://pcsl.ecs.umass.edu/cbs/software.html> Último acceso(5/09/2013)

⁵Página oficial: <http://www.calerga.com/products/Sysquake/> Último acceso(5/09/2013)

⁶Página oficial: <http://ctb.dia.uned.es/asig/qftit/principal.html> Último acceso(5/09/2013)

⁷Página oficial: <http://cesc.case.edu/OurQFTCT.htm> Último acceso(5/09/2013)

de plantas con diferentes formatos para la incertidumbre, incluyendo distribuciones de probabilidad.

Estos son los principales software que cumplen una finalidad similar a la desarrollada en este prototipo, como se puede observar ninguno de ellos es software libre, además de que están programados en lenguajes no libres.

2 Metodología

Este proyecto está incluido dentro del ámbito de la informática industrial. Esta disciplina suele ser una de las más alejadas de lo que se entiende por informática clásica, aunque como es lo más lógico, en el mundo de las ingenierías no existe nada que esté completamente aislado. Este proyecto en particular lo está mucho menos, ya que como se comentó en la introducción (1) el objetivo es desarrollar un software que recopile las distintas investigaciones realizadas en este ámbito. Por tanto al final, este trabajo se trata de un proyecto software, con todas las implicaciones que eso conlleva.

Es por ello, que se vió muy importante guiar el desarrollo del proyecto desde el punto de vista del *análisis y diseño orientado a objetos*, así pues se decidió realizar de forma disciplinada un proceso de desarrollo de software. Se utilizó para ello el *proceso unificado de desarrollo de software*, que es un marco extensible que se adaptó al proyecto de forma específica.

2.1 Proceso Unificado de Desarrollo de Software

El Proceso Unificado se caracteriza por estar dirigido a casos de uso, centrado en la arquitectura y ser iterativo e incremental. El refinamiento más conocido es el *Proceso Unificado de Rational* [IBM [1996]], para este proyecto se ha utilizado una adaptación similar a RUP, en este caso, orientada a un proyecto de tamaño pequeño.

El Proceso Unificado no es simplemente un proceso, sino un marco de trabajo extensible que debe ser adaptado al proyecto específico que se esté realizando. De la misma forma, el Proceso Unificado de Rational, también es un marco de trabajo extensible, por lo que muchas veces, como en este caso, resulta imposible definir si un refinamiento particular del proceso ha sido derivado del Proceso Unificado o del RUP. Por dicho motivo, los dos nombres suelen utilizarse para referirse a un mismo concepto. El proceso unificado está fuertemente ligado al uso de UML (2.2) como lenguaje de modelado.

Las características principales del Proceso Unificado son las siguientes:

- Es iterativo e incremental: Está orientado a iteraciones cortas que constan de cuatro fases denominadas: Inicio, Elaboración, Construcción y Transición. Estas fases, a su vez pueden tener iteraciones internas, las fases de Elaboración y Construcción son muy propensas a necesitar iteraciones, dado que suelen ser las más complejas. La clave de las iteraciones es que ofrecen como resultado un *incremento* verificable del sistema con retroalimentación continua hacia el cliente y los propios desarrolladores.
- Dirigido por casos de uso: En el Proceso Unificado los casos de uso se utilizan para capturar los requisitos funcionales y para definir los contenidos de las iteraciones. La idea es que cada iteración tome un conjunto de casos de uso o escenarios y desarrolle todo el trabajo a través de las distintas fases: diseño, especificación, implementación, prueba, etc. En el caso concreto de este proyecto cada iteración se ha centrado en un caso de uso, dado que se diseñaron casos de uso generales y amplios. En la sección (3) hay una parte dedicada a los casos de uso, donde se explican con detalle.

- Centrado en la arquitectura: Al igual que un arquitecto dibuja los planos de un edificio antes de construirlo, lo mismo se puede hacer con el software, para un desarrollo completo es necesario “dibujar” la arquitectura del sistema antes de comenzar a implementarlo. Por supuesto, estos diagramas están sujetos a las iteraciones y mejoras que vayan surgiendo en el desarrollo, pero es necesario tener una base sobre la que empezar a construir el sistema y un buen diseño desemboca en una posterior construcción más sencilla.
- Enfocado a los riesgos: Es fundamental detectar para cada iteración cuales son las partes críticas del sistema y abordarlas cuanto antes, dado que así la construcción se hará de forma más fiable. Es lógico atacar en primer lugar las partes más importantes del sistema, dado que estas son la base de dicho sistema, y a partir de ahí, realizar el resto de partes que las acompañan.

Estas características aplicadas al proyecto se traducen en una serie de actuaciones que se describen en la siguiente sección (2.2). Como se comentó en la introducción (1) la motivación principal del proyecto es que el grupo de investigación Informática Industrial tenía la necesidad de una aplicación como la que se ha empezado a desarrollar en este proyecto. Se puede entender pues que este grupo de investigación es el cliente del proyecto. Para la captura de requisitos se hicieron una serie de reuniones/entrevistas con los miembros de dicho grupo, se realizó una entrevista por cada caso de uso desarrollado, que en este caso coinciden con las fases del proceso de diseño en QFT. En las entrevistas se hizo la captura de requisitos con las solicitudes de los clientes. En muchos casos estas fueron excesivamente amplias para el ámbito de un proyecto fin de carrera y se tuvieron que recortar. De cada entrevista se elaboró un acta donde están reflejados los requisitos capturados, actas que posteriormente fueron utilizados para el desarrollo del proyecto.

La facilidad que da QFT en estos casos es que las distintas fases son prácticamente independientes, con lo cual, la captura de requisitos se pudo hacer de forma individual e iterativa.

2.2 Introducción a UML

Lenguaje de Modelado Unificado (UML acrónimo en inglés). En 1994 Boosh, Rumbaugh (OMT) y Jacobson (Objectory/OOSE) decidieron unificar los métodos de modelado existentes en uno solo, así nació UML, actualmente es el lenguaje de modelado de sistemas software más usado. Está respaldado por el *Object Management Group* (OMG⁸). Es un lenguaje gráfico para especificar, documentar y finalmente construir un sistema software, se puede entender pues que UML se utiliza para diseñar los “planos” de dicho sistema. UML ofrece un estándar para describir el modelo de un sistema, incluyendo aspectos conceptuales tales como el proceso del negocio, funciones del sistema, y aspectos concretos como expresiones del lenguajes de programación, esquemas de bases de datos, etc. En la figura (9) se puede observar la evolución de UML a lo largo de los años.

UML es un lenguaje para modelar la “realidad” como primer paso hacia su programación, es muy distinto a un lenguaje de programación, su objetivo es ser un lenguaje

⁸Página web OMG: <http://www.omg.org/>

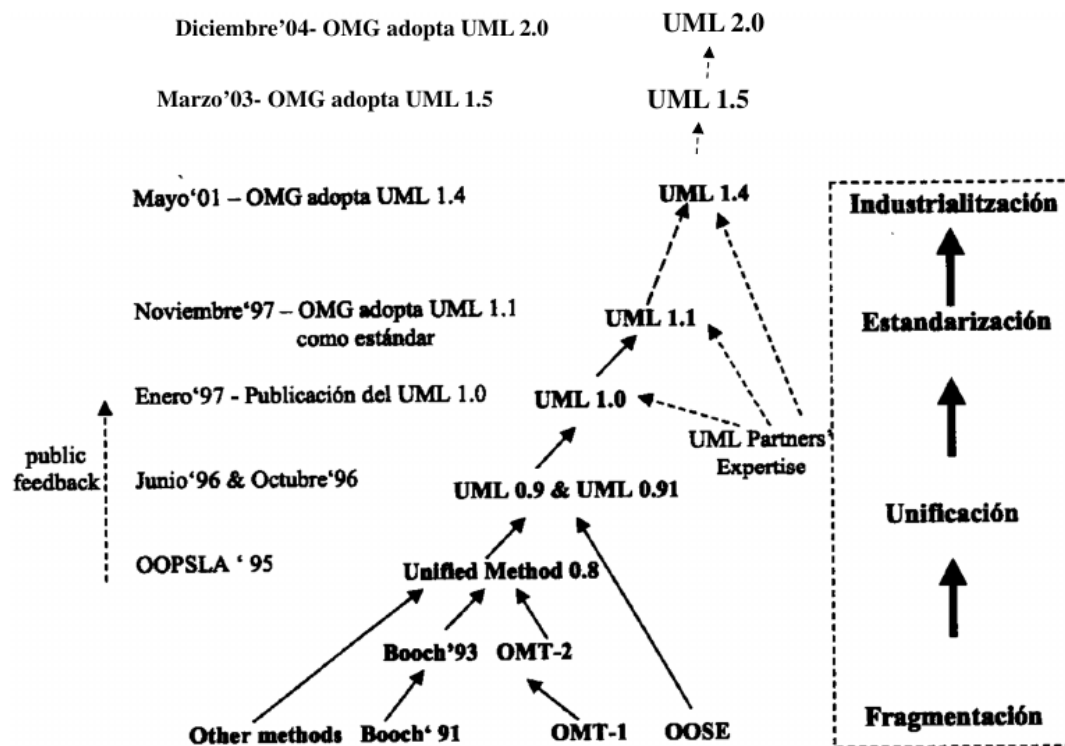


Figura 9: Historia de UML.

intermedio que pueda ser “entendido” tanto por una máquina como por los seres humanos. Asociado a esta última afirmación viene una de las grandes críticas hechas sobre UML, su semántica en muchas ocasiones no es precisa, con lo cual su interpretación puede no ser objetiva. Su aplicación a un desarrollo software se puede hacer de muchas formas, desde aplicarlo con una rigurosidad que permita posteriormente generar código automáticamente, hasta ser aplicado de manera informal solo con el objetivo de ser orientativo para el equipo de desarrollo.

La forma por la cual UML modela un sistema es a través de distintos diagramas, los cuales tienen una funcionalidad muy concreta. Durante el estudio UML realizado para este proyecto, se han utilizado varios de los principales diagramas presentes en UML, los cuales se van a explicar a continuación para una mejor comprensión de la sección (3) del documento.

Los distintos diagramas pueden agruparse en tres grupos fundamentales:

- **Comportamiento:**

- Diagrama de casos de uso: En primer lugar no hay que confundir un diagrama de casos de uso, con los casos de uso en sí, en este caso se trata solo del diagrama, donde se pueden observar de forma gráfica y en alto nivel los distintos comportamientos que tiene el sistema. Este diagrama refleja acciones concretas que tienen un principio y un fin, con la suficiente entidad para ser una

parte importante del sistema. En este diagrama aparecen también los actores relacionados con dichas funcionalidades.

- Diagrama de actividad: Es la representación gráfica del algoritmo o proceso, representa los flujos de trabajo, paso a paso, del sistema. En este diagrama se pueden representar de manera clara las interacciones entre los distintos elementos operacionales y los flujos de trabajo del sistema.

- **Estructura:**

- Diagrama conceptual: Representa de manera gráfica los distintos conceptos que forman parte del sistema y la relación entre ellos, este es uno de los diagramas básicos de UML, que se realiza al principio del modelado, en él se pueden representar en alto nivel entidades que están presentes en el sistema y cómo se relacionan entre ellas. Se suelen indicar también, los atributos principales de las entidades presentes en el diagrama, así como las cardinalidades en las relaciones entre entidades.
- Diagrama de clases: Un diagrama de clases es un tipo de diagrama estático que describe la estructura de un sistema mostrando sus clases, atributos y las relaciones entre ellos. Como su nombre indica, este diagrama representa las clases que formarán parte del sistema. Se puede hacer de manera suficientemente precisa como para poder generar código automáticamente a partir de este diagrama; esto no suele ser habitual, dado que para un proyecto grande este diagrama adquiriría una complejidad extremadamente grande. Lo normal suele ser que refleje las principales clases del sistema, las que tienen una funcionalidad importante, los atributos principales de estas clases y los métodos fundamentales.

- **Interacción:**

- Diagrama de secuencia: Es un tipo de diagrama usado para modelar la interacción entre objetos en un sistema a través del tiempo, contiene detalles de la implementación del escenario, incluyendo los objetos, las clases implementadas en el escenario y mensajes intercambiados entre ellos. Existen dos tipos de mensajes: síncronos y asíncronos. Los mensajes síncronos se corresponden con llamadas a métodos del objeto que recibe el mensaje. El objeto que envía el mensaje queda bloqueado hasta que termina la llamada. Este tipo de mensajes se representan con flechas con la cabeza llena. Los mensajes asíncronos terminan inmediatamente, y crean un nuevo hilo de ejecución dentro de la secuencia. Se representan con flechas con la cabeza vacía. Es posible representar la respuesta a un mensaje con una flecha discontinua.
- Diagrama de colaboración: Es esencialmente un diagrama que muestra interacciones organizadas alrededor de los roles. A diferencia de los diagramas de secuencia, los diagramas de colaboración, también llamados diagramas de comunicación, muestran explícitamente las relaciones de los roles. Por otra parte, un diagrama de comunicación no muestra el tiempo como una dimensión aparte, por lo que resulta necesario etiquetar con números de orden tanto la secuencia de mensajes como los hilos concurrentes.

Este diagrama tiene principalmente dos objetivos:

- * Muestra cómo las instancias específicas de las clases trabajan juntas para conseguir un objetivo común.
- * Implementa las asociaciones del diagrama de clases mediante el paso de mensajes de un objeto a otro. Dicha implementación es llamada “enlace”.

Los diagramas de colaboración muestran la implementación de una operación, la comunicación entre los roles muestra los parámetros y las variables locales de la operación, así como las asociaciones más permanentes. Cuando se implementa el comportamiento, la secuencia de los mensajes corresponde a la estructura de llamadas anidadas y el paso de señales del programa.

Para la realización del modelado UML se ha utilizado la herramienta CASE (*Ingeniería de Software Asistida por Computadora*, acrónimo en inglés) Magic Draw 17 [MAGIC [2011]]. Dicha herramienta es una de las más avanzadas para la realización del modelado UML, soporta todos los diagramas nombrados en esta sección además de otras muchas características de trabajo colaborativo. El uso principal que se le ha dado durante la realización del proyecto ha sido la construcción de los distintos diagramas pertenecientes al modelado UML presentado en la sección (3).

3 Especificación de requisitos

3.1 Introducción

Este estudio fue realizado como prácticas de la asignatura *Ingeniería de requisitos* en 4º curso de Ingeniería en Informática, contó con la colaboración del profesor *Rafael Valencia*, que aceptó que se realizaran las prácticas usando como modelo este prototipo.

La realización de este estudio UML no fue tarea sencilla, dado que es el primer contacto serio que se tiene con UML en la carrera y no se hizo con una práctica preparada para la asignatura, sino con un sistema del que no había ni siquiera un enunciado claro. De hecho, las entrevistas con los clientes se realizaron periódicamente al principio del inicio del proyecto. La obtención de requisitos fue tan amplia que se tuvo que orientar de manera concreta hacia una serie de objetivos para que la realización del sistema fuera viable en el marco de un proyecto fin de carrera.

En la sección (6.2) se describen las vías futuras de desarrollo del proyecto, como es habitual, dada la naturaleza del proyecto, está orientada de manera general y desde una perspectiva de alto nivel. Se ha estimado interesante que durante el desarrollo del estudio realizado en esta sección, haya anotaciones explicativas sobre cuestiones concretas del sistema y posibles ampliaciones que fueron descritas en las entrevistas con los clientes, pero que se han decidido no implementar por exceder ampliamente la carga de trabajo que corresponde a un proyecto fin de carrera.

Una de las motivaciones importantes para la realización del proyecto es la aglutinación de algoritmos realizados de manera independiente, es por ello, que se contempla la realización en el proyecto de unos catálogos de algoritmos donde se pudieran incluir nuevos algoritmos aportados por el usuario. Esta idea no se ha implementado por la dificultad que representa. Aún así, aparece en este estudio, dado que es un objetivo futuro que se espera conseguir. Por lo tanto, el caso de uso “Introducir Algoritmo” queda pendiente de implementación en un futuro.

Otro de los objetivos interesantes del proyecto era la actualización automática de los datos calculados cuando se produjera alguna modificación en los datos de entrada. Se pensó utilizar un patrón Observer para la realización de esta tarea, la explicación concreta de esta idea está explicada en la sección (4.5.2).

El resto de mejoras que fueron pensadas durante el estudio del proyecto serán comentadas en los diagramas correspondientes, para poder realizar una mejor explicación de cada mejora concreta.

El caso de uso “introducir especificaciones” no se ha implementado de forma completa, sino solamente para las especificaciones de estabilidad. Esta restricción ha permitido la implementación y prueba del módulo de fronteras sin haber invertido todo el tiempo necesario para implementar las ventanas de diálogo para el resto de tipos de especificaciones.

Teniendo en cuenta todas las limitaciones mencionadas respecto a lo no implementado, lo que sí se ha implementado en el prototipo descrito en la sección (4) es:

- Módulo de introducción de plantas e incertidumbre (completo).
- Cálculo de plantillas por fuerza bruta (un solo algoritmo de cálculo de plantillas).
- Cálculo de la envolvente de la nube de puntos de plantillas conexas mediante el algoritmo de ϵ -hull [[NORDIN \[1993\]](#)] (un solo algoritmo de cálculo de envolventes).
- Introducción del conjunto de frecuencias de diseño.
- Introducción de especificaciones de estabilidad robusta.
- Cálculo de fronteras de estabilidad mediante el método [[MORENO ET AL. \[2006\]](#)].
- Visualización de fronteras y lazo en diagrama de Nichols y lazo en diagrama de Bode.

3.2 Diagrama conceptual

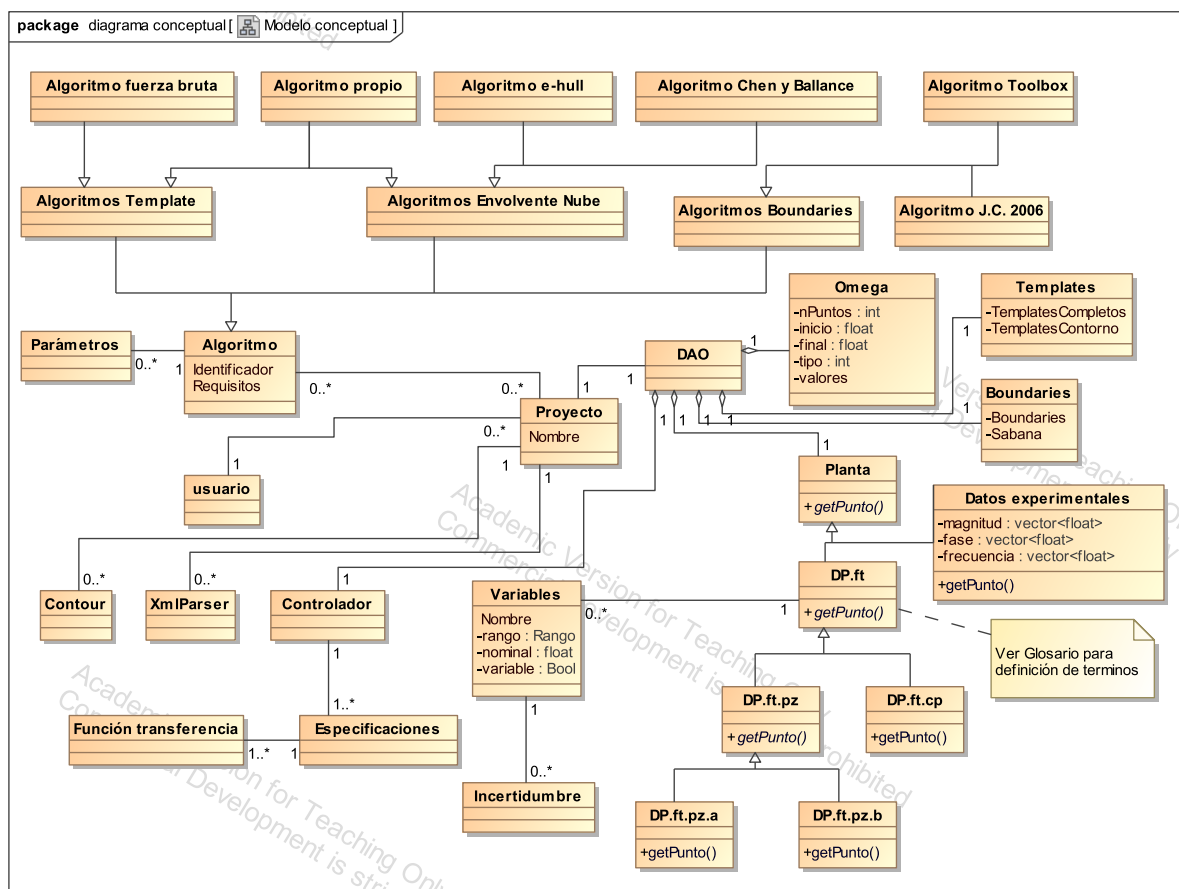


Figura 10: Diagrama conceptual.

3.3 Diagrama de actividad

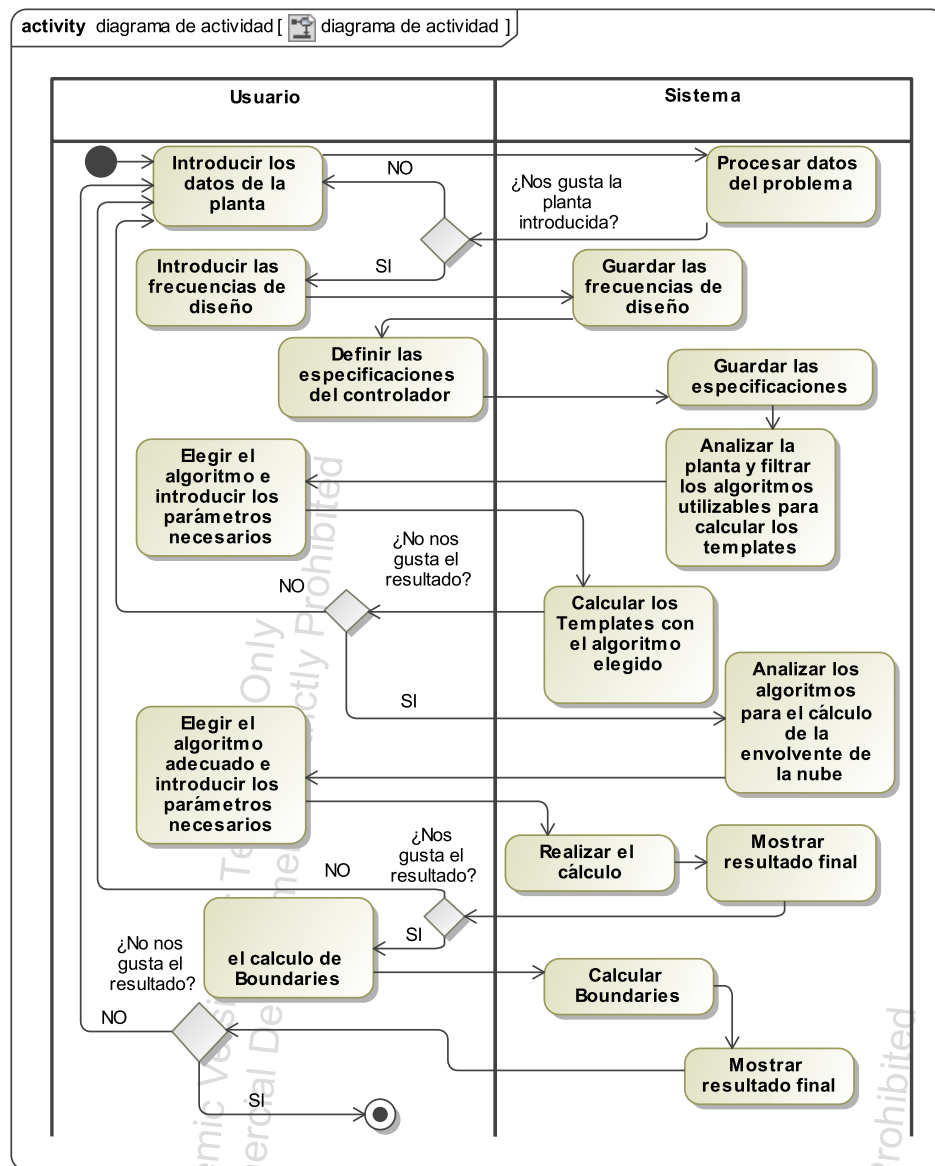


Figura 11: Diagrama de actividad.

Anotación: En este diagrama se puede observar que en este sistema es muy importante poder volver a fases anteriores y poder modificar los datos introducidos. Al ser una herramienta de investigación este hecho se vuelve más importante si cabe, por ello este diagrama de actividad es muy importante para entender la construcción posterior del sistema.

3.4 Glosario

Formas de definir una planta:

- **DP.ft:** Especificación de una función de transferencia junto con su incertidumbre.
 - **DP.ft.pz:** Mediante los polos (p_i), ceros (z_j) y ganancia (k) de la función de transferencia, añadiendo quizás incluso un retardo puro R . Es decir, se almacenarían los reales R y k y dos vectores de números complejos correspondientes a los polos, $P_0 = [p_1 \cdots p_n]$ y ceros $Z_e = z_1 \cdots z_m$, teniendo en cuenta que conviene permitir al usuario introducir los polos y ceros en dos formatos alternativos según k represente o no la ganancia de baja frecuencia:
 - * **DP.ft.pz.a:** Los polos y ceros se representan como términos de la forma $(s+a)$ (primer orden) o $(s^2+2\zeta\omega_n+\omega_n^2)$ (segundo orden) en sus respectivos polinomios denominador o numerador. En ese caso k no es la ganancia de baja frecuencia.
 - * **DP.ft.pz.b:** Los polos y ceros se representan como términos de la forma $(\tau s + 1)$ (primer orden) o $\left(\frac{s^2}{\omega_n^2} + \frac{2\zeta}{\omega_n}\right)$ (segundo orden) en sus respectivos polinomios denominador o numerador. En ese caso k sí es la ganancia de baja frecuencia.
 - **DP.ft.cp:** Mediante los coeficientes de polinomios denominador y numerador, $D = d_1 \cdots d_n$ y $N = n_1 \cdots n_m$.

3.5 Diagrama de casos de uso

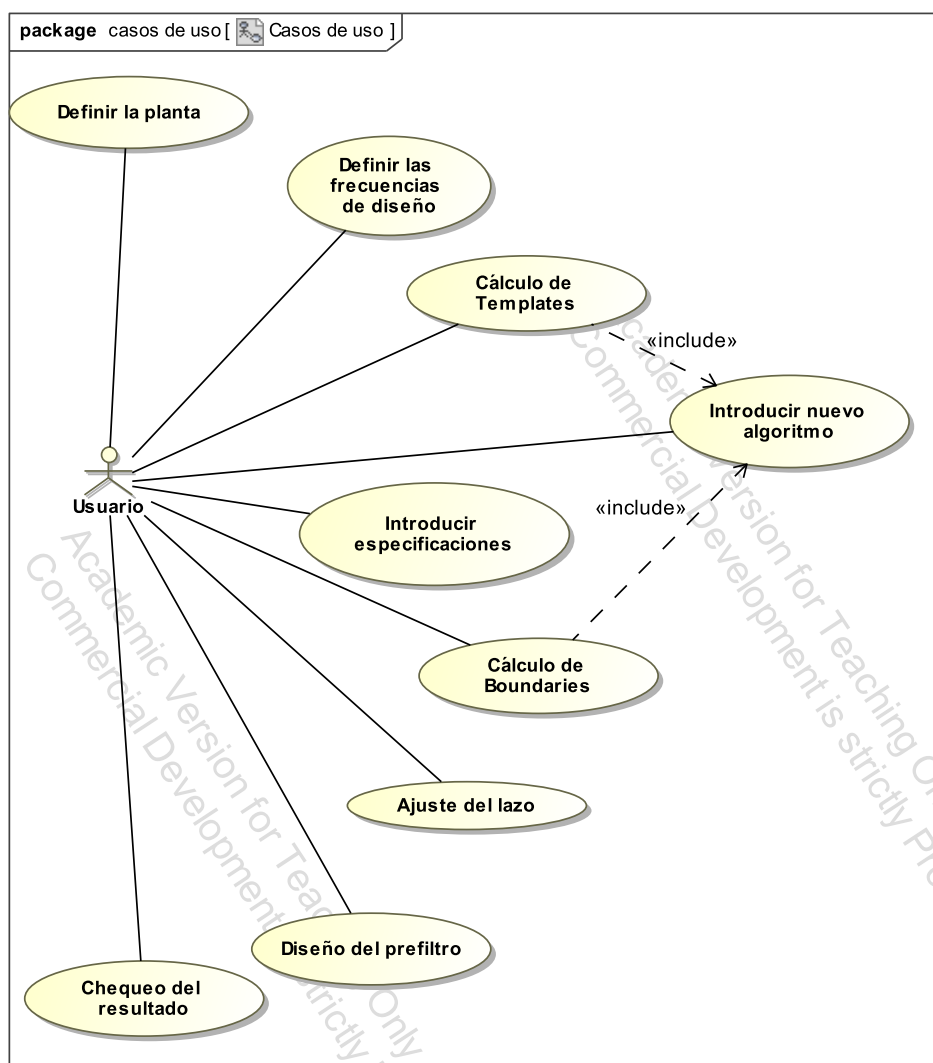


Figura 12: Diagrama de casos de uso.

Explicación de los Casos de Uso

- **Definir el problema:**

- Definir la estructura de la planta, $P(s)$, la planta es la entrada sobre la que queremos resolver el problema.
- Introducir la incertidumbre sobre la planta, ya sea estructurada y/o no.

- **Introducir frecuencias de diseño:** Introducir un vector de frecuencias las cuales vamos a utilizar posteriormente durante todo el proceso, ω .

- **Cálculo de Plantillas:** T es el conjunto de plantillas, uno para cada frecuencia de ω . Se calculan a partir de ω , $P(s)$ y la definición de la incertidumbre sobre $P(s)$. Para cada frecuencia ω_i de ω tenemos T_{ω_i} , una plantilla, que es un conjunto de puntos en el plano complejo que representan las posibles localizaciones de la planta P a esa frecuencia, es decir, $P(j\omega_i)$, que por tener incertidumbre pueden ser varias. Si no hubiera incertidumbre T_{ω_i} tendría un único punto.

- **Introducir Especificaciones:** El usuario introduce al sistema el conjunto de especificaciones a satisfacer por el sistema una vez haya sido controlado. Hay distintos tipos de especificaciones. Las fundamentales son las ecuaciones de la 5 a la 10.

Siempre se trata de que una cierta función de transferencia ha de estar por encima, debajo, o entre uno o dos valores o funciones W_{si} . La función de transferencia de cada especificación hace uso de las funciones de transferencia de la planta, P , el controlador, C , el prefiltro, F , y el sensor, H ⁹. En la figura (13) se pueden observar las posiciones de cada elemento en el esquema típico de control realimentado.

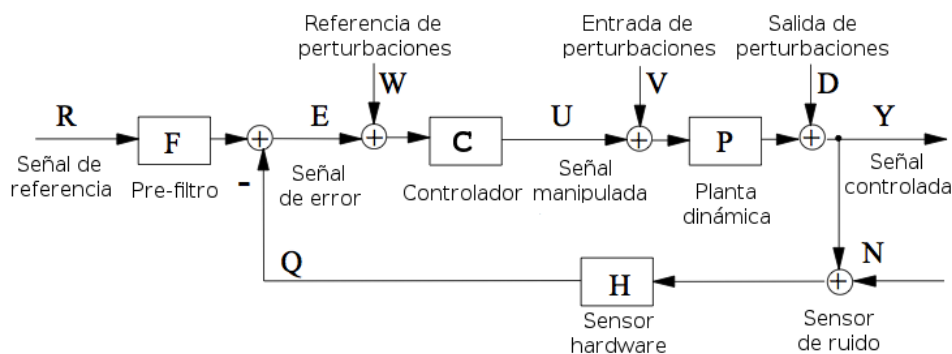


Figura 13: Casos de Uso: Esquema típico de control realimentado.

- **Cálculo de Boundaries:** A partir de las frecuencias de diseño, las plantillas y las especificaciones, se calculan los boundaries (fronteras) que determinan, para cada frecuencia de diseño ω_i , si el nominal del lazo abierto, $L_0(j\omega_i) = P_0(j\omega_i) \cdot C(j\omega_i)$, puede estar dentro/fuera de las regiones delimitadas por la frontera a esa frecuencia.

⁹Es la función de transferencia que representa la diferencia entre la medida real tomada por el sensor, y el valor a la salida, en un sensor ideal no sería necesario indicarla.

- **Resto de casos de uso:** El resto de casos de uso que aparecen en el diagrama corresponden a las fases de QFT explicadas anteriormente, pero no se han contemplado en este estudio dado que salían del ámbito de este proyecto.

3.6 Requisitos

Requisitos funcionales

- RF1: El usuario podrá añadir un nuevo algoritmo al sistema sin tenerlo que asociar a un proyecto.
- RF2: El sistema debe permitir iterar sobre los pasos del proyecto, es decir, volver atrás a modificar datos introducidos anteriormente.
- RF4: Cuando se modifiquen los datos de entrada el sistema, de forma manual o automática, permitirá una reacción en cadena para recalcular todos los datos calculados hasta ese momento.

Requisitos no funcionales

- RNF1: La velocidad de respuesta tiene que ser rápida para que el cambio de pantalla sea fluido.
- RNF2: El sistema debe ser lo más usable posible para usuario no expertos, esto se consigue utilizando valores por defecto donde sea posible y en los casos restantes tendrá indicaciones intuitivas.
- RNF3: El sistema debe de dar la facilidad de ser ampliado donde sea posible dado que es un campo orientado a la investigación y puede sufrir variaciones en un breve espacio de tiempo.

Restricciones de diseño

- RN1: Se usará software libre siempre que sea posible.
- RN2: El sistema será multiplataforma, pudiendo ejecutarse en los SO más usados de las familias Windows, Linux y Mac.

3.7 Definir el problema

Caso de uso

Personal involucrado e intereses:

- Usuario: Definir el problema
- Sistema: Facilitar al usuario definir el problema y guardar los datos para utilizarlos posteriormente.

Precondiciones: Ninguna.

Garantías de éxito (postcondiciones): Que la planta quede correctamente definida, es decir queda definida $P(j\omega)$ para todo ω ¹⁰, que la incertidumbre de la planta quede correctamente definida y que las especificaciones del controlador queden definidas.

Escenario principal de éxito (o Flujo Básico):

1. El usuario inicia un nuevo proyecto
2. El sistema muestra al usuario las distintas formas que tiene de definir una planta.
3. El usuario elige una de las formas mostradas por el sistema (si elige introducir una planta con datos experimentales, ver camino alternativo).
4. El sistema muestra al usuario una estructura para introducir los datos de la planta.
5. El usuario introduce los datos que estime oportunos y pulsa “siguiente”.
6. El sistema abre una ventana adecuada donde introducir la incertidumbre.
7. El sistema mantiene una ventana donde muestra todas las variables, dado que hay parámetros que pueden estar influidos por otros.
8. El usuario elige la variable sobre la que introducir la incertidumbre, si esta variable ya tiene incertidumbre se puede modificar.
9. El sistema analiza y muestra que formas que hay de introducir la incertidumbre para esa variable, poniendo valores por defecto modificables donde sea posible.
10. El sistema muestra las posibilidades disponibles para introducir la incertidumbre.
11. El sistema muestra al usuario una estructura para introducir la incertidumbre de la planta.
12. El usuario introduce los datos que estime oportunos y acepta.
13. El sistema guarda los datos.
14. Se vuelve al paso 8 y se repite con todas las variables.

¹⁰En el caso DP.de, para todo ω del que se tenga información tras la identificación.

Extensiones (o Flujos Alternativos):

3-5a. El usuario puede volver atrás y elegir otro modo de introducir la planta.

3-5b. El usuario elige definir una planta con datos experimentales, lo cual se representa con un conjunto de vectores, lo más sencillo es indicar el fichero donde estén estos vectores.

1. El usuario elige introducir una planta con datos experimentales.
2. El sistema da la opción al usuario de indicar un fichero de texto.
3. El usuario indica qué fichero es el que contiene los datos y acepta.
4. El sistema analiza los datos de entrada y muestra los diagramas de Bode resultantes dando la posibilidad al usuario de modificar, añadir o eliminar puntos de los diagramas.
5. El usuario modifica, si lo cree necesario, los diagramas de Bode y guarda los datos resultantes.

1-14a. El usuario puede cancelar el proyecto.

Diagrama de secuencia del sistema

Diagrama de secuencia del camino principal:

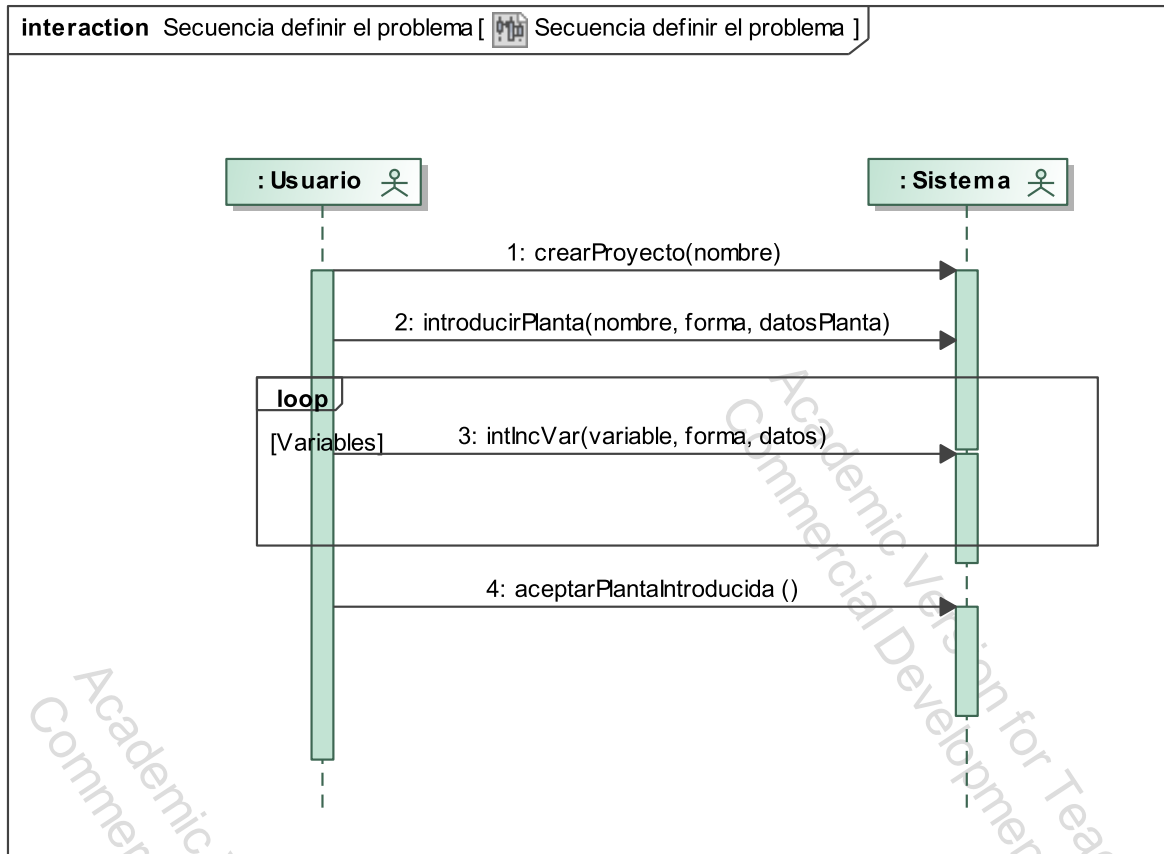


Figura 14: Diagrama de secuencia del sistema: Definir el problema (camino principal).

Diagrama de secuencia de camino alternativo, usando datos experimentales:

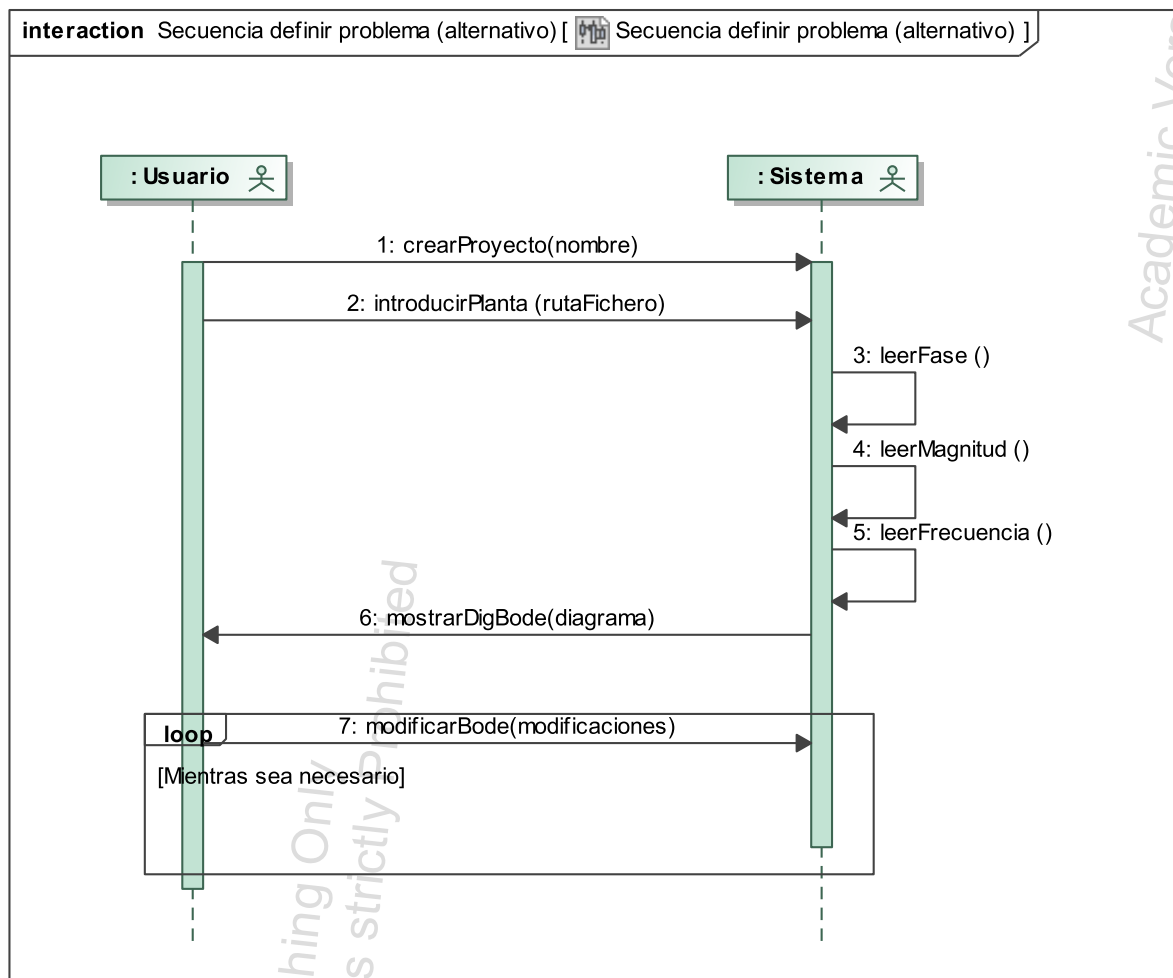


Figura 15: Diagrama de secuencia del sistema: Definir el problema usando datos experimentales.

Contratos y colaboraciones

Crear Proyecto:

Operación: crearProyecto(String: Nombre)

Referencias: Casos de uso: Definir el problema.

Controlador: Controlador Proyecto

Precondiciones:

Postcondiciones:

- Se creó una instancia “p” de proyecto.
- Se creo una instancia “sistemaDAO” de la clase DAO y se asigno a “p”.

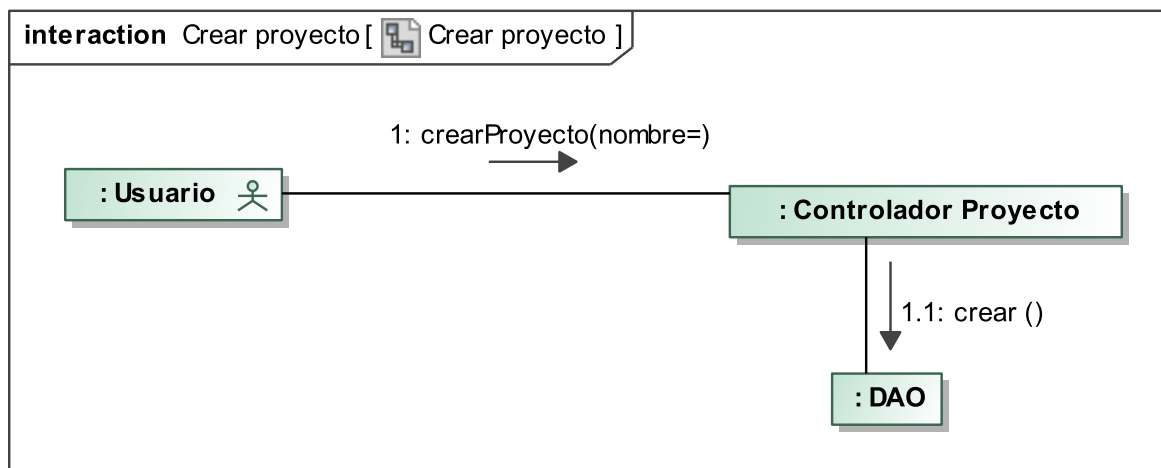


Figura 16: Colaboraciones: Crear Proyecto.

Anotación: Durante el estudio inicial, se planteo la posibilidad de poder tener varios proyectos a la vez abiertos. Cuestión muy interesante por la orientación investigadora que tiene el sistema, de este modo se podrían hacer pruebas paralelas y comparar resultados. Su realización no es muy complicada, al igual que existe un controlador de proyecto, se podría implementar un controlador de programa que manejara diferentes controladores de proyecto. Estos sería completamente independientes, con lo cual se podrían tener varios proyectos abiertos que no tuvieran relación entre si. Por lo tanto queda dispuesto como vía futura de desarrollo.

Introducir Planta:

Operación: introducirPlanta(String: Nombre, Int : Forma, Datos)

Referencias: Casos de uso: Definir el problema

Controlador: Proyecto

Precondiciones: Existe un proyecto “p” en curso al cual asignar la planta.

Postcondiciones:

- Se creó una instancia “adaptadorPlanta” de AdaptadorPlantaDAO y se asoció a “p”.
- Se introduce al proyecto “p” una planta “pa” ya creada que se asocia a “adaptadorPlanta”.

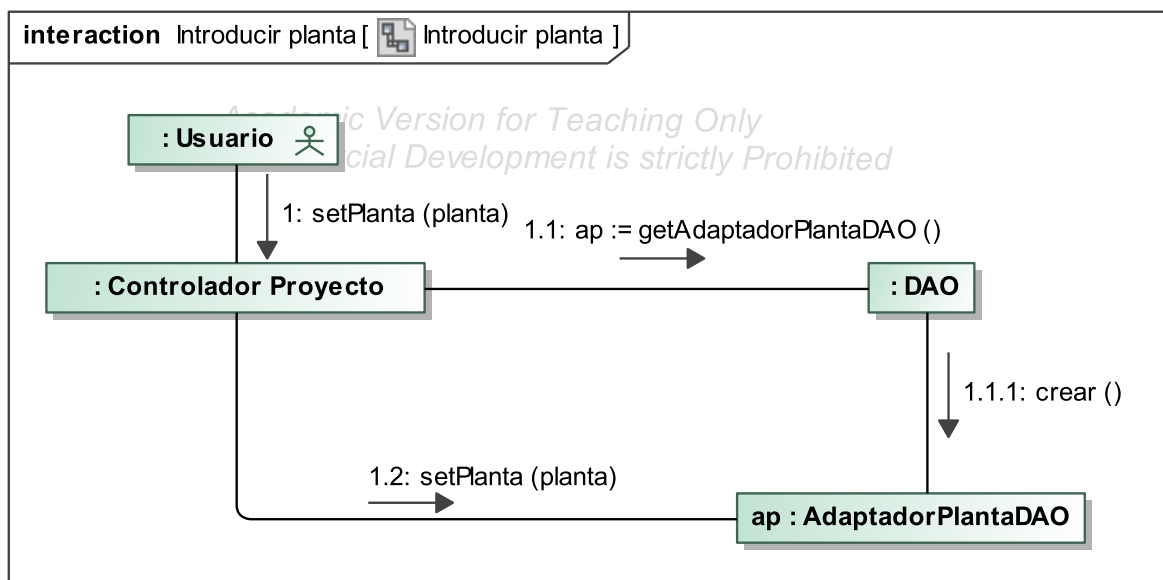


Figura 17: Colaboraciones: Introducir Planta.

Anotación: En este punto se planteó la posibilidad de que en un mismo proyecto existieran varias plantas con las que poder trabajar. Queda dispuesto como vía futura de desarrollo.

Modificar Diagrama de Bode:

Operación: modificarBode(nuevoBode)

Referencias: Casos de Uso: Definir el problema

Controlador: Controlador Proyecto

Precondiciones: Tener una planta “pa” en curso.

Postcondiciones:

- La planta quedará completamente definida para avanzar al siguiente paso.

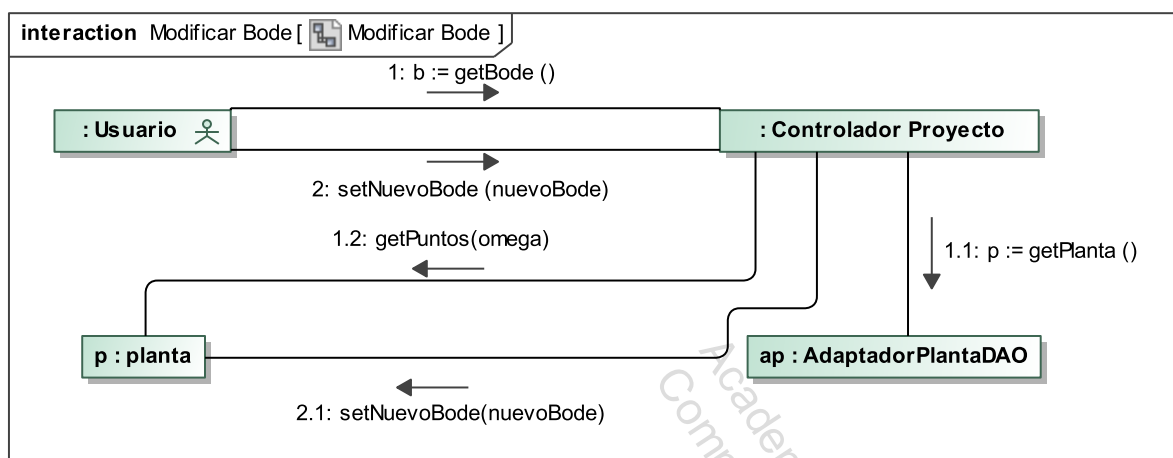


Figura 18: Colaboraciones: Modificar diagrama de Bode.

3.8 Introducir frecuencias de diseño

Caso de uso

Personal involucrado e intereses:

- Usuario: Introducir las frecuencias de diseño que quiere utilizar durante el proceso.
- Sistema: Guardar las frecuencias de diseño.

Precondiciones: Ninguna.

Garantías de éxito (postcondiciones): Las frecuencias de diseño queda definidas en el sistema.

Escenario principal de éxito (o Flujo Básico):

1. El sistema ofrece al usuario distintas formas de introducir las frecuencias de diseño.
2. El usuario elije una forma e introduce los datos necesarios.
3. El sistema lee los datos introducidos por el usuario, valida que sean correctos y los transforma a vector si fuese necesario.
4. El sistema guarda el vector de frecuencias de diseño.

Extensiones (o Flujos Alternativos):

- 1-4a: el Usuario puede cancelar el proceso si lo estima conveniente.
- 3a: Si el sistema detecta errores en los datos vuelve al paso 1.

Diagrama de secuencia del sistema

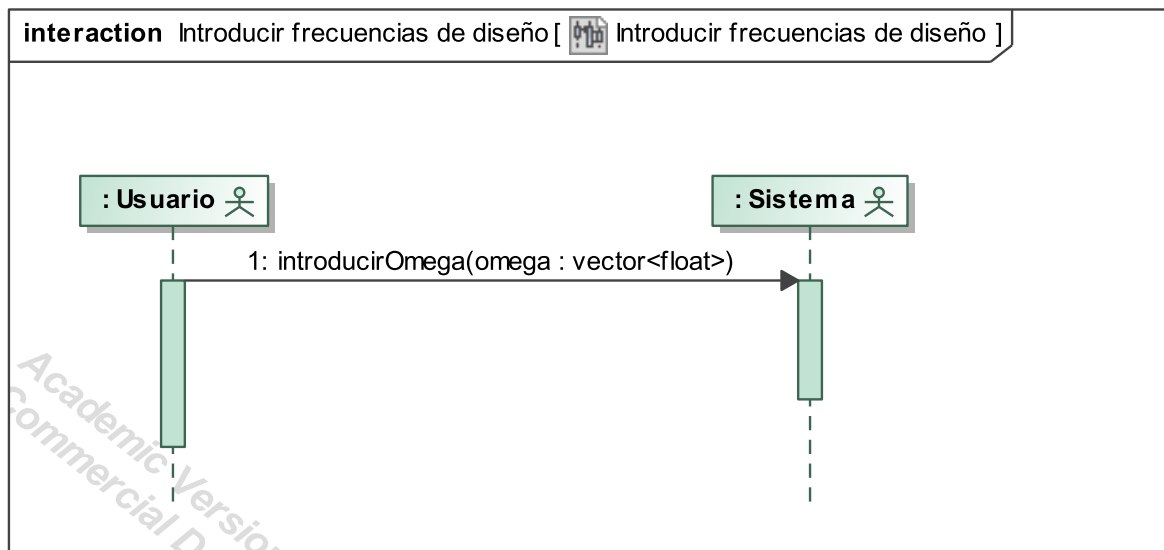


Figura 19: Diagrama de secuencia del sistema: introducir frecuencias de diseño.

Contratos y colaboraciones

Introducir las frecuencias de diseño en el sistema:

Operación: introducirOmega (omega : vector)

Referencias: Casos de Uso: Introducir frecuencias de diseño.

Controlador: Controlador Proyecto

Precondiciones: Ninguna

Postcondiciones:

- Las frecuencias de diseño quedan guardadas en el sistema.

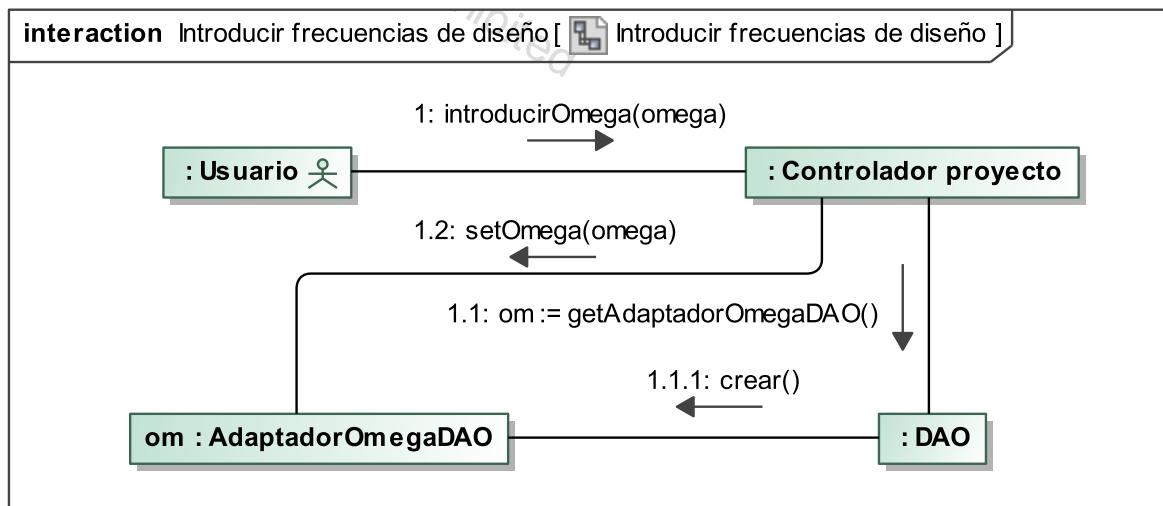


Figura 20: Colaboraciones: Introducir las frecuencias de diseño en el sistema.

3.9 Cálculo de plantillas

Caso de uso

Personal involucrado e intereses:

- Usuario: Elegir el algoritmo más adecuado para la planta, ya sea de las opciones que ha dado el sistema o introduciendo uno particular y elegir el algoritmo más adecuado para calcular la envolvente de la nube de puntos de las plantillas.
- Sistema: Analizar la planta y ofrecer al usuario una lista de algoritmos adecuados, ejecutar el algoritmo elegido, analizar las plantillas resultantes y hacer una lista con los posibles algoritmos para calcular la envolvente de la nube para que el usuario elija, a continuación calcular la envolvente de la nube con el algoritmo elegido.

Precondiciones: Tener una planta correctamente definida y unas frecuencias de diseño elegidas.

Garantías de éxito (postcondiciones):

- Presentar al usuario una lista adecuada de algoritmos para calcular las plantillas.
- Calcular las plantillas correctamente con la elección del usuario.
- Presentar al usuario una lista adecuada de algoritmos para calcular la envolvente de la nube.
- Calcular la envolvente de la nube correctamente con el algoritmo elegido por el usuario.

Escenario principal de éxito (o Flujo Básico):

1. El sistema analiza la planta y de los algoritmos que tiene disponibles en su catálogo selecciona una lista con los posibles para el tipo concreto de planta. El sistema siempre seleccionara uno como mínimo, que es el de fuerza bruta¹¹.
2. El usuario elige uno de los algoritmos que de la lista que le ha proporcionado el sistema.
3. El sistema pide al usuario los parámetros necesarios para ejecutar el algoritmo, poniendo valores por defecto modificables para todos los parámetros posibles.
4. El usuario introduce los datos que estime oportunos completando los indispensables.
5. El sistema valida que todos los parámetros sean correctos.
6. El sistema ejecuta el algoritmo sobre la planta para calcular las plantillas.
7. El sistema muestra al usuario las plantillas calculadas.
8. El sistema analiza las plantillas y filtra una lista de algoritmos posibles para calcular la envolvente de la nube de puntos.
9. El usuario elige uno de los algoritmos.

¹¹ Algoritmo de fuerza bruta: Es aquel que basa su resolución en probar todas las posibles combinaciones.

10. El sistema pide al usuario los parámetros necesarios para ejecutar el algoritmo, poniendo valores por defecto modificables para todos los parámetros posibles.
11. El usuario introduce los datos que estime oportunos completando los indispensables.
12. El sistema valida que todos los parámetros sean correctos.
13. El sistema ejecuta el algoritmo sobre las plantillas para calcular la envolvente de la nube de puntos.
14. El sistema muestra al usuario la envolvente de la nube de puntos calculada.

Extensiones (o Flujos Alternativos):

- 1-14a. El usuario puede cancelar el proyecto.
- 2a. El usuario puede introducir su propio algoritmo para calcular las plantillas, para ello ejecuta el caso de uso “Introducir Algoritmo”.
- 5a. Si los parámetros no son válidos el sistema vuelve a paso 3.
- 7a. Si el resultado no es satisfactorio el usuario puede volver al paso 2 y empezar de nuevo.
- 7b. Si el resultado no es satisfactorio el usuario puede volver al paso 4 y cambiar los valores de los parámetros si los hubiera.
- 7c. Si el resultado no es satisfactorio el usuario puede volver a introducir la planta.
- 7d. Si el resultado no es satisfactorio el usuario puede volver a introducir incertidumbre para modificar la incertidumbre de la planta.
- 12a. Si los parámetros no son válidos el sistema vuelve al paso 10.
- 14a. Si el resultado no es satisfactorio el usuario puede volver al paso 8 y empezar de nuevo.

Diagrama de secuencia del sistema

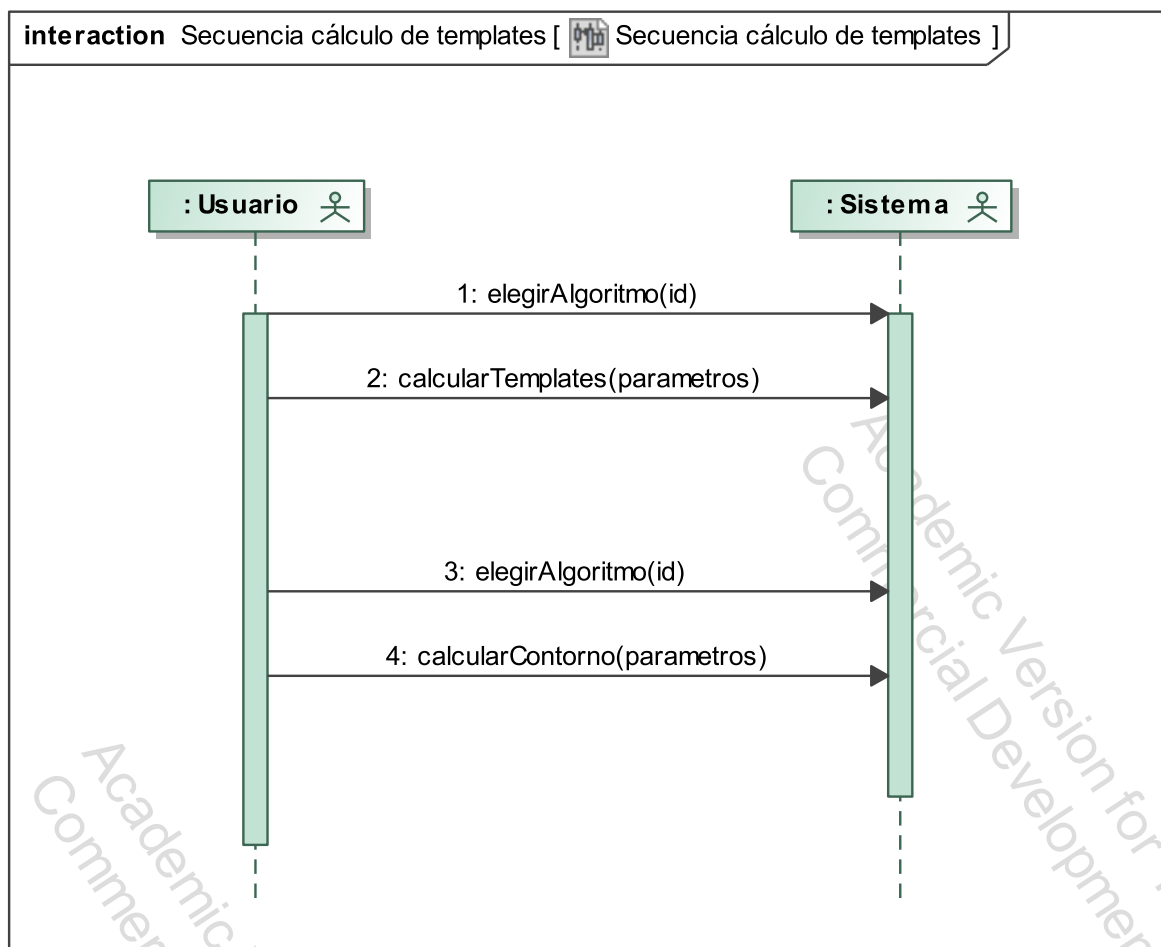


Figura 21: Diagrama de secuencia del sistema: cálculo de plantillas.

Contratos y colaboraciones

Elegir algoritmo para cálculo de plantillas:

Operación: elegirAlgoritmoTem(String: Id)

Referencias: Casos de Uso: Cálculo de Plantillas.

Controlador: Controlador Proyecto

Precondiciones: Tener un proyecto “p” y una planta “pa” en curso para calcularle las plantillas.

Postcondiciones:

- El Controlador recupera el algoritmo deseado del Catálogo de Algoritmos y lo pone como predeterminado.

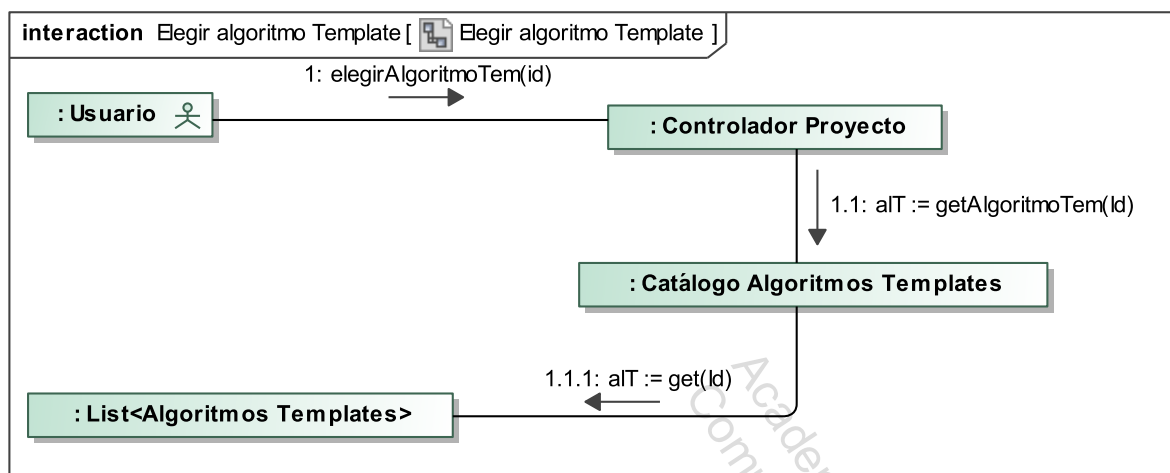


Figura 22: Colaboraciones: Elegir Algoritmo para el cálculo de plantillas.

Ejecutar el algoritmo de cálculo de plantillas introduciendo para ello los parámetros que sean necesarios.:

Operación: ejecutarAlgTem(Param)

Referencias: Casos de Uso: Cálculo de Plantillas.

Controlador: Controlador Proyecto

Precondiciones: Tener un proyecto “p”, una planta “pa” en curso, unas frecuencias de diseño para calcularle las plantillas y un algoritmo preparado.

Postcondiciones:

- El proyecto “p” ejecuta el algoritmo predeterminado sobre la planta y guarda los resultados en AdaptadorTemplateDAO.

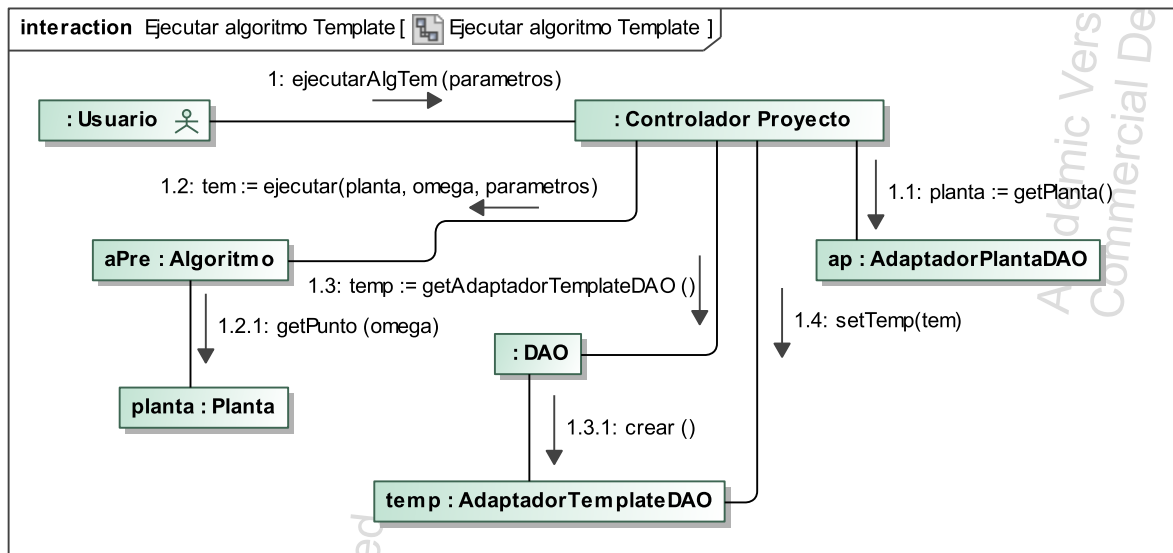


Figura 23: Colaboraciones: Ejecutar algoritmo para calcular las plantillas.

Elegir algoritmo para cálculo de la envolvente de la nube:

Operación: elegirAlgoritmoNube(String: Nombre)

Referencias: Casos de Uso: Cálculo de Plantillas

Controlador: Controlador Proyecto

Precondiciones: Tener una planta “pa” en curso con las plantillas calculadas.

Postcondiciones:

- El Controlador recupera el algoritmo deseado del Catálogo de Algoritmos y lo pone como predeterminado.

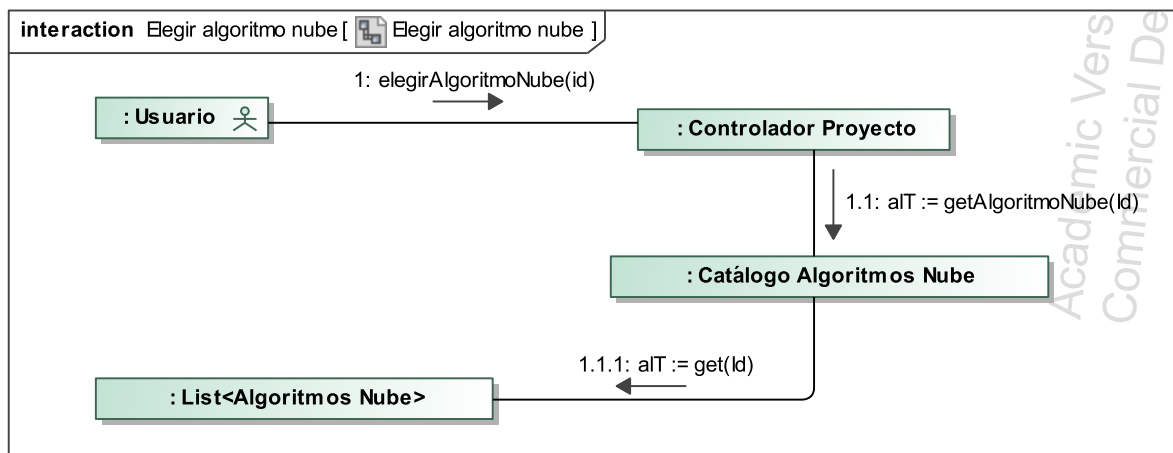


Figura 24: Colaboraciones: Elegir algoritmo para el cálculo de la envolvente de la nube.

Ejecutar el algoritmo para calcular la envolvente de la nube introduciendo para ello los parámetros necesarios

Operación: ejecutarAlgNube(param)

Referencias: Casos de Uso: Cálculo de Plantillas

Controlador: Controlador Proyecto

Precondiciones: Tener un proyecto “p” y una planta “pa” en curso para calcularle la envolvente de la nube sobre las plantillas calculados anteriormente y un algoritmo preparado.

Postcondiciones:

- El proyecto “p” ejecuta el algoritmo sobre la planta y guarda los resultados en AdaptadorTemplateDAO.

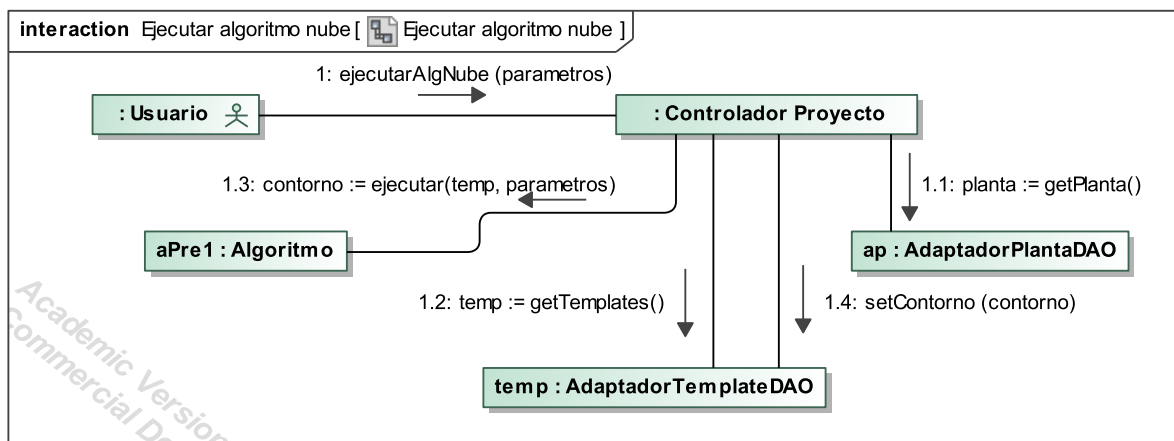


Figura 25: Colaboraciones: Ejecutar el algoritmo de cálculo de la envolvente de la nube

3.10 Introducir especificaciones

Caso de uso

Personal involucrado e intereses:

- Usuario: Introducir las especificaciones correctamente de una de las formas posibles.
- Sistema: Facilitar al usuario introducir las especificaciones, analizarlas y comprobar que son correctas.

Precondiciones: Tener una planta “pa” sobre la que introducir las especificaciones del controlador.

Garantías de éxito (postcondiciones): Las especificaciones quedan definidas en el sistema.

Escenario principal de éxito (o Flujo Básico):

1. El usuario indica que quiere introducir las especificaciones del controlador.
2. El sistema muestra al usuario las distintas posibilidades para introducir las especificaciones del controlador ¹².
3. El usuario elige una de ellas e introduce los datos mínimos necesarios ¹³.
4. El sistema restringirá las decisiones que no tengan sentido a la hora de introducir las especificaciones.
5. Una vez que el usuario ha introducido las especificaciones el sistema las guarda.
6. Se vuelve al paso 1 y se repite hasta introducir todo el conjunto de especificaciones que el usuario crea conveniente, cada una de las especificaciones son independientes.

Extensiones (o Flujos Alternativos):

- 1-6a. El usuario puede cancelar y no introducir las especificaciones.
2. El sistema da la opción de introducir un tipo de especificación genérica donde el usuario pueda combinar los elementos F , C , P y H ¹² como quiera. Esta especificación nueva puede ser guardada en el sistema como una opción más.

¹²Ver ecuaciones de la 5 a la 10.

¹³Para los tipos 2-6, una constante o función de transferencia, correspondientes a ω_{si} , así como un rango de frecuencias ω_{min} y ω_{max} (alguno podría ser $+/ - \infty$) entre los cuales la restricción establecida por la especificación se aplica, para el tipo 1, en vez de una, dos funciones de transferencia.

Diagrama de secuencia del sistema

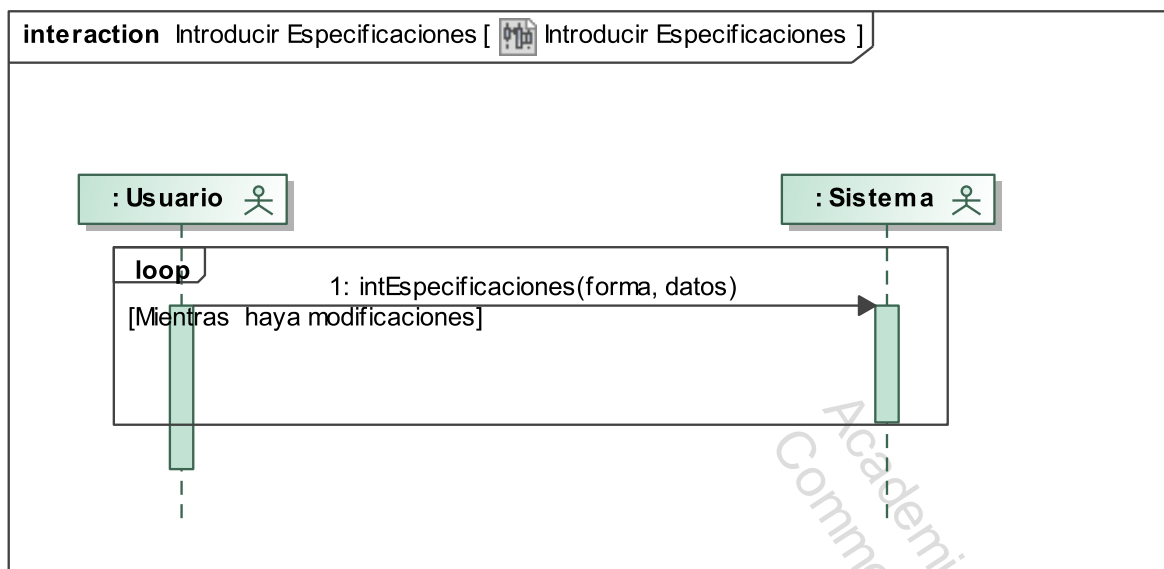


Figura 26: Diagrama de secuencia del sistema: introducir especificaciones del controlador.

Contratos y colaboraciones

Introducir especificaciones del controlador:

Operación: introducirEspecificaciones(forma, datos)

Referencias: Casos de Uso: Introducir Especificaciones del Controlador.

Controlador: Controlador Proyecto

Precondiciones: Tener un proyecto “p” sobre el que introducir las especificaciones del controlador.

Postcondiciones:

- Las especificaciones del controlador o controladores quedan definidas en el sistema.

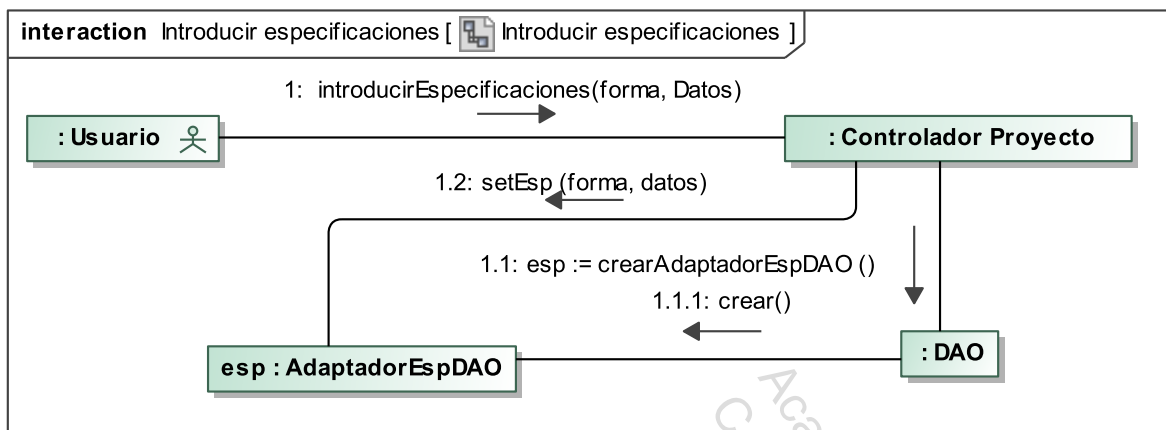


Figura 27: Colaboraciones: Introducir especificaciones del controlador.

3.11 Introducir nuevo algoritmo

Caso de uso

Personal involucrado e intereses:

- Usuario: Introducir un nuevo algoritmo que quede guardado en el sistema para su posterior uso.
- Sistema: Guardar el algoritmo que le introduzca el usuario correctamente.

Precondiciones: nada

Garantías de éxito (postcondiciones): Que un nuevo algoritmo quede guardado en el sistema.

Escenario principal de éxito (o Flujo Básico):

1. El usuario indica que quiere introducir un nuevo algoritmo.
2. El sistema le da a elegir qué tipo de algoritmo quiere introducir¹⁴.
3. El usuario elige qué tipo de algoritmo quiere introducir e indica donde está guardado el algoritmo.
4. El sistema registra el algoritmo y lo guarda en su lugar correspondiente dependiendo del tipo que sea.

Extensiones (o Flujos Alternativos):

- 1-4a. El usuario puede cancelar y no introducir ningún algoritmo.

¹⁴Para calcular las plantillas, la envolvente de la nube de puntos, las fronteras...

Diagrama de secuencia del sistema

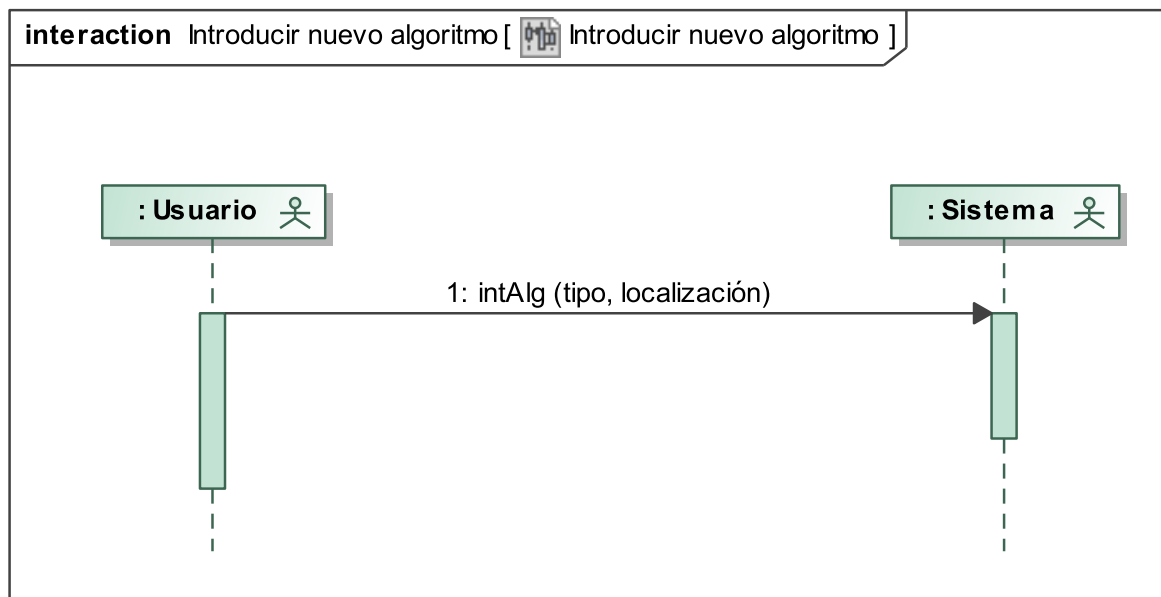


Figura 28: Diagrama de secuencia del sistema: introducir nuevo algoritmo.

Contratos y colaboraciones

Introducir nuevo algoritmo:

Operación: `intNuevoAlg(tipo, String: localizacion)`

Referencias: Casos de Uso: Introducir nuevo algoritmo

Controlador: Controlador Programa

Precondiciones: nada.

Postcondiciones:

- El Controlador del Programa crea una instancia de algoritmo a partir de los parámetros de entrada y lo guarda en el Catálogo de Algoritmos.

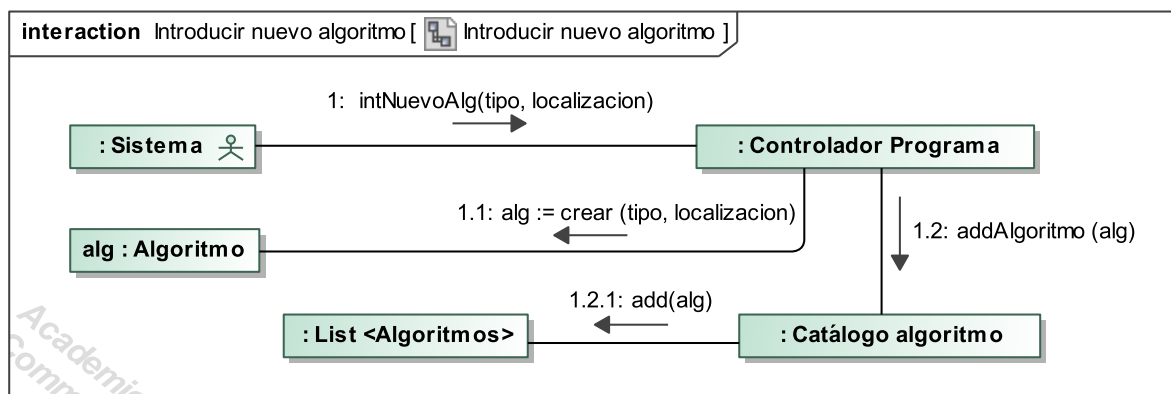


Figura 29: Colaboraciones: Introducir nuevo algoritmo.

3.12 Cálculo de boundaries

Caso de uso

Personal involucrado e intereses:

- Usuario: Elegir el algoritmo que se considere, ya sea de las opciones que ha dado el sistema o introduciendo uno particular.
- Sistema: Aplicar el algoritmo elegido.

Precondiciones: Tener una planta correctamente definida, las frecuencias de diseño elegidas y las plantillas calculadas.

Garantías de éxito (postcondiciones): El algoritmo ha terminado correctamente y se han obtenido las fronteras.

Escenario principal de éxito (o Flujo Básico):

1. El usuario elige un algoritmo de los disponibles en el sistema.
2. El sistema pide al usuario los parámetros necesarios para ejecutar el algoritmo, poniendo valores por defecto modificables para todos los parámetros posibles.
3. El usuario introduce los datos que estime oportunos completando los indispensables.
4. El sistema valida que todos los parámetros sean correctos.
5. El sistema ejecuta el algoritmo sobre la planta para calcular los boundaries.
6. El sistema muestra al usuario los boundaries calculados.

Extensiones (o Flujos Alternativos):

- 1-6a. El usuario puede cancelar el proyecto.
- 1a. El usuario puede introducir su propio algoritmo para calcular los boundaries, para ello ejecuta el caso de uso “Introducir Algoritmo”.
- 4a. Si los parámetros no son válidos el sistema vuelve a paso 3.
- 6a. Si el resultado no es satisfactorio el usuario puede volver al paso 1 y empezar de nuevo.

Diagrama de secuencia del sistema

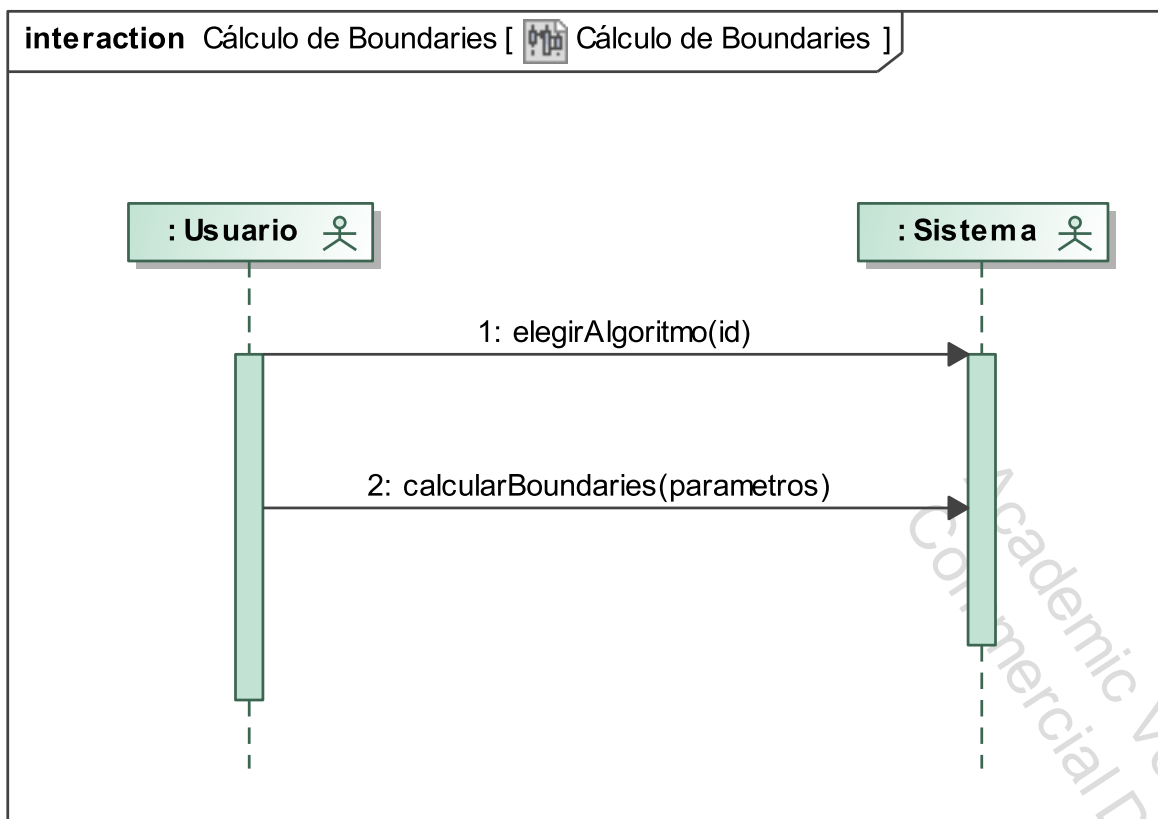


Figura 30: Diagrama de secuencia del sistema: Cálculo de fronteras.

Contratos y colaboraciones

Elegir algoritmo para cálculo de Boundaries:

Operación: elegirAlgoritmoBoun(String: Id)

Referencias: Casos de Uso: Cálculo de Boundaries.

Controlador: Controlador Proyecto

Precondiciones: Tener un proyecto “p”, una planta “pa” en curso y las plantillas calculadas para calcularle los Boundaries.

Postcondiciones:

- El Controlador recupera el algoritmo deseado del Catálogo de Algoritmo y lo pone como predeterminado.

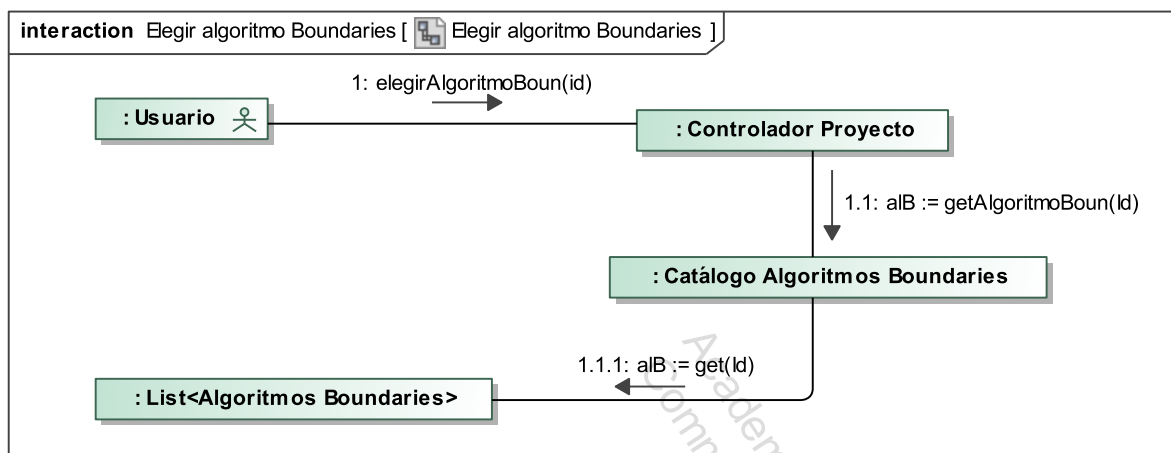


Figura 31: Colaboraciones: Elegir algoritmo para el Cálculo de fronteras.

Ejecutar el algoritmo de cálculo de Boundaries introduciendo para ello los parámetros necesarios:

Operación: ejecutarAlgBound(param)

Referencias: Casos de Uso: Cálculo de Boundaries.

Controlador: Controlador Proyecto

Precondiciones: Tener un proyecto “p”, una planta “pa” en curso, las plantillas calculadas y un algoritmo preparado para el cálculo de boundaries.

Postcondiciones:

- El proyecto “p” ejecuta el algoritmo predeterminado y guarda los resultados en AdaptadorBoundariesDAO.

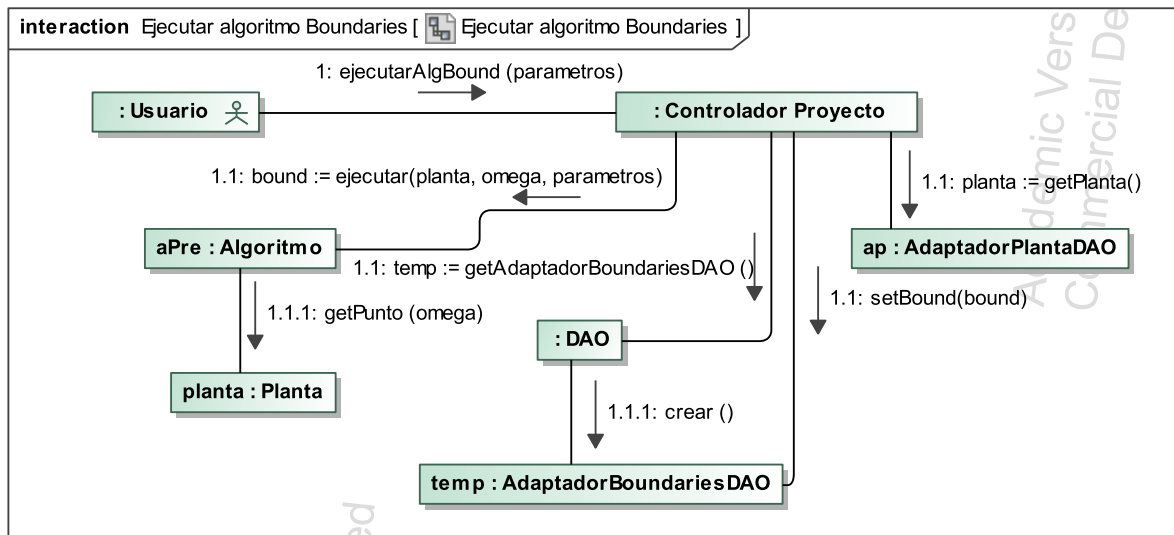


Figura 32: Colaboraciones: Ejecutar el algoritmo para calcular los boundaries.

4 Diseño del prototipo

La parte principal del proyecto consistió en el desarrollo de un prototipo inicial que implementara el modelo descrito en la sección (3). Este prototipo se ha construido centrado en conseguir una serie de mínimos que lo hicieran funcional hasta la etapa del *cálculo de fronteras*, es decir, durante su construcción se han dejado de lado las partes del sistema más superficiales, como por ejemplo la robustez frente a las entradas del usuario. En la sección (5) se harán una serie de pruebas donde se podrá observar el funcionamiento del prototipo.

Antes de comenzar el desarrollo del prototipo se tomaron una serie de decisiones relacionadas con los requisitos de diseño nombradas en la sección (3), dos de los requisitos más importantes son los siguientes:

- La velocidad de respuesta tiene que ser rápida para que el cambio de pantalla sea fluido.
- El sistema debe ser lo más usable posible para usuario no expertos, esto se consigue utilizando valores por defecto donde sea posible y en los casos restantes el software incluirá indicaciones intuitivas.
- El sistema será multiplataforma, pudiendo ejecutarse en los SO más usados de las familias Windows, Linux y Mac.

Ante estos requisitos la elección del lenguaje de programación se convierte en una tarea fundamental, a continuación en la sección (4.1) se detalla el estudio donde se justifican las decisiones tomadas.

4.1 Estudio de alternativas

El estudio de alternativas para este prototipo se basa principalmente en la elección del lenguaje de programación para realizar el proyecto. Refiriéndose a esta cuestión encontramos los requisitos nombrados en la sección (4).

Para hacer una selección pequeña de lenguajes de programación posibles para realizar este proyecto, se pusieron unas condiciones básicas que deben cumplir para ser estudiados. En primer lugar, el cliente establece como requisito que el prototipo sea multiplataforma, con lo que se reducen notablemente las opciones disponibles.

Como segunda condición es que el lenguaje debe ser orientado a objetos, dado que este tipo de lenguajes nos ofrecen una serie de ventajas que son muy favorables para el desarrollo presente y futuro del proyecto, así como su mantenimiento. Entre las ventajas a valorar está la modularidad absoluta que ofrece la Orientación a Objetos que permite por ejemplo una ampliación futura del proyecto si necesidad de modificar el código ya escrito. Cuestión muy importante que se nombro en la sección (4). La siguiente condición sería la del rendimiento, pero está no es fácil de resolver, dado que no existe un lenguaje mejor en todo que se pueda elegir con absoluta seguridad.

Una vez definidas las condiciones básicas se han seleccionado 3 lenguajes de programación que cumplen dichos requisitos y que además son los más usados, por supuesto deben ser de licencia libre. En el índice *TIOBE*¹⁵ los lenguajes de programación más usados se pueden observar en la figura (34).

Position Aug 2013	Position Aug 2012	Delta in Position	Programming Language	Ratings Aug 2013	Delta Aug 2012	Status
1	2	↑	Java	15.978%	-0.37%	A
2	1	↓	C	15.974%	-2.96%	A
3	4	↑	C++	9.371%	+0.04%	A
4	3	↓	Objective-C	8.082%	-1.46%	A
5	6	↑	PHP	6.694%	+1.17%	A
6	5	↓	C#	6.117%	-0.47%	A
7	7	=	(Visual) Basic	3.873%	-1.46%	A
8	8	=	Python	3.603%	-0.27%	A

Figura 34: Tabla con los lenguajes de programación más usados.

Como se puede observar, *Java* y *C++* son los dos lenguajes de programación orientados a objetos más usados, en cambio *python* se situa en la posición 8. Pero los lenguajes que le superan no cumplen los requisitos mencionados:

- *(Visual) Basic*: No es orientado a objetos, sino a eventos y su licencia no es libre.
- *C#*: Su licencia no es libre, al igual que el anterior, es propietaria de *Microsoft*.
- *PHP*: Esta diseñado originalmente diseñado para del desarrollo web, aunque se ha expandido mucho, fuera del desarrollo web su utilización es excasa, debido a que es interpretado por un servidor web.
- *Objective-C*: Su uso está limitado principalmente *Mac OS X*, *iOS* y *GNUstep* y no es multiplataforma.

Teniendo en cuenta el análisis anterior los tres lenguajes de programación elegidos que cumplen los requisitos mencionados son los siguientes:

- ***Python***: Lenguaje de programación interpretado, multiparadigma, con tipado dinámico y multiplataforma.
- ***Java***: Lenguaje de programación interpretado, orientado a objetos y multiplataforma.
- ***Qt/C++***:
 - ***C++***: Lenguaje de programación multiparadigma, tipado estático, facilidades de bajo nivel, soporta *C* y no es multiplataforma.

¹⁵<http://www.tiobe.com/index.php/content/paperinfo/tpci/index.html>

- **Qt:** Biblioteca multiplataforma para desarrollo gráfico basada en *C++* y que aporta multiplataforma a éste.

Estos son los tres lenguajes posibles para realizar el proyecto, los dos primeros al ser interpretados son multiplataforma pero dependen de una máquina virtual para funcionar. En cambio *C++*, si se hace un uso correcto de las librerías de *Qt*, es multiplataforma y no necesita máquina virtual para funcionar.

4.1.1 Análisis con benchmark

Para analizar el rendimiento de estos lenguajes se utilizaron una serie de benchmark o banco de pruebas [DEBIAN [2013]].

Esta página nos ofrece una serie de programas de pruebas conocidos como benchmark que nos permiten medir el rendimiento, consumo de memoria, etc de distintos tipos de algoritmos sobre muchos lenguajes de programación, entre ellos los tres que seleccionados para el proyecto. Ese primer análisis se centra exclusivamente en el lenguaje de programación sin hacer uso de una interfaz gráfica, el uso de esta se deja para un análisis posterior.

En primer lugar se analiza el tiempo de ejecución, el gráfico de caja de la figura (35) es una vista general de los lenguajes más rápidos teniendo en cuenta los 10 benchmark de los que dispone la página web. Obviamente al ser con todas las pruebas y sin tener en cuenta que uso del lenguaje va a hacer nuestro sistema, no se puede tomar ninguna decisión en base a este gráfico, solo es una visión general. Para todas las pruebas se va a utilizar una máquina con un procesador Intel Q6600 de un core con un sistema operativo Ubuntu x86.

Una vez mostrado esta primera visión general, es conveniente seleccionar una serie de pruebas que se asemejen lo máximo posible a los algoritmos implementados en el prototipo. Por lo tanto hay que analizar que tipo de uso de la máquina va a hacer el prototipo implementado:

- Uso intensivo de CPU en el cálculo con números complejos, dichos números complejos son en punto flotante, con el coste computacional que eso conlleva.
- Uso de estructuras de almacenamiento de datos utilizadas para recuperar y guardar los datos necesarios para la ejecución de los algoritmos.

Ante este uso por parte del prototipo de la máquina, los algoritmos benchmark seleccionados son los siguientes:

Reverse-complement: Lee secuencias de ADN y escribe la inversa de su complemento. Este algoritmo hacer un uso intensivo de la CPU con cálculos con números reales, así como también hace accesos a estructuras de datos para lectura y escritura.

Para mostrar los resultados se muestra el lenguaje más rápido y a continuación los tres lenguajes seleccionados para el proyecto:

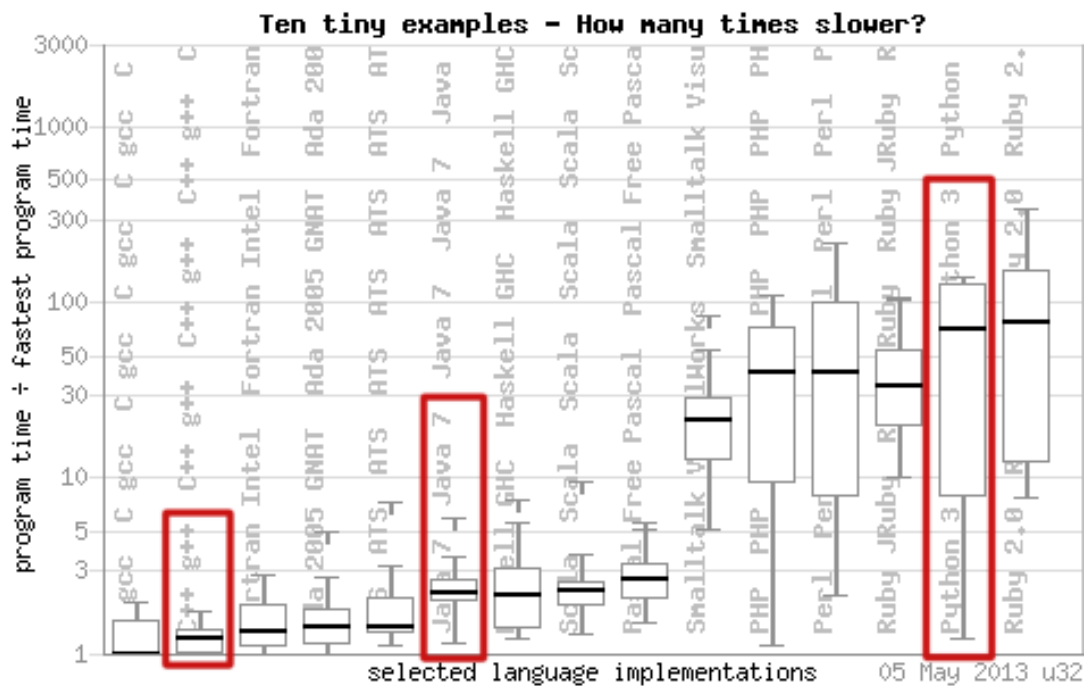


Figura 35: Gráfico general de tiempos de ejecución de los benchmark.

x	Lenguaje	CPU secs	Memoria en KB
1.0	C gcc	0.82	248,636
1.6	c++ g++	1.33	244,808
2.3	java 7	1.88	511,484
7.9	python 3	6.50	616,588

En primer lugar el estudio se centra en el tiempo de ejecución, se puede observar que *C++* es el más rápido de los tres lenguajes seleccionados, que *Java 7* tiene un rendimiento cercano y en cambio *python 3* su rendimiento se aleja de manera notable con respecto a los otros dos.

En segundo lugar el estudio se centra en el consumo de memoria, se puede observar que *Java 7* y *python 3* tiene un consumo de memoria muy superior al que tiene *C++*, que es menos de la mitad que los dos anteriores, este factor junto con el tiempo de ejecución son los fundamentales a tener en cuenta.

A continuación se realiza la segunda prueba.

K-nucleotide: Actualizar repetidamente tablas hash y cadenas k-nucleótidos. Este algoritmo tiene un consumo superior de CPU con respecto al uso de estructuras de datos, es decir, la proporción de tiempo que gasta en uso de CPU es superior a la que gasta en acceso a estructuras de datos:

x	Lenguaje	CPU secs	Memoria en KB
1.0	c++ g++	23.69	137,448
2.2	java 7	50.67	494,040
21	python 3	7 min	426,212

Ante estos resultados es evidente que python 3 por su rendimiento no es una opción a valorar, entre los dos lenguajes restantes se puede observar que *Java* obtiene un tiempo de ejecución peor a *C++* pero no demasiado alejado.

4.1.2 Comparación basada en el “white paper” [DALHEIMER. [2002]]

En esta segunda comparación entre *Java* y *C++*, entra en juego un aspecto no tratado hasta ahora, las librerías gráficas que se han utilizado en el proyecto. Si nos centramos en *Java* la librería gráfica *AWT/Swing* se utiliza únicamente para programar dicha interfaz, en cambio para *C++*, *Qt* se utiliza, aparte de para programar la interfaz, para dotar a *C++* de multiplataforma entre otras mejoras a la hora de programar. Al utilizarse *Qt* para la programación del núcleo del sistema, el uso de estas librerías afecta al rendimiento. Para tomar la decisión final sobre el lenguaje a utilizar, no solo el rendimiento es un factor a tener en cuenta, es también importante las facilidades que otorga el lenguaje para la programación, la reutilización futura del código, la ampliación del sistema, etc. Es por ello que se utiliza en esta segunda comparación este “white paper” que hace un análisis profundo de estas cuestiones fundamentales para el desarrollo del prototipo.

El documento se divide en una serie de secciones que tratan distintos aspectos fundamentales a la hora de la elección del lenguaje de programación, a continuación se desarrollan.

Eficiencia en la programación:

Un aspecto importante a la hora de valorar el lenguaje a utilizar es la eficiencia en la programación, dado que el tiempo que se tiene que dedicar a la programación de dicho proyecto es un factor muy importante a tener en cuenta, sobre todo reflejado en el esfuerzo de las personas encargadas.

La principal diferencia en este aspecto entre *Java* y *C++* es la liberación automática de memoria que implementa *Java* con su recolector de basura, y propicia que el programador pueda olvidarse de esta tarea, disminuyendo así la cantidad de errores introducidos en el código. En este sentido se puede utilizar un software clásico de modelos de estimación CoCoMo [BOEHM [1981]] que predice el costo y el tiempo de un proyecto teniendo en cuenta factores como la experiencia general de los programadores, la experiencia de los programadores con el lenguaje en de programación, etc. La conclusión es que la cantidad de esfuerzo es independiente del lenguaje de programación.

Otras investigaciones, por ejemplo, [WALSTON Y FELIX [1977]] apunta en la misma dirección.

Estos estudios son muy anteriores a la creación de *Java*, un estudio más reciente, posterior a *Java*, es la comparación empírica de varios lenguajes de programación, entre ellos

C, *C++* y *Java*, que hizo Lutz Prechelt en la University of Karlsruhe [PRECHELT [2000]], en el cual se les asignó una tarea a los estudiantes de informática donde podía elegir el lenguaje de programación que quisieran. Los resultados fueron que entre *C++* y *Java* la diferencia fue mínima. Al final, la conclusión es que la eficiencia programando depende la experiencia del programador con el lenguaje en concreto para lenguajes tan similares como son *Java* y *C++*.

Eficiencia en tiempo de ejecución:

En el mismo estudio realizado por Lutz Prechelt la cantidad de datos que proporciona con respecto a este tema es enorme, pero llega a la conclusión de que “*Java* es como mínimo 1.22 veces más lento que *C/C++* para programas equivalentes”. La razón de esta diferencia está seguramente en que *Java* es un lenguaje interpretado mientras que *C++* es compilado. También hay que tener en cuenta que en las pruebas benchmark realizadas anteriormente no se mide el tiempo de arranque de la máquina virtual de *Java*.

Eficiencia en el uso de memoria:

El enfoque sobre la gestión de memoria entre *C++* y *Java* son completamente diferentes, en *C++* el programador debe gestionar la memoria manualmente, tanto su asignación como su liberación. En cambio en *Java* el recolector de basura se encarga de liberar la memoria que ya no se utiliza periódicamente. El problema que se presenta con *Java* es que hasta la siguiente actuación del recolector de basura la memoria no se libera, con lo cual esta se acumula hasta su liberación y hace efecto sierra a lo largo del tiempo. Si utilizamos, otra vez, los datos proporcionados por Lutz Prechelt, estos dicen, con una confianza del 80 %, que *Java* consume por lo menos un 297 % más de memoria.

Comparación entre *AWT/Swing* y *Qt*:

La conclusión que se llega en esta sección es que *AWT/Swing* tiene problemas similares a los nombrados con respecto *Java*, y es su gran consumo de recursos de la CPU, y de memoria. En cambio *Qt* al estar programado sobre *C++* y utilizar bibliotecas nativas para la creación de los elementos simples, obtiene un nivel de rendimiento similar a un programa *C++* sin interfaz gráfica.

4.1.3 Conclusiones

Las conclusiones que se obtienen en la comparación de lenguajes de programación es que *Qt/C++* es la opción más rápida y con menor consumo de memoria, y que además no introduce ineficiencias a la hora de la programación como se podía pensar inicialmente. Con lo cual es el lenguaje elegido para el desarrollo del prototipo. Como *IDE* básico para desarrollar el prototipo se decidió usar *QtCreator* que es el *IDE* por defecto de *Qt*.

4.2 Introducción a *Qt*

En 1991, dos programadores noruegos (Eirik Eng y Haavard Nord de Quasar Technologies, más tarde Trolltech) crearon un Framework en *C++* que permitía usar el mismo

código en el desarrollo de una interfaz gráfica tanto para Windows como para UNIX/X11. Entre 1995 y 1998 se desarrolló con este Framework un escritorio para GNU/Linux llamado KDE que hoy sigue siendo uno de los escritorios más usados. Entre los años 2000 y 2005 *Qt*, que originalmente no era libre, fue liberado bajo licencia *GLP*.

Actualmente, a mediados de 2013 acaba de ser liberada a la versión 5.1 de *Qt* que tiene soporte para dispositivos móviles (Android, iOS) y electrodomésticos, y funciona en las principales plataformas. *Qt* incorpora un estilo de programación a *C++*, que le hace menos vulnerable a fallos de programación, sobre todo en el manejo de memoria, y que lo hacen muy agradable de cara al desarrollador.

El API de la biblioteca cuenta con métodos para acceder a bases de datos mediante *SQL*, así como uso de *XML*, gestión de hilos, soporte de red, un API multiplataforma unificada para la manipulación de archivos y acceso a ficheros, además de estructuras de datos tradicionales.

La elección de *Qt* para el desarrollo de prototipo supuso la necesidad de aprender el uso de las librerías de *Qt* desde cero, este aprendizaje se basó en un libro, facilitado por el departamento de informática y sistemas, [BLANCHETTE Y SUMMERFIELD [2008]], en el manual anónimo *Aprenda Qt4 desde hoy mismo* [ANÓNIMO [2010]] y por supuesto en la documentación oficial de *Qt* [DIGIA [1991]].

El proceso de aprendizaje de *Qt* es diferente al resto de librerías gráficas, por el hecho, de que *Qt* no es solo una librería gráfica, sino que también dota a *C++* de multiplataforma con lo cual ofrece formas alternativas de programar todo lo que en *C++* es dependiente del Sistema operativo. Durante la realización del prototipo se han utilizado diferentes conceptos aportados por *Qt* por ello es interesante hacer un resumen rápido para entender cómo se ha construido el prototipo.

Qt ofrece amplias estructuras para la gestión de datos, dichas estructuras se basan completamente en la orientación a objetos y aportan una facilidad de programación notable a *C++*. Están a disposición del programador desde objetos básicos para el manejo de cadenas hasta listas enlazadas, vectores o mapas hash.

Otras de las novedades importantes que aporta *Qt* son las *señales* y *slot* que permiten la comunicación entre dos objetos aunque estos no se conozcan entre sí. Esta es la forma básica de conectar objetos en *Qt*, que permite definir nuevas señales por el programador, consiguiendo con ello la señal adecuada para cada momento.

En la gestión de ficheros, tanto para lectura como escritura, *Qt* ofrece funciones de alto nivel para un acceso multiplataforma a dichos ficheros, en la sección (4.3) se explicará con más detalle en el contexto del sistema desarrollado para guardar los datos de un proyecto.

Para terminar, y como última parte fundamental utilizada de *Qt*, las librerías gráficas han sido la parte fundamental para el desarrollo de la *GUI*. Se ha utilizado el lenguaje interpretado QML para el desarrollo de las ventanas sencillas y estáticas, y la programación en código para el desarrollo de ventanas no estáticas, es decir, aquellas cuya composición

dependía de los valores de entrada proporcionados por el usuario. En el contexto del prototipo, la interfaz gráfica se usa principalmente para la entrada de datos por parte del usuario y posteriormente para mostrar los resultados obtenidos a partir de esos datos. Ante esto lo ideal habría sido proporcionar una entrada robusta de datos que no permitiera al usuario llevar al sistema hasta un estado de error. Pero como se mencionó al principio de este capítulo, al ser un prototipo y con el coste en tiempo que habría llevado, se decidió centrar el desarrollo en el núcleo del sistema dejando estas cuestiones menores para futuras revisiones.

4.3 Sistema de guardado

El funcionamiento resumido del prototipo, con respecto a los datos que maneja, consiste en la obtención de datos por parte de un usuario, utilizar dichos datos para hacer una serie de cálculos y mostrar al usuario los resultados obtenidos. Es por ello que se estimó fundamental poder guardar los datos, introducidos y calculados, a un fichero para poder recuperarlos posteriormente para mayor comodidad del usuario.

La decisión más importante con respecto a este apartado fue elegir que formato era el más adecuado para guardar los datos, se estudiaron distintas opciones, desde formatos binarios, hasta texto plano sin formato. Al final se decidió por una solución intermedia entre un fichero binario y uno en texto plano, que es guardar los datos en texto plano pero utilizando *XML* [W3C [1996]] para ello. *XML* es un lenguaje de marcas extensible, estandarizado por el W3C (*World Wide Web Consortium*) en 1996 y que aporta una forma estructurada de escribir datos en un fichero. Es por ello que se eligió este sistema para guardar los datos, permite guardar y recuperar datos de forma sencilla.

La estructura de *XML* se conforma de una cabecera donde se indica la versión de *XML* usada y la codificación elegida. A continuación un documento *XML* se construye con elementos, dichos elementos tienen un nombre y pueden contener uno o varios atributos. Dentro de un elemento se pueden definir más elementos recursivamente, esto permite crear diferentes secciones para agrupar datos. Por último un elemento puede contener caracteres, que es la forma característica de almacenar datos, de esta forma se pueden escribir datos de forma ordenada. Para que un documento *XML* sea sintácticamente correcto todos los elementos abiertos (secciones) deben ser cerrados.

El siguiente fichero ejemplo corresponde a la planta:

$$P(s) = k \frac{s + 1}{(s + a)(s + b)} \quad (12)$$

$$k \in [1, 6]$$

$$a \in [3, 5]$$

$$b \in [1, 3]$$

Dicha planta ha sido guardada en un fichero y corresponde al siguiente código ejemplo:

```

1
2 <!-- cabecera -->
3 <?xml version="1.0" encoding="UTF-8"?>
4 <!-- Sección general -->
5 <QFT>
6   <!-- Sección Planta con un atributo nombre -->
7   <Planta nombre="planta-1">
8     <!-- Sección tipo con un atributo tipo -->
9     <tipo tipo="K-Ganancia">
10      <!-- Sección numerador con un atributo size -->
11      <numerador size="1">
12        <!-- Sección variable-0 -->
13        <variable-0>
14          <!-- Elemento nominal con el valor 1 -->
15          <nominal>1</nominal>
16          <!-- Elemento variable valor false-->
17          <variable>false</variable>
18          <!-- Se cierra sección variable -->
19          </variable-0>
20        </numerador>
21        <!-- Sección denominado con longitud 2 -->
22        <denominador size="2">
23          <!-- Sección variable-0 -->
24          <variable-0>
25            <!-- Elemento nominal con valor 4 -->
26            <nominal>4</nominal>
27            <!-- Elemento variable valor true -->
28            <variable>true</variable>
29            <!-- Elemento nombre con valor a -->
30            <nombre>a</nombre>
31            <!-- Sección rango -->
32            <rango>
33              <!-- Elemento inicio con valor 3 -->
34              <inicio>3</inicio>
35              <!-- Elemento final con valor 5 -->
36              <final>5</final>
37              <!-- Se cierra la sección rango -->
38              </rango>
39            <!-- Se cierra la sección variable -->
40            </variable-0>
41            <!-- Comienza variable-1 igual que la anterior -->
42            <variable-1>
43              <nominal>2</nominal>
44              <variable>true</variable>
45              <nombre>b</nombre>
46              <rango>
47                <inicio>1</inicio>
48                <final>3</final>
49              </rango>
50            </variable-1>

```

```

51      <!-- Se cierra la sección denominador -->
52      </denominador>
53      <!-- Sección variable-k -->
54      <variable-k>
55          <nominal>1</nominal>
56          <variable>true</variable>
57          <nombre>k</nombre>
58          <rango>
59              <inicio>1</inicio>
60              <final>6</final>
61          </rango>
62      </variable-k>
63      <!-- Sección variable-ret -->
64      <variable-ret>
65          <nominal>0</nominal>
66          <variable>false</variable>
67      </variable-ret>
68      <!-- Se cierra sección tipo -->
69      </tipo>
70      <!-- Se cierra sección planta -->
71      </Planta>
72  <!-- Se cierra sección general -->
73  </QFT>

```

Como se puede observar en el ejemplo se puede guardar información de manera muy ordenada, todo está dividido en secciones y subsecciones permitiendo con ello una fácil lectura posterior.

Qt facilita la gestión de *XML* al aportar un parser¹⁶ para lectura y escritura¹⁷, la utilización de dicho parser ha sido sencilla, a pesar de la gran cantidad de datos a guardar, sobre todo en la etapa del cálculo de fronteras, donde los datos crecen de manera significativa. En (C) se puede observar un ejemplo completo de un fichero de este tipo, en concreto, guarda los datos generados en el ejemplo de la sección de pruebas (5).

4.4 Trabajo colaborativo

Antes de empezar el proyecto se hizo un estudio a fondo de las distintas posibilidades de trabajo cooperativo de las que se podía disponer, al comiendo del proyecto, éramos dos los alumnos interesados en el mismo, y se decidió utilizar un sistema de control de versiones como mecanismo para poder trabajar de manera colaborativa. Como se explicó en la sección (4.1.3) la plataforma de desarrollo elegida fue el *IDE* por defecto de *Qt*, *QtCreator*, por lo tanto era fundamental conectar dicho *IDE* con el control de versiones. En un primer momento se valoraron dos de las opciones más utilizadas actualmente para el control de versiones, que son *git* [HAMANO Y TORVALDS [2012]] y *subversion* [COMUNIDAD ET AL. [2000]]. Las dos aplicaciones son ampliamente utilizadas y cuentan

¹⁶Un parser es un traductor de unos datos origen en un formato a unos datos destino en otro formato, en este caso traduce datos guardados en estructuras de datos al lenguaje *XML*.

¹⁷<http://qt-project.org/doc/qt-4.8/xml-processing.html> Último acceso (01/08/20013)

con clientes para todos los sistemas operativos, por lo tanto la elección debe basarse exclusivamente en las necesidades de proyecto.

La principal diferencia es la forma de concepción del repositorio, *git* trabaja de forma distribuida mientras que *subversion* lo hace de manera centralizada. La ventaja que aporta *git* es que cada usuario puede trabajar de manera individual con su propio repositorio local, incluso sin conexión a Internet, y unir cuando lo crea conveniente su trabajo al resto de repositorios conectados. En cambio *subversion* siempre está conectado al repositorio central y los cambios efectuados son reflejados en el siguiente *commit*. En un proyecto de grandes dimensiones es recomendable trabajar con *git* dado que las distintas partes del proyecto se trabajan frecuentemente de forma separada. Como contrapartida configurar un control de versiones con *git* es mas complicado, dado que debe configurarse un repositorio local en cada equipo de trabajo y no basta con un solo repositorio central.

Para un proyecto de dimensiones como el actual, *subversion* es apropiado dado que las distintas partes del prototipo van fuertemente ligadas y han de ser unidas al final de cada sesión de trabajo. Además, como se ha dicho anteriormente *subversion* es más sencillo de configurar.

Como conclusión decir que *git* ofrece muchas mas opciones y posibilidades para el trabajo colaborativo que serían fuertemente aprovechadas en la realización de proyectos de tamaño medio y grande. En cambio para un proyecto pequeño, esas cualidades son difícilmente aprovechables con lo cual es más interesante y sencillo usar *subversion*.

Como servidor de *subversion* se decidió utilizar el servidor gratuito *Assembla*¹⁸. Una vez elegido el control de versiones y creado el proyecto en el servidor, el siguiente paso fue configurarlo en el *QtCreator*. El único requisito requerido es que depende de un cliente externo, por lo tanto hay que instalar un cliente adecuado dependiendo del sistema operativo que se esté utilizando, una vez hecho esto la configuración es muy sencilla.

4.5 Patrones de diseño utilizados

Durante el desarrollo del prototipo se han utilizado diversos patrones de diseño para mejorar el funcionamiento y solventar problemas clásicos que han surgido durante el desarrollo. En esta sección se pretende hacer un resumen de los distintos patrones utilizados, explicar su funcionamiento y la finalidad de su uso.

Se pueden distinguir dos tipos de patrones, los más básicos, conocidos como patrones *GRASP* que son *patrones generales de software para asignación de responsabilidades*. Dichos patrones definen una forma de programar más que dar una solución técnica para un problema concreto, en cualquier software actual no se concibe su desarrollo sin seguir estos patrones.

El segundo tipo de patrones de diseño están en una amplia recopilación recogida en el libro [GAMMA ET AL. [1995]] más conocidos como *GoF* (*Gang of Four*) por los 4 autores del libro. En dicho libro se ofrecen una serie de patrones divididos en tres categorías:

¹⁸<https://www.assembla.com> Último acceso (01/08/2013)

“creacionales”, “estructurales”, y de “comportamiento”. Dichos patrones proveen soluciones técnicas ampliamente validadas para problemas típicos en el desarrollo de software.

4.5.1 *Modelo-Vista-Controlador*

El *Modelo-Vista-Controlador* (*MVC*) es un patrón de arquitectura de software perteneciente a los patrones denominados *GRASP*, que separa los datos y la lógica de negocio de una aplicación de la interfaz de usuario y el módulo encargado de gestionar los eventos y las comunicaciones. Para ello *MVC* propone la construcción de tres componentes distintos que son el modelo, la vista y el controlador, es decir, por un lado define componentes para la representación de la información, por otro lado para la interacción del usuario y propone un controlador como comunicación entre ambas partes. La utilización de un controlador permite tener una forma única de comunicación, es decir, el controlador actúa como interfaz de comunicación, tanto entrada como salida de información entre el modelo y la interfaz de usuario. Este patrón de diseño se basa en las ideas de reutilización de código y la separación de conceptos, características que buscan facilitar la tarea de desarrollo de aplicaciones y su posterior mantenimiento.

Este patrón se ha utilizado durante el desarrollo del prototipo, cumpliendo los postulados que en él se reflejan, en la figura (36) se puede observar un diagrama de clases del *MVC*, en la figura (37) un diagrama de secuencia y en la figura (38) se puede observar cómo cambiar la interfaz gráfica y sustituirla por una interfaz web es muy sencillo. Es por ello que se ha aplicado este patrón, en un futuro puede ser interesante cambiar las librerías gráficas por otras o hacer una interfaz en consola, este patrón permite todo eso sin tener que modificar el modelo.

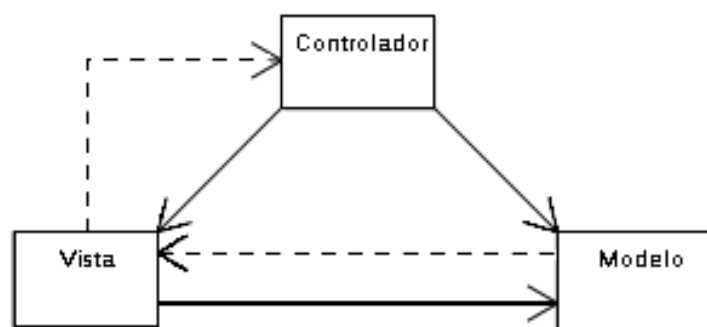


Figura 36: Diagrama de clases del patrón Modelo-Vista-Controlador.

4.5.2 *Observer*

La finalidad de este patrón es definir una dependencia “uno-a-muchos” entre objetos, de tal forma que cuando el objeto cambie de estado, todos sus objetos dependientes sean notificados automáticamente. Se trata de desacoplar la clase de los objetos clientes del objeto, aumentando la modularidad del lenguaje, creando las mínimas dependencias y evitando bucles de actualización (espera activa o polling). En definitiva, se usará el

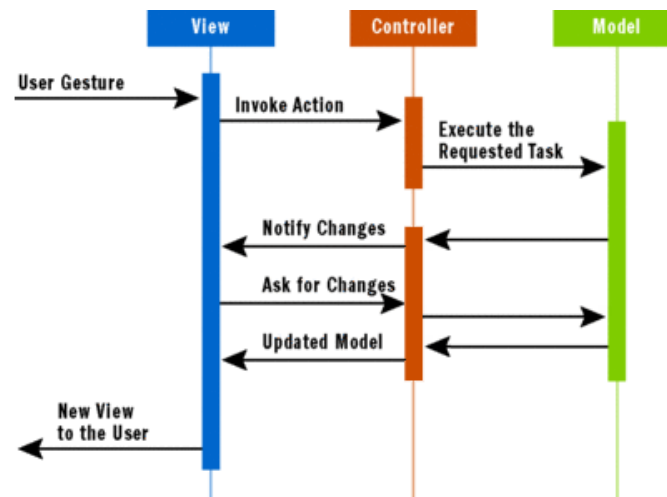


Figura 37: Diagrama de secuencia del patrón Modelo-Vista-Controlador.

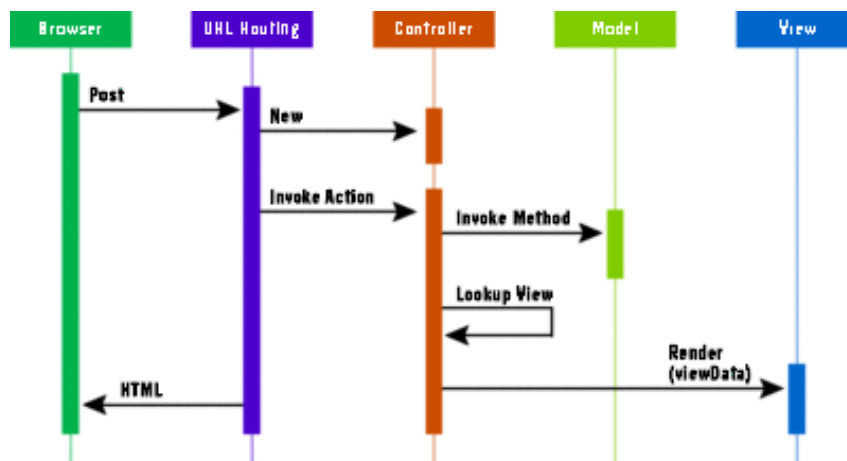


Figura 38: Diagrama de secuencia del patrón Modelo-Vista-Controlador sustituyendo la GUI por un navegador.

patrón *observer* cuando un elemento “quiere” estar pendiente de otro, sin tener que estar encuestando de forma permanente si éste ha cambiado o no.

Los actores participantes en el patrón son de dos tipos, por un lado está el *sujeto* que es el generador de avisos, y por otro lado están los *observadores* que son los receptores de los avisos generados. Generalmente en la implementación de este patrón se definen dos clases abstractas que definen las funciones típicas para los dos tipos de actores. Así después sólo se tiene que heredar de esas clases e implementarlas según las necesidades concretas, esto permite tener varios patrones *observer* al mismo tiempo con funcionamientos distintos. En la figura (39) se puede observar la estructura básica del patrón, las funciones principales del *Sujeto* tienen la funcionalidad de añadir y eliminar *observadores*, y notificarlos cuando el sujeto concreto se actualice. Dichas funciones, en los casos generales, pueden ser implementadas en la misma clase abstracta. En casos más específicos, donde sea necesario enviar datos a los *observadores* se tendrá que hacer una implementación particular para

cada caso. En la clase abstracta de los *observadores* la función *actualizar* es la principal para su funcionamiento, esta función debe ser implementada en cada observador concreto con las acciones que debe llevar a cabo con la notificación.

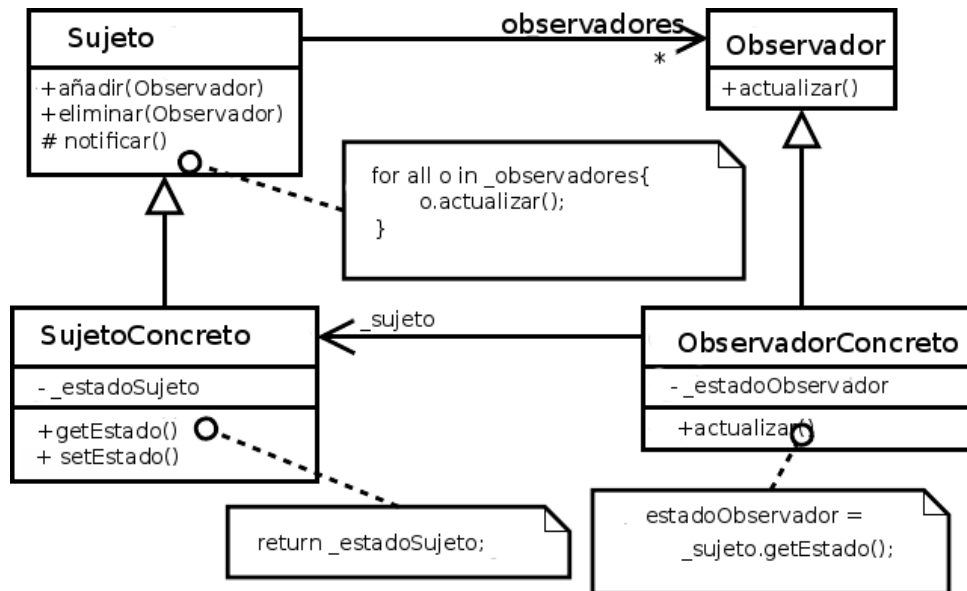


Figura 39: Diagrama de clases del patrón Observer

Este patrón se usa ampliamente en *Qt*, las *señales* y *slot* nombrados en la sección (4.2) utilizan este patrón para la comunicación entre objetos, con lo cual se ha usado para el desarrollo del proyecto de manera implícita.

Una utilización directa de este patrón estaba planeada en el sistema de recálculo de valores al cambiar los datos de entrada, pero en el caso del prototipo resultante se aplicaba el patrón con algunas modificaciones, adaptándolo a las necesidades del prototipo. La implementación particular propuesta se encuentra con la problemática de que se basa en una comunicación “muchos-a-muchos”, es decir, las posibles fuentes de eventos son varias, cualquiera de las partes del sistema donde el usuario puede introducir datos o datos que se recalculen por orden del usuario. En cuanto a la notificación de los *observadores* también es diferente, dado que dependiendo del generador del evento se deberá notificar a mas o a menos *observadores*. Teniendo en cuenta estas particularidades y modificando el diseño del patrón en consonancia el prototipo tendría implementado el recálculo automático de datos, lo cual es una característica muy interesante.

4.5.3 DAO

El patrón *DAO* (*Data Access Object*) es un patrón de tipo *adaptador*, este tipo de patrón se utiliza para transformar una interfaz en otra, de tal modo que una clase que no pudiera utilizar la interfaz original, haga uso de ella a través de la interfaz adaptadora.

El patrón *DAO* es una especialización del *adaptador* que ofrece la funcionalidad de separar la implementación de la persistencia de su uso por parte del controlador. En cual-

quier momento se podría decidir cambiar de utilizar una base de datos a un fichero y dicho cambio sería transparente para el resto del sistema. El diagrama de clases de la figura (40) representa la implementación hecha en el prototipo, la clase *ControladorDAO* aplica el patrón *factoría* para crear los *adaptadores*, dicho patrón define una clase constructora que automáticamente construye las clases *adaptadores* que envía al *controlador* del sistema.

En la figura (41) se puede observar un diagrama de secuencia del patrón y en la figura (23) se puede observar un diagrama de colaboración.

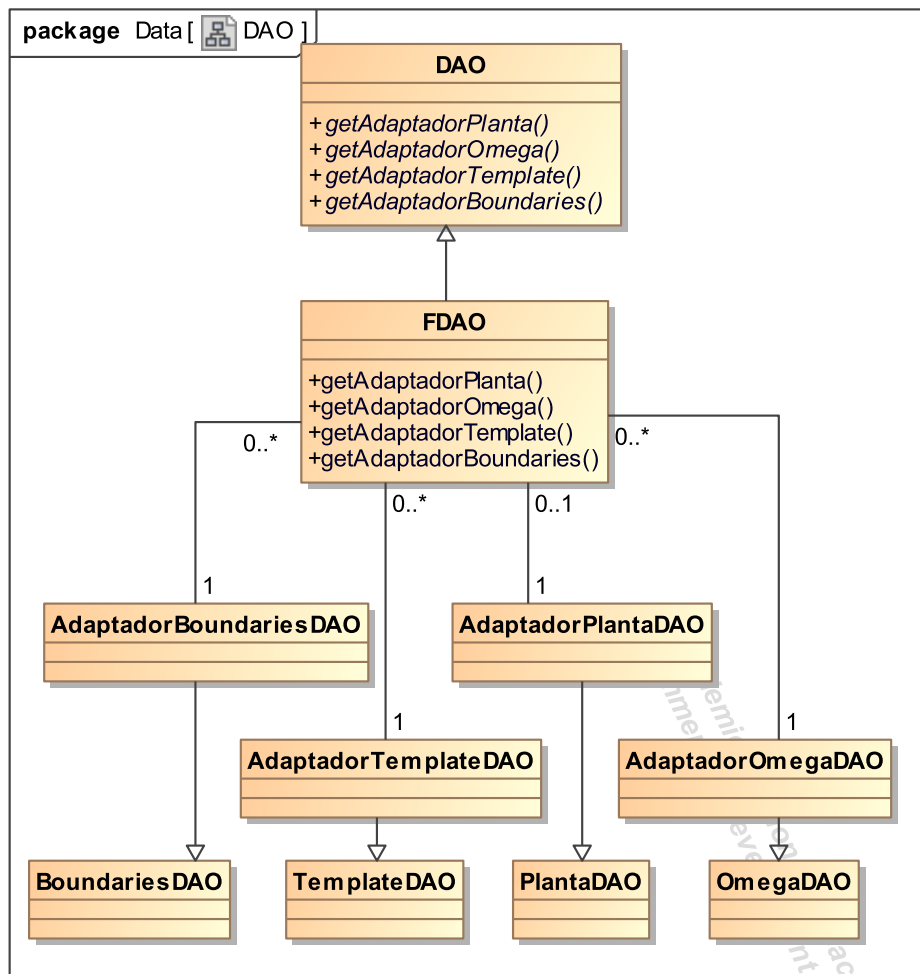
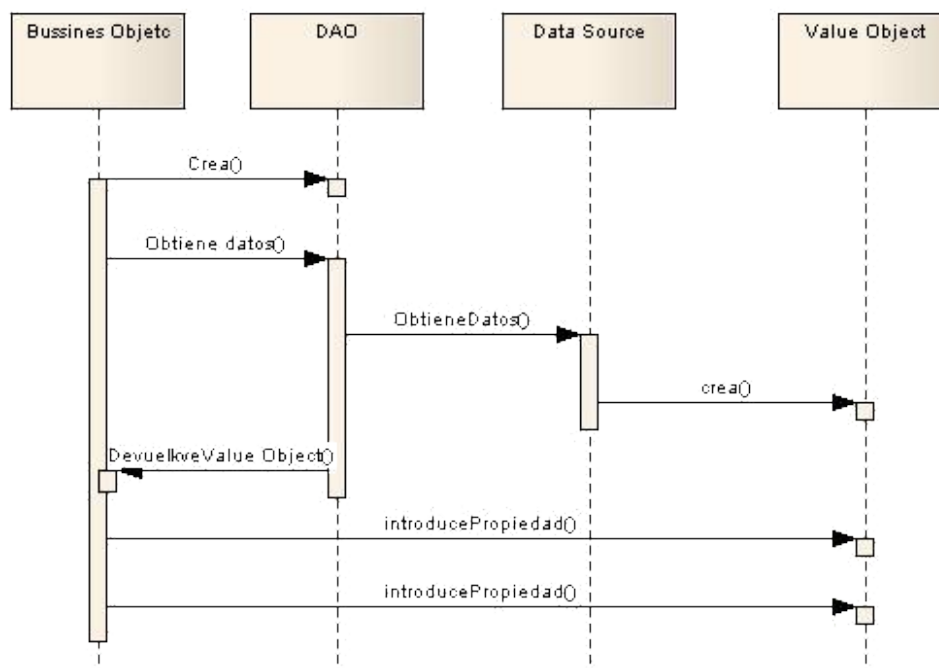


Figura 40: Diagrama de clases del patrón *DAO* implementado en el sistema.

4.6 Librerías utilizadas

Durante el desarrollo del prototipo se han utilizado una serie de librerías externas que ofrecían funcionalidades útiles para resolver problemas concretos, concretamente se han utilizado dos librerías y una función que se explican a continuación.

Figura 41: Diagrama de secuencia del patrón *DAO*.

4.6.1 Librería *Contour*

La función *Contour* trabaja sobre una sábana de valores que se define como una función $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ con dominio en el plano de $\mathbb{R}^2$ e imagen en la recta real, tal que $f(x, y) = h$ para todo (x, y) perteneciente a $\mathbb{R}^2$ y h perteneciente a la imagen de f . Dado que el sábana es un *plano de nichols* de números complejos, también se puede definir como una función $f : \mathbb{C} \rightarrow \mathbb{R}$ con dominio en el conjunto de los números complejos e imagen en la recta real, tal que $f(z) = h$, para todo z perteneciente a C y h perteneciente a la imagen de f , siendo h un número real que determina una altura para un punto (x, y) . Estas dos definiciones son equivalentes dado que $\mathbb{R}^2$ y $\mathbb{C}$ son isomorfos, siendo $\mathbb{C}$ el conjunto de los números complejos.

La función *Contour* recoge el conjunto de soluciones de las ecuaciones $f(x, y) = h_i$ con h_i perteneciente a un conjunto J . Es decir, es una función $g : J \rightarrow 2^{\mathbb{R}^2}$ tal que para todo h_i perteneciente a J da un subconjunto de $2^{\mathbb{R}^2}$ como imagen de la función. O dicho de otra forma, la función *Contour* recoge el conjunto de curvas de nivel $f(x, y)$ que son una familia de funciones de la forma: $f(x, y) = h_i$ para cada h_i perteneciente J .

En la realización del prototipo el uso de una función *Contour* fue fundamental, dado que es necesario utilizarla en el algoritmo de cálculo de fronteras. Ni *C++* ni *Qt* disponen de una implementación de dicha función, por lo tanto fue necesario buscar una librería que la implementara. Concretamente para una matriz $N \times M$ de alturas, y dada una altura h como objetivo, el algoritmo obtendrá como resultado la curva de nivel de las alturas de la matriz a la altura h elegida. Dicha matriz de alturas se genera como paso previo hacia la solución del algoritmo de cálculo de fronteras, de hecho la solución del algoritmo

es la aplicación de la función *Contour* a dicha matriz de alturas. En la sección (4.7.3) se explica de forma completa el algoritmo de cálculo de fronteras.

La búsqueda de dicha librería no fue sencilla, dado que este tipo de funciones suelen estar incluidos en paquetes software más grandes de tratamiento de gráficos, generalmente con licencia privativa. Este tipo de funciones son muy usadas en topografía con lo cual buscando por Internet en seguida se encuentran páginas web útiles, el problema fue que la mayoría del software encontrado había sido programado para uso personal y compartido posteriormente, con lo cual era necesario adaptarlo para cada proyecto concreto.

Después de mucho buscar se encontró la librería de cálculo de contornos realizada por [DE HALLEUX [2002]], dicha librería fue funcional con unas mínimas modificaciones. Concretamente está programada en forma de herencia con dos clases hijas, una de ellas con su propia interfaz gráfica, la otra clase hija, la que interesa para este proyecto, presenta los resultados en un fichero. La función que escribe los resultados en un fichero ha sido modificada para que simplemente retorne los resultados en vectores y poder ser así usada por el resto del sistema. La parte gráfica del código ha sido eliminada por completo, dado que no es necesaria para el proyecto. Otra modificación necesaria fue modificar los tipos de datos nativos y portarlos a *Qt* para que fuera multiplataforma y funcionara tanto en 32 como en 64 bits.

Esta librería está basada en la tesis [ARAMINI [1980]], dicho algoritmo está orientado a trabajos de topografía, lo cual da interesantes opciones, como pueden ser, poder dar sucesivos “cortes” a diferentes alturas. El código base de algoritmo está en la clase padre (*CContour*), de la que hace uso la clase hija (*CListContour*).

4.6.2 Librería *QCustomPlot*

La librería *QCustomPlot* [EICHHAMMER [2012]] provee gran cantidad de funciones y tipos gráficos para poder hacer representaciones gráficas en 2 dimensiones. Aunque *Qt* ofrece librerías gráficas para el desarrollo de estas representaciones¹⁹ las facilidades y mejoras que provee *QCustomPlot* son notables con respecto a las anteriores. Es por ello que se consideró utilizar dichas librerías y así poder ofrecer interesantes opciones en la representación 2D. Por ejemplo en todos los gráficos existe la opción de mover los ejes, hacer zoom, exportar a imagen, etc. En vías futuras, y tal y como está modelado en la especificación de requisitos (sección 3), en la funcionalidad de introducir una planta mediante *datos experimentales* es posible modificar el *diagrama de Bode* resultante para eliminar puntos no deseados. La manera más sencilla es poder modificar el gráfico 2D resultante e ir eliminando puntos no deseados del mismo. La librería *QCustomPlot* solo utiliza las librerías de *Qt*, con lo que realmente no ofrecen nada nuevo, pero si una manera mucho más sencilla de hacerlo, es por ello que se decidió utilizarla.

¹⁹<http://qt-project.org/doc/qt-4.8/graphicsview.html> Último acceso (2/08/2013)

4.6.3 Función *pnpoly*

La función *pnpoly* [FRANKLIN [2003]] es usada por el algoritmo de cálculo de fronteras, la finalidad de dicha función, es dado un polígono irregular en 2 dimensiones y un punto (x, y) determinar si dicho punto está dentro del polígono. En la sección (4.7.3), donde se explica el algoritmo de cálculo de fronteras, se explicará con más profundidad su uso. La función ha sido adaptada para usar las mejoras que aportan las librerías de *Qt*, el código original se puede encontrar en la bibliografía [FRANKLIN [2003]].

4.7 Algoritmos implementados en el prototipo

Durante el desarrollo del prototipo se han implementado diversos algoritmos para la resolución de las distintas fases de QFT (1.2), concretamente han sido tres, a continuación son explicados.

4.7.1 Algoritmo de fuerza bruta para el cálculo de las plantillas

Para el cálculo de las plantillas, la solución más sencilla es calcular todas las soluciones de la planta junto con las frecuencias de diseño y obtener las nubes de puntos correspondientes. Las plantillas son utilizadas para el cálculo de fronteras, y realmente sólo es necesario el contorno de dicha nube de puntos, por lo tanto, calcular todas las posibilidades de la planta es la forma más lenta computacionalmente hablando. Pero también es la forma más sencilla, el algoritmo de fuerza bruta nos asegura encontrar siempre una solución, lo cual no se puede asegurar para otro tipo de algoritmos. Si se conoce la planta a la perfección, es posible aplicar otro tipo de algoritmos que basen su cálculo en la búsqueda del contorno necesario para el cálculo de fronteras.

La dificultad planteada para implementar el algoritmo de fuerza de bruta, es que el número de variables participantes en la resolución es indeterminado en tiempo de compilación, depende de la entrada del usuario, con lo cual no es posible resolver utilizando bucles anidados. Cada variable tiene una serie de valores posibles, generalmente un rango con un espaciado (lineal o logarítmico) para sus valores intermedios. La solución implementada utiliza un vector de enteros donde cada variable tiene una posición asignada, el contenido de dicha posición indica cuál de los valores posibles de cada variable se tiene que utilizar. Dicho valor cambia conforme va avanzando el algoritmo. El código principal se puede encontrar en el anejo 2 ([Algoritmo de cálculo de plantillas](#)).

4.7.2 Algoritmo e-hull para el cálculo del contorno de las plantillas

Como se indicó en la sección (4.7.1) cuando el algoritmo de cálculo de fronteras utiliza las plantillas, solo necesita su contorno, aunque admitiría trabajar con la plantilla completa generada por el algoritmo de fuerza bruta, el tiempo de CPU aumentaría enormemente. Por ello es conveniente implementar algún tipo de algoritmo que calcule el contorno de cada plantilla. El algoritmo elegido para el prototipo es ϵ - *hull* [NORDIN [1993]] utilizando como base el algoritmo desarrollado por [MONTROYA [1998]], el funcionamiento del algoritmo se explica a continuación.

El inicio del algoritmo es la nube de puntos calculada por el algoritmo de fuerza bruta, y el objetivo es obtener el contorno de dicha nube de puntos. En primer lugar el algoritmo requiere como parámetro un número real que actuará como distancia (ϵ), tiene que ser indicada por el usuario y es variable dependiendo de la forma de la nube. El algoritmo se aplica para cada plantilla, cada una de ellas corresponde a una frecuencia de diseño.

La explicación gráfica del funcionamiento del algoritmo es la siguiente: imaginemos que trazamos círculos de radio $\frac{\epsilon}{2}$ alrededor de todos los puntos de la nube, todo punto cuyo círculo no quede completamente cubierto por otros (es decir, sobresale de la nube de círculos) debe formar parte del contorno.

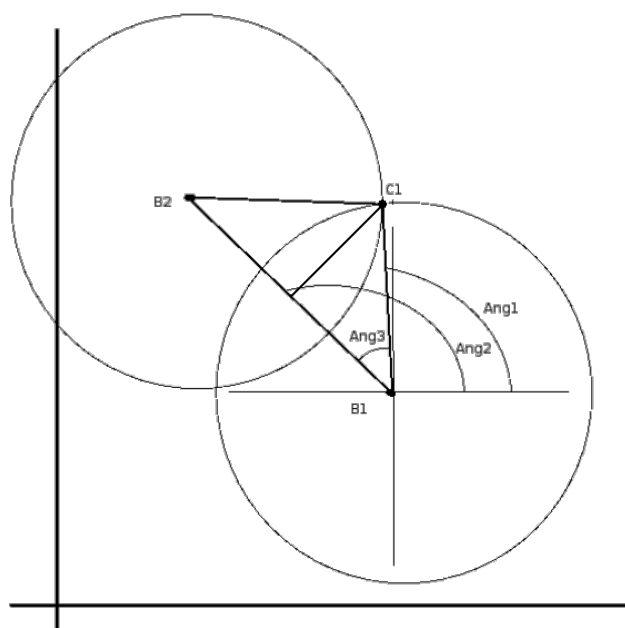


Figura 42: Imagen que representa la búsqueda del segundo punto en el algoritmo de ϵ -hull

A continuación se explica de forma matemática el funcionamiento del algoritmo. En primer lugar el algoritmo selecciona el primer punto, para ello busca qué punto está más a la derecha de todos, en el *plano de Nyquist* o *plano complejo* sería el punto con la parte real más grande. A continuación el objetivo del algoritmo es buscar el segundo punto, para ello inicia un recorrido en sentido antihorario, en la figura (42) se puede observar el funcionamiento del algoritmo. B1 representa el primer punto encontrado por el algoritmo, B2 representa el segundo punto en potencia, los candidatos a segundo punto serán todos aquellos que estén a una distancia menor a ϵ del primer punto, B2 es uno de los candidatos. Se traza alrededor de dichos puntos una circunferencia con radio $\frac{\epsilon}{2}$, el punto de corte de las dos circunferencias es C1. A continuación se traza un triángulo isósceles entre B1, B2 y C1. De entre el conjunto de posibles segundos puntos, el elegido será el que tenga el ángulo denominado Ang1 menor, al ser el ángulo menor el elegido el punto seleccionado será el que su círculo de radio $\frac{\epsilon}{2}$ no quede cubierto por otros círculos. Para

poder calcular este ángulo se necesita calcular el ángulo denominado $Ang2$, si situáramos el origen de coordenadas en el punto $B1$, la recta $B1-B2$ da la posición de $B2$ con respecto al origen de coordenadas, y el ángulo denominado $Ang2$ es el ángulo del número complejo con respecto al eje de coordenadas. A continuación se calcula el ángulo denominado $Ang3$ para ello se traza la bisectriz del triángulo isósceles formado por los tres puntos, obtenemos dos triángulos rectángulos. El ángulo $Ang3$ puede calcularse como el arcocoseno de la división entre cateto contiguo (longitud del segmento $B1-B2$ dividida entre 2) y la hipotenusa ($\frac{\epsilon}{2}$). Para terminar, obtenemos el ángulo denominado $Ang1$ restando $Ang2 - Ang3$. Conociendo este ángulo podemos obtener el segundo punto del contorno.

Como se acaba de explicar el segundo punto se obtiene en una función específica para ello, en cambio los restantes se obtienen en una función general que se repite hasta que se haya completando el contorno, a continuación se explica dicha función general.

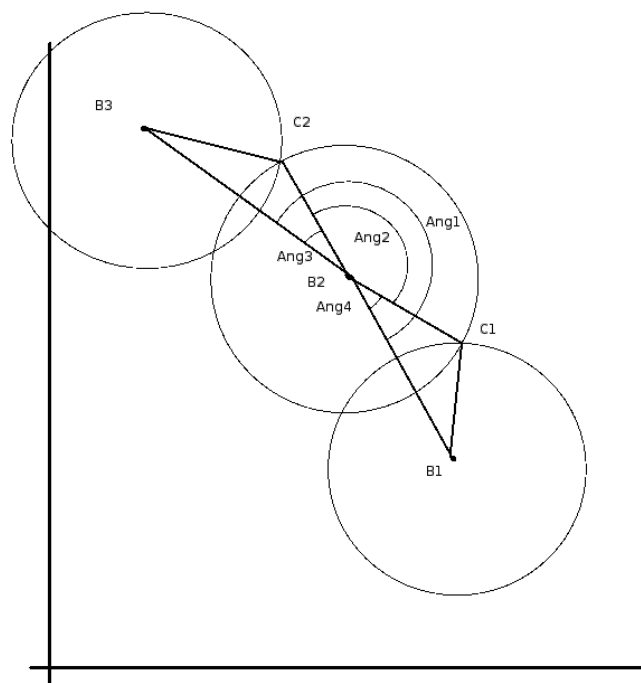


Figura 43: Imagen que representa la búsqueda del siguiente punto en el algoritmo de $\epsilon - hull$.

El algoritmo de búsqueda parte de la base de que conocemos dos puntos como mínimo, el actual y el anterior, por lo tanto la búsqueda del siguiente punto estará influida por los dos anteriores. Al igual que en la búsqueda del segundo punto, el objetivo es buscar el punto el cual su círculo no quede cubierto por la nube de círculos y esté dentro de la distancia ϵ . Serán candidatos, al igual que antes, todos los puntos que estén en un radio inferior al valor de ϵ del punto actual. En la figura (43) se puede observar el funcionamiento del algoritmo, el objetivo es minimizar el valor del ángulo denominado $Ang2$, así se asegura que el punto elegido será el que su círculo de radio $\frac{\epsilon}{2}$ no quede cubierto por otros círculos. Para ello, en primer lugar hay que calcular el valor del ángulo $Ang1$, si

imaginamos que se sitúa el eje de coordenadas en $B2$ la resta $B3 - B2$ da el ángulo de $B3$ con respecto al eje x positivo. A continuación, la resta $B1 - B2$ da el ángulo con respecto al eje x positivo, se trata de un ángulo negativo, con lo cual para obtener $Ang1$ habría que restar los dos ángulos obtenidos. Como se está trabajando con números complejos dicha resta se convierte en la división $(\frac{B3-B2}{B1-B2})$, el resultado es el ángulo denominado $Ang1$.

A continuación se necesitan los valores de los ángulos denominados $Ang3$ y $Ang4$, dichos ángulos se calculan de la misma forma que el ángulo $Ang3$ de la figura (42), a partir de los triángulos rectángulos resultantes de la bisectriz de los dos triángulos isósceles. Una vez calculados los tres ángulos, solo se tiene que restar al ángulo $Ang1$ el valor de $Ang3$ y $Ang4$ y se obtendría el valor del ángulo $Ang2$.

La explicación de este algoritmo ha sido tan detallada por dos motivos fundamentales:

- Durante el estudio del algoritmo no hubo material previo que facilitara el entendimiento del algoritmo, más allá de los comentarios que hizo el profesor *Francisco José Montoya Dato* que no permitían comprender al completo su funcionamiento. Por ello fue una parte importante del proyecto fue entender dicho algoritmo de forma correcta para poder así portarlo a *C++* haciendo la nueva implementación de forma adecuada.
- Dada la falta de información detallada sobre el algoritmo, este estudio puede servir para consultas futuras de quien necesite usar el algoritmo.

El código completo del algoritmo se puede observar en el anejo 2 ([Algoritmo e – hull para el cálculo de contornos de plantillas](#)).

4.7.3 Algoritmo para el cálculo de fronteras

El algoritmo implementado está desarrollado en [MORENO ET AL. [2006]], dicho algoritmo presenta una forma de calcular las fronteras que soporta fronteras multivaluadas. Para poder utilizar este algoritmo se necesitan una serie de datos calculados previamente: la planta nominal, es decir, la planta calculada con los valores nominales de cada variable y las plantillas de la planta calculadas para cada frecuencia de diseño.

Se explicará el funcionamiento del algoritmo para la especificación de estabilidad robusta, que corresponde al tipo de frontera cuyo cálculo se han implementado en el prototipo. La especificación de estabilidad adopta la forma indicada en la ecuación (13).

$$\left| \frac{L(j\omega)}{1 + L(j\omega)} \right|_{dB} \leq \lambda \quad \forall \omega > 0 \quad (13)$$

Donde $L(j\omega)$ representa al lazo abierto,

$$L(j\omega) = P(j\omega)C(j\omega) \quad (14)$$

y a su vez:

- $P(j\omega) = P_0(j\omega)\Delta P(j\omega)$ es el conjunto de posiciones que adopta la planta P a la frecuencia ω , incertidumbre que se puede expresar como un conjunto de variaciones $\Delta P(j\omega)$ respecto a la planta nominal $P_0(j\omega)$.
- C es el controlador a diseñar, que por tanto no es conocido a priori. Precisamente la misión de las fronteras es definir, de forma indirecta, a través de restricciones sobre L , restricciones en forma de regiones prohibidas del *plano de Nichols* tales que los controladores $C(j\omega)$ que las violen serán considerados no admisibles.

El algoritmo basa su funcionamiento en reescribir la ecuación (14) de este modo:

$$L(j\omega) = P_0(j\omega)\Delta P(j\omega)C(j\omega) = L_0(j\omega)\Delta P(j\omega) \quad (15)$$

donde $L_0(j\omega)$ representa la posición del lazo nominal (el que corresponde a la planta $P_0(j\omega)$ de la plantilla) para un cierto controlador $C(j\omega)$ por diseñar.

La restricción dada por la ecuación (13) se puede interpretar ahora como: para una posición elegida del *plano de Nichols* que ocupe $L_0(j\omega)$, para cualquier L dada por este valor de $L_0(j\omega)$ y $\Delta P(j\omega)$ aplicando (15), se debe cumplir (13). Esto puede reescribirse como $D \leq \lambda$, con

$$D = \max \left| \frac{L_0(j\omega)}{\frac{P_0(j\omega)}{P(j\omega)} + L_0(j\omega)} \right|_{dB} \quad (16)$$

La determinación de qué puntos del plano de Nichols son admisibles o no para $L_0(j\omega)$ se realiza estableciendo un grid en el eje de fases y otro en el de magnitudes (en dB.). Para cada punto de ese grid se calcula el valor de D . Esto genera una sábana o función $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ según se define en la sección 4.6.1. La frontera entre las zonas permitidas y prohibidas se calcula mediante el algoritmo *contour* (sección 4.6.1) aplicado a la altura λ .

Problemática con el punto $(-180^\circ, 0dB)$:

Si el punto $(-180^\circ, 0dB)$ está contenido en la región definida por la plantilla se generan problemas numéricos en el cálculo de la ecuación (16):

$$M = |L|_{dB} - 10 \log \left(1 + 2 \cdot 10^{\frac{|L|_{dB}}{20}} \cos \theta + 10^{\frac{|L|_{dB}}{10}} \right)$$

Para el punto $(-180^\circ, 0dB)$:

$$M = -10 \log (1 + 2 \cos(-180) + 1) = -10 \log 0 = \infty$$

Por lo tanto:

$$\lim_{(|L|_{dB} \rightarrow 0, \theta \rightarrow -180^\circ)} M = \infty$$

Con lo calculados en las ecuaciones predecesoras se concluye que si la plantilla acoge en su interior el mencionado punto automáticamente es descartado como zona válida. Por

lo tanto el procedimiento a seguir será comprobar antes de calcular la fórmula (16) si la plantilla contiene al punto $(-180^\circ, 0dB)$.

En cuanto a la implementación del algoritmo surgieron algunos problemas que hubo que solucionar, el principal de ellos fue el creado por la función de la librería estándar de *C++*, *arg*. Dicha función se utiliza para calcular el ángulo de un número complejo, y da los resultados entre el rango $[-180, 180)$. Como se ha explicado anteriormente el algoritmo utiliza un grid de valores donde la fase suele ir de $[-360, 0)$ por lo tanto el valor retornado por la función *arg* tiene que ser movido al rango de valores adecuado, en una primera aproximación se puede pensar como solución adecuada realizar una resta que mueva dicho valor, pero entonces surge un problema grave. Se ha explicado anteriormente que se debe comprobar si el punto $(-180^\circ, 0dB)$ está contenido en la región definida por la plantilla, y si ese es el caso, poner el valor de infinito en esa celda del grid. Por tanto si la plantilla corta el eje de coordenadas situado en el $(0, 0)$, si se realiza una resta para mover los puntos al rango $[-360, 0)$ la plantilla quedará partida en dos, desdibujando completamente el resultado obtenido. También se comprobó que dado el rango de soluciones de la función *arg* está problemática puede darse aunque no se realice la resta, por lo tanto, para el correcto funcionamiento del algoritmo hay que resolver este problema.

Para ello se utilizó la función *unwrap*, dicha función convierte las fases al rango $[-360, 360)$ manteniendo las plantillas unidas, tiene como parámetros de entrada el ángulo actual y el anterior, de esta forma consigue que las plantillas no queden partidas. Después solo hay que comprobar que la plantilla no contenga el punto $(-180^\circ, 0dB)$ ni el punto $(180^\circ, 0dB)$. De esta forma queda solucionado el problema, el código completo del algoritmo se puede observar en el anejo 2 ([Algoritmo de cálculo de fronteras](#)).

4.8 Documentación generada con *Doxygen*

Doxygen [VAN HEESCH [2013]] es un generador de documentación para *C++*, *C*, *Java*, *Objective-C*, *Python*, *IDL* (versiones Corba y Microsoft), *VHDL* y en cierta medida para *PHP*, *C#* y *D*. Dado que es fácilmente adaptable, funciona en la mayoría de sistemas Unix así como en Windows y Mac OS X.

Como se explicó en la sección de introducción al la especificación de requisitos (3.1), el prototipo está implementado solo en las primeras fases y aún queda mucho desarrollo sobre él, con lo cual se estimó importante hacer una documentación que facilitara la ampliación y mejora del prototipo por parte de otras personas en el futuro.

La documentación ha sido generada tanto en pdf (a partir de $\text{\LaTeX}$) como en HTML, además *Doxygen* ofrece interesantes posibilidades como generar documentación para el manual de Linux que pueden ser explotadas en un futuro instalador de la aplicación.

La extensión final de la documentación generada ronda a las 200 páginas, por ello no se ha incluido en este documento como adjunto, se puede encontrar pues en el CD anejo al proyecto junto con el código completo del prototipo.

5 Manual de usuario y batería de pruebas

En esta sección se va a realizar un breve manual de usuario mostrando las distintas características que ofrece la interfaz de usuario del prototipo implementado. Para ejemplificar el proceso se va a usar el mismo ejemplo que en la sección de introducción a *QFT* (1.2) que corresponde con el ejemplo 1 del *QFT toolbox* [BORGHESANI ET AL. [2003]], con lo cual se pueden comparar los resultados obtenidos por el prototipo y comprobar su correcto funcionamiento.

La planta ejemplo tiene los siguientes datos:

$$P(s) = \frac{k}{(s+a)(s+b)} \quad (17)$$

$$k \in [1, 10] \quad \text{Nominal} = 1$$

$$a \in [1, 5] \quad \text{Nominal} = 5$$

$$b \in [20, 30] \quad \text{Nominal} = 30$$

Y las frecuencias de diseño elegidas son:

$$\omega = 0.1, 5, 10, 100$$

5.1 Pantalla principal

Cuando se inicia el programa se muestra la pantalla principal del mismo, se optó por una pantalla principal pequeña desde la cual se pudiera acceder al resto de ventanas del prototipo, con este diseño se pueden abrir distintas ventanas simultáneas y observar los resultados obtenidos para diferentes fases de *QFT*. La pantalla principal corresponde con la figura (44).

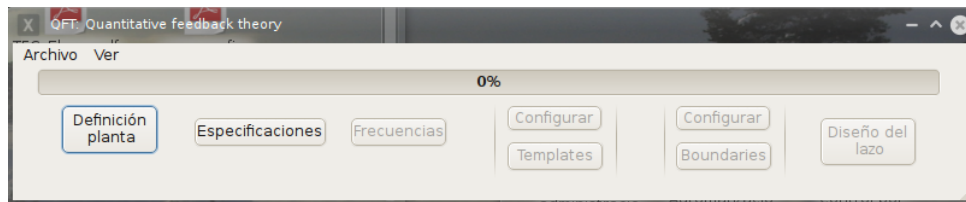


Figura 44: Pantalla principal del prototipo.

5.2 Introducir la planta

Esta fase tiene dos ventanas de usuario, la primera de ellas para introducir los datos básicos de la planta, donde se pueden elegir los distintos tipos de plantas disponibles, como puede observar en la figura (45), una vez seleccionado el tipo de planta, se inserta el numerador y el denominador separando los distintos valores por espacios. Si alguno de esos valores corresponde con una variable se indica con una letra. La ganancia (k) y el

retardo (ret) tienen valores por defecto que no afectan al valor final de la planta, dicho valor será el nominal en caso de que se conviertan en variables.

Figura 45: Pantalla de introducción de la planta en el prototipo prototipo.

Una vez rellenados los datos de la planta, en una nueva ventana se procede a introducir la incertidumbre asociada a las variables, el prototipo detecta las variables que han sido introducidas y las muestra para poder introducir los valores adecuados para cada una de ellas. Se puede observar en la figura (46).

Figura 46: Pantalla de introducción de la incertidumbre de la planta en el prototipo.

5.3 Introducir las frecuencias de diseño (ω)

El siguiente paso es introducir las frecuencias de diseño elegidas por el usuario, en este caso son las nombradas al principio de esta sección. La ventana para introducir di-

chas frecuencias de diseño ofrece cuatro posibilidades, utilizar un espaciado logaritmico, un espaciado lineal, introducirlas manualmente o leerlas desde fichero. En este caso se introducen de forma manual, se puede observar en la figura (47).

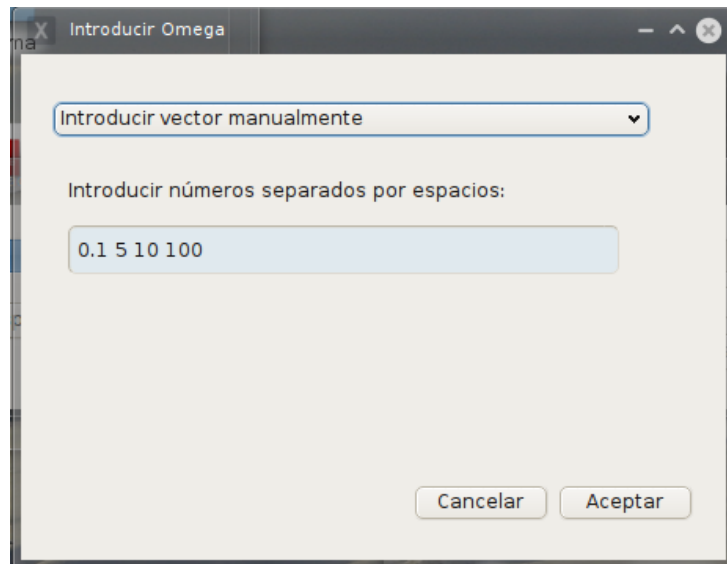


Figura 47: Pantalla de introducción de las frecuencias de diseño en el prototipo.

5.4 Diagrama de Bode

Una vez introducida la planta junto con su incertidumbre y las frecuencias de diseño se puede visualizar el *diagrama de Bode*, dicho diagrama se puede guardar en imagen.

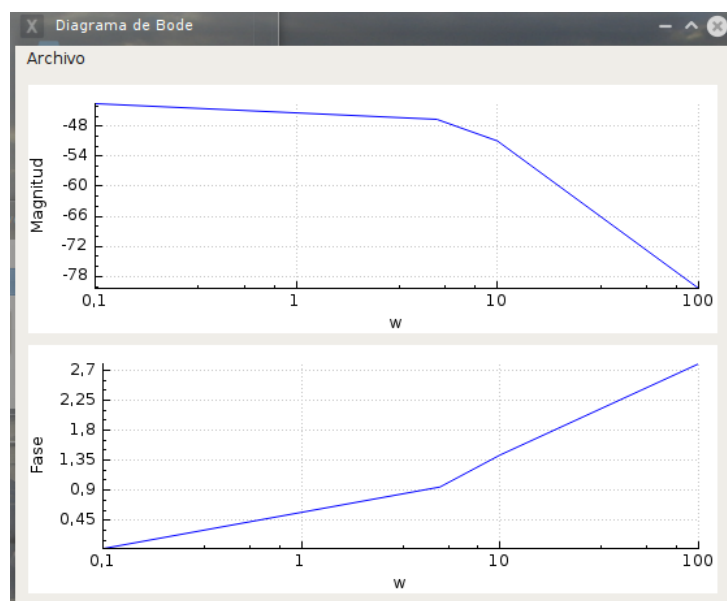


Figura 48: Pantalla del *diagrama de Bode* en el prototipo.

5.5 Cálculo de plantillas

Para realizar el cálculo de plantillas hay que introducir una serie de datos necesarios para los dos algoritmos, cálculo de plantillas y cálculo del contorno de la nube ($\epsilon - hull$), para ello se muestra la ventana mostrada en la imagen (49). En primer lugar hay que indicar cuantos valores van a tener las variables, es decir, dentro del rango de incertidumbre introducido anteriormente, en cuantos valores se va a dividir dicho rango. Se puede indicar de manera global para todas las variables o se puede indicar de manera individual para cada una de ellas. A continuación, hay que indicar el valor de ϵ que se quiere utilizar con el algoritmo de $\epsilon - hull$. Para facilitar esta tarea se puede utilizar un valor grande para ϵ e indicar que se muestre el *diagrama de Nyquist*, donde se pueden observar las distancias entre los distintos puntos y calcular el valor de ϵ más adecuado.

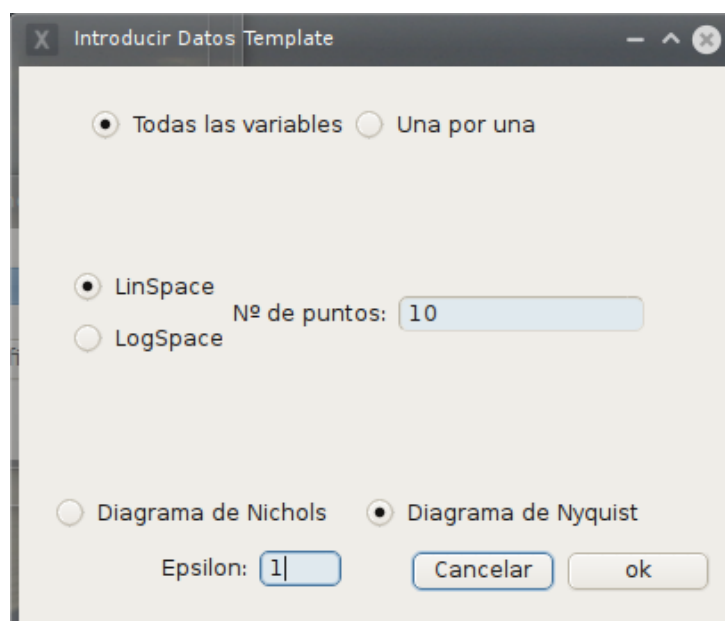


Figura 49: Pantalla de introducir datos para el algoritmo de cálculo de plantillas en el prototipo.

El resultado obtenido para el *diagrama de Nyquist* con los datos introducidos se puede observar en la figura (50).

A continuación, una vez decidido el valor adecuado para ϵ , se muestra otra vez la pantalla (49) donde se selecciona el *diagrama de Nichols* y se indica el valor adecuado para ϵ . En la imagen (51) se puede observar el resultado del cálculo de plantillas y en la imagen (52) el resultado de calcular la envolvente de dichas plantillas. Como se muestra en dicha imagen, en el contorno de las plantillas resultantes existe una separación grande entre algunos puntos, dicho problema surge porque se utiliza la misma ϵ para todas las plantillas, lo cual produce que en algunas plantillas se salten puntos al estar demasiado cerca con respecto al valor de *epsilon* y con menor ángulo que el punto seleccionado. Aún así, lo importante es que el contorno de la plantilla no se deforme y por tanto influya negativamente en el cálculo de las fronteras, en este caso la forma de las plantillas no

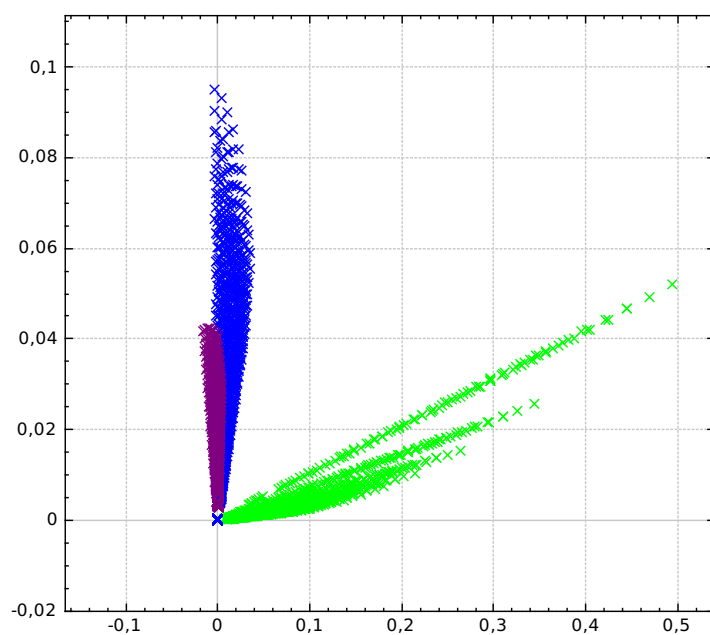


Figura 50: *Diagrama de Nyquist* de las plantillas calculado por el prototipo.

varía y por ello no supone un problema. El único inconveniente que puede generar para el cálculo de las fronteras es una pequeña pérdida de precisión. La solución a este problema sería poder introducir una ϵ adecuada para cada plantilla por separado, esta idea se puede encontrar explicada en la sección de vías futuras (6.2).

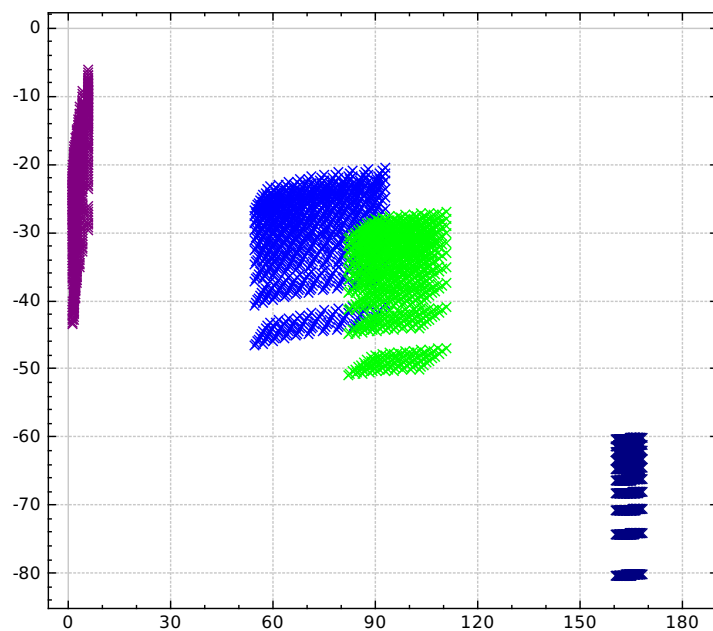


Figura 51: *Diagrama de Nichols* de las plantillas calculado por el prototipo.

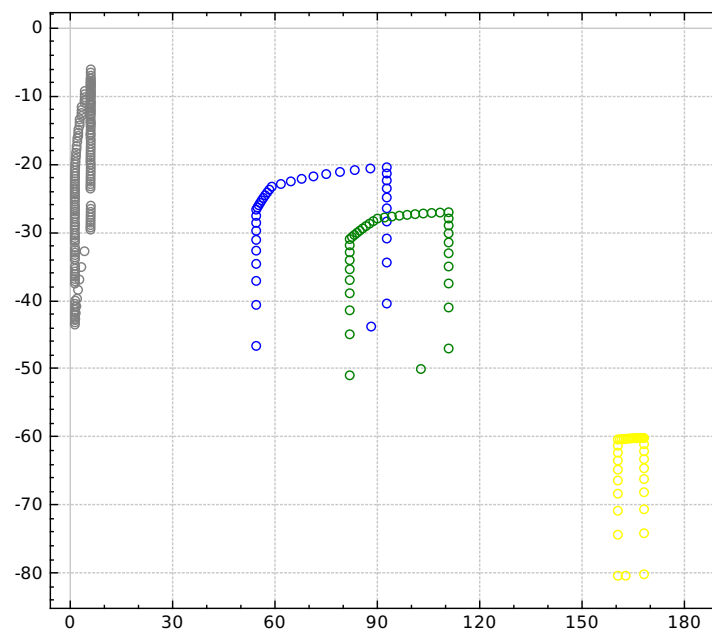


Figura 52: *Diagrama de Nichols* del contorno de las plantillas calculado por el prototipo.

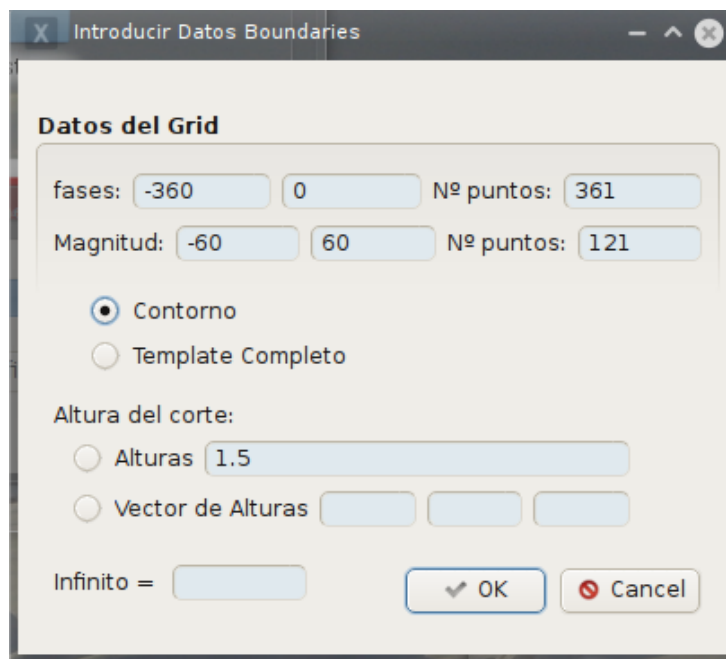
5.6 Cálculo de fronteras

La última fase de *QFT* implementada es el cálculo de fronteras, en primer lugar hay que introducir una serie de datos para poder calcular las fronteras, en la imagen (53) se pueden observar que casi todos ellos están ya por defecto y no es necesario cambiarlos para obtener un resultado correcto. En primer lugar se puede modificar el grid por defecto a utilizar en el algoritmo y el número de puntos en cada eje, en cierto momento puede ser interesante generar un grid con menos precisión pero más rápido de calcular.

A continuación se puede señalar qué se quiere utilizar para el algoritmo de cálculo de fronteras, si las plantillas completas o solo su contorno, esta opción se ha incluido para el caso de que no se pueda calcular el contorno de las plantillas de forma correcta y se decida utilizar las plantillas completas.

El siguiente dato a introducir es la altura de corte para la función *Contour* (4.6.1), se puede introducir una sola altura, o un vector de alturas para realizar sucesivos cortes y así comprobar cual puede ser el más adecuado.

Para terminar el último dato a introducir es el “valor” de infinito, como se explicó en la sección (4.7.3) el algoritmo de cálculo de fronteras puede generar en determinadas ocasiones valores infinitos en la sábana la cual se utiliza después en la función *Contour*. Como los datos de dicha sábana pueden ser exportados a un fichero de texto para ser utilizados en otros programas, los valores infinitos podrían generar problemas. Por tanto se permite indicar que valor se desea para infinito y así hacer más sencilla la compatibilidad con otros software.



Introducir Datos Boundaries

Datos del Grid

fases: -360 0 Nº puntos: 361

Magnitud: -60 60 Nº puntos: 121

☒ Contorno
☐ Template Completo

Altura del corte:

☒ Alturas 1.5
☐ Vector de Alturas

Infinito =

OK Cancel

Figura 53: Pantalla de introducir datos para el algoritmo de cálculo de fronteras en el prototipo.

Una vez introducidos los datos necesarios para el algoritmo de cálculo de fronteras, el resultado que se obtiene se puede observar en la figura (54).

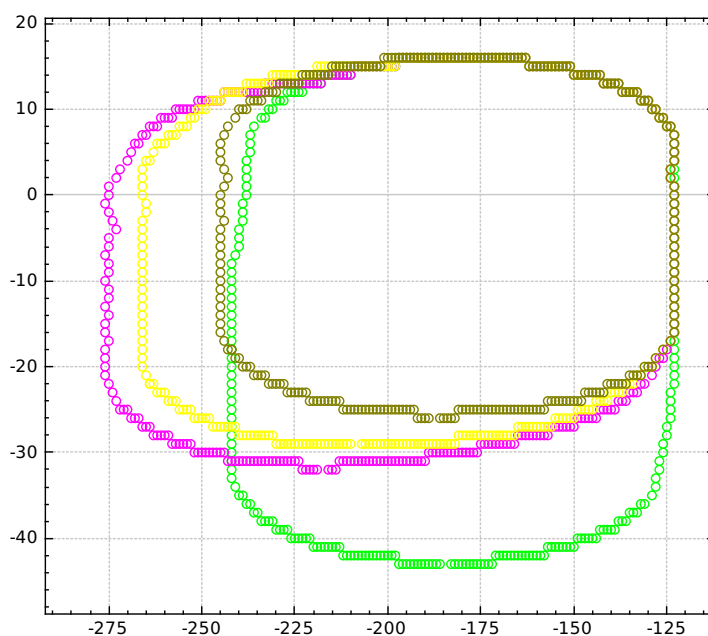


Figura 54: *Diagrama de Nichols* con las fronteras calculadas por el prototipo.

6 Conclusiones y vías futuras

6.1 Conclusiones

En este proyecto se ha abordado el estudio y desarrollo de un prototipo inicial que implemente las primeras fases de la técnica de diseño de controladores robustos *QFT*. La finalidad de dicho prototipo es proveer al grupo de investigación de *Informática Industrial* del departamento de *Informática y Sistemas* un software orientado a la investigación que aglutine las distintas investigaciones realizadas hasta el momento.

Como paso previo a la implementación del prototipo en la sección (3) se hizo un estudio y especificación de los requisitos del software que se iba a crear. Este estudio es muy importante dentro del trabajo realizado, dado que deja una base firme para poder continuar el desarrollo del prototipo en el futuro. En esta especificación se muestran los “planos” del software implementado posteriormente, junto con vías futuras de ampliación.

En la sección (4) se explican las diferentes partes que componen el prototipo desarrollado, en primer lugar se hizo un estudio de la plataforma más adecuada para el desarrollo del prototipo (4.1), buscando principalmente un lenguaje de programación rápido y que el prototipo resultante fuera multiplataforma. En este sentido, el primer objetivo se ha conseguido parcialmente, la interactividad del usuario con la *GUI* del software muestra una fluidez adecuada, los algoritmos de cálculo de plantillas y cálculo de la envolvente de las plantillas, muestra un tiempo de CPU casi instantáneo y no se percibe el tiempo de cálculo por parte del usuario. En cambio del algoritmo de cálculo de fronteras muestra un rendimiento peor, para el ejemplo desarrollado en la sección de pruebas (5) tarda aproximadamente 5 segundos en acabar. El desarrollo de estos algoritmos en el proyecto solo ha incluido su correcta implementación y no la optimización del código, en la sección de vías futuras (6.2) se explica que soluciones se le pueden dar a este problema de cara a mejoras futuras.

En la sección (4.3) se explica el sistema de guardado desarrollado para poder salvar los datos introducidos y calculados por el usuario, el guardado no es completo, dado que de los datos introducidos por el usuario solo guarda la planta junto con su incertidumbre y las frecuencias de diseño. Los demás datos que se introducen para cada algoritmo concreto no se guardan, se guardan los resultados calculados pero para poder repetir los cálculos habría que modificar el sistema de guardado para que fuera completo.

A continuación en la sección (4.6) se explican las librerías utilizadas para el desarrollo del prototipo, las dos principales son la librería *Contour* (4.6.1) y la librería *QCustomPlot* (4.6.2). De la primera se han portado a *Qt* todos los tipos de datos básicos que utiliza para que el prototipo fuera multiplataforma y funcionara en 32 y 64 bits, pero los tipos de datos abstractos, como pueden ser listas y vectores, no se han portado. El funcionamiento de la librería es correcto y da los resultados adecuados, por ello no se vio la necesidad de hacer estas modificaciones, que pueden quedar para vía futura.

En cuanto a la segunda librería, supuso una gran mejoría en la producción de los gráficos necesarios para el proyecto, las facilidades que provee fueron muy útiles para realizar de forma rápida dichas ventanas gráficas. La versión utilizada era la última disponible

cuando se incluyó en el prototipo, ahora mismo hay nuevas versiones las cuales puede ser interesante explorar que mejoras ofrecen por si puede resultar adecuado en el futuro migrar a la nueva versión.

Para terminar con el prototipo, en la sección (4.7) se explican los diferentes algoritmos implementados, esta es la sección fundamental del proyecto, puesto que dichos algoritmos constituyen el trabajo más importante realizado. En general se han cumplido los objetivos propuestos, que eran conseguir un prototipo funcional hasta el cálculo de fronteras. El estudio y comprensión de estos algoritmos fue largo y tedioso, dado que tienen un funcionamiento complicado, fueron muchas las horas dedicadas para este propósito y por ello las explicaciones contenidas en la sección (4.7) son amplias pues buscaban reflejar todo el trabajo realizado y servir de ayuda para futuras personas que quieran investigar el funcionamiento de dichos algoritmos.

Para terminar se ha desarrollado una sección de pruebas y manual de usuario (5) en el cual se ha explicado y probado el funcionamiento del prototipo realizado, comprobando su correcto funcionamiento y dejando un manual de usuario para quien lo necesite.

6.2 Vías futuras

Las vías futuras de este proyecto son innumerables debido a que, como se ha comentado en secciones anteriores, es un proyecto de investigación abierto a muchas posibilidades. De manera obvia hay vías futuras que son más inmediatas, en la sección de estudio de requisitos (3) se han propuesto diferentes opciones futuras en cuestiones concretas en la ampliación del prototipo. En esta sección se plantean las vías futuras de manera general, sin entrar en detalles de bajo nivel, las principales son las siguientes:

- El prototipo implementado tiene carencias, no solo en robustez y estabilidad, sino también en opciones que no han sido implementadas, muchas de ellas están reflejadas en el estudio de requisitos (3). Otras fueron analizadas durante la planificación y a lo largo del desarrollo del proyecto, fueron ideas interesantes que surgieron, pero que se dejaron como vías futuras.
- Mejora de los algoritmos implementados, en este prototipo se han implementado tres algoritmos principales:
 - Cálculo de plantillas: El algoritmo implementado (4.7.1) resuelve el cálculo de plantillas por fuerza bruta, es decir, calculando toda la variabilidad de una planta asegurándose así encontrar siempre la solución completa. Obviamente este tipo de algoritmos es computacionalmente muy lento, como se explicó en la sección introducción (1.2.3) el cálculo de plantillas solo está resuelto completamente con un algoritmo de fuerza bruta. Pero si se analiza la planta a estudiar, se puede determinar la aplicabilidad de algoritmos específicos para según qué tipo de plantas, con costes computacionales significativamente menores.
 - Cálculo del contorno de las plantillas: El algoritmo implementado en el prototipo (4.7.2) es conocido como ϵ - *hull* es un algoritmo similar a los de fuerza bruta en tanto que comprueba todas las posibilidades para alcanzar su solución. Se puede usar otros algoritmos, programarlos y añadirlos a *QFTbx* es

- una vía futura. Si nos centramos en el algoritmo de $\epsilon - hull$ el usuario debe introducir el valor de ϵ que es dependiente del tipo de plantilla, sobretodo de la separación entre los distintos puntos. Este valor se utiliza de manera constante durante todo el cálculo del contorno, sería muy interesante el poder determinar este valor de manera automática y que sea adaptativo según en la parte de la plantilla que se encuentre el algoritmo.
- Cálculo de fronteras: El algoritmo implementado (4.7.3) utiliza una rejilla introducida por el usuario, usualmente se utiliza los valores por defecto, que van de uno en uno en el eje de las fases de $[0, -360]$ y en el eje de la ganancia de $[60, -60]$. Para cada frecuencia de diseño debe recorrerse esta rejilla en un bucle doble, para cada posición debe recorrerse la plantilla de la planta y realizar diversos cálculos con números complejos. La cantidad de operaciones que tiene que realizar el procesador es enorme, y con ello el tiempo que tarda el resolver dicho algoritmo, una de las mejoras que reducirían enormemente el tiempo de ejecución sería determinar de manera automática qué tamaño de rejilla es necesario. Generalmente los cálculos útiles para la solución se realizan en determinadas zonas concretas, pero esto no se puede saber a priori, con lo cual una manera automática que analizando la planta determinada la rejilla más adecuada sería una mejora muy importante. En este proyecto solo se ha implementado el cálculo de las fronteras de estabilidad, en la introducción (sección (1.2.4)) están el resto de especificaciones principales de *QFT*, todas ellas se pueden resolver ampliando el algoritmo implementado, su estudio teórico se puede encontrar en el artículo [MORENO ET AL. [2006]], ahí se explican como implementar el resto las especificaciones clásicas de *QFT*.
 - Como se ha explicado en el punto anterior los algoritmos desarrollados consumen mucho tiempo de CPU en cuanto que tienen que realizar gran cantidad de cálculos, pero se puede observar como todos esos cálculos se repiten para cada frecuencia de diseño, es decir, si el usuario introduce 7 frecuencias de diseño, los cálculos se tendrán que repetir con las 7 y de manera totalmente independiente unas de otras. Con lo cual una forma de conseguir una importante optimización sería paralelizar los cálculos para cada frecuencia de diseño, como son completamente independiente no hay dependencias entre los diferentes cálculos. Con ello se conseguiría aprovechar las CPUs modernas de mas de un núcleo y además sería el propio sistema operativo quien gestionaría los distintos hilos con la consiguiente mejoría en el rendimiento.
 - Ampliando lo explicado en el punto anterior y centrándose en los tipos de cálculos que realizan los distintos algoritmos del prototipo, todos ellos trabajan con números complejos en coma flotante. El cálculo con este tipo de números es muy costoso desde el punto de vista computacional, una vía de estudio futura sería incorporar al prototipo código que utilizara las instrucciones específicas de bajo nivel que ofrecen los procesadores para el cálculo con números reales. El problema de esta perspectiva es la dependencia que se establecería entre el código y el procesador, dado que cada familia de procesadores suele variar este tipo de operaciones, un punto de vista interesante sería desarrollar esta opción para las familias más comunes dejando para el resto código genérico. Otra forma similar de mejorar el rendimiento sería utilizar las GPUs de los equipos, este tipo de procesadores están especializados en el cálculo en punto flotante para el desarrollo de gráficos. Por tanto investigar cómo explotar

las opciones que ofrecen las GPU para el cálculo en punto flotante es una vía de estudio muy interesante.

- En las fases iniciales de desarrollo del proyecto se estudio la posibilidad de poder leer datos guardados en ficheros con extensión *.m*, este tipo de extensión corresponde al lenguaje *Matlab*, uno de los más utilizados en esta disciplina. Este tipo de ficheros son binarios, con lo cual la dificultad de manipularlos era alta, para ello se estudio la librería *Matio*²⁰ (MAT File I/O Library), pero se decidió que no entraba en el ámbito del proyecto implementar esta funcionalidad y por tanto queda como vía futura.
- Lo explicado hasta ahora en esta sección se centra en las vías futuras de la parte del prototipo ya implementado, como se explicó en la sección de diseño del prototipo (4) solo se han implementado las primeras fases hasta el cálculo de fronteras, en la sección de introducción a QFT (1.2) se explican todas las fases. La síntesis automática del controlador (1.2.6) es la fase más complicada, la cual está en proceso de investigación, el objetivo final es poder implementar un ajuste automático del lazo, es decir, ajuste del controlador sin necesidad de intervención del usuario. Actualmente distintas investigaciones han llegado a soluciones parciales que ayudan al usuario a sintetizar el controlador, por ello desarrollar el prototipo para incorporar estas investigaciones es una de las vías futuras más importantes. Completar el prototipo para que cubriera, aunque fuera de forma básica, todas las fases de QFT permitiría que pudiera ser usado para desarrollar un controlador de forma completa. Las demás fases que componen QFT que no han sido implementadas son más sencillas que la síntesis del controlador, aún así el trabajo a dedicarle es importante para implementarlas, terminarlás es una vía de desarrollo futuro importante.

²⁰Página de descarga <http://sourceforge.net/projects/matio/> Último acceso (7/09/2013)

A Anejo 1: GNU GENERAL PUBLIC LICENSE

Version 3, 29 June 2007

Copyright © 2007 Free Software Foundation, Inc. <http://fsf.org/>

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

Preamble

The GNU General Public License is a free, copyleft license for software and other kinds of works.

The licenses for most software and other practical works are designed to take away your freedom to share and change the works. By contrast, the GNU General Public License is intended to guarantee your freedom to share and change all versions of a program—to make sure it remains free software for all its users. We, the Free Software Foundation, use the GNU General Public License for most of our software; it applies also to any other work released this way by its authors. You can apply it to your programs, too.

When we speak of free software, we are referring to freedom, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for them if you wish), that you receive source code or can get it if you want it, that you can change the software or use pieces of it in new free programs, and that you know you can do these things.

To protect your rights, we need to prevent others from denying you these rights or asking you to surrender the rights. Therefore, you have certain responsibilities if you distribute copies of the software, or if you modify it: responsibilities to respect the freedom of others.

For example, if you distribute copies of such a program, whether gratis or for a fee, you must pass on to the recipients the same freedoms that you received. You must make sure that they, too, receive or can get the source code. And you must show them these terms so they know their rights.

Developers that use the GNU GPL protect your rights with two steps: (1) assert copyright on the software, and (2) offer you this License giving you legal permission to copy, distribute and/or modify it.

For the developers' and authors' protection, the GPL clearly explains that there is no warranty for this free software. For both users' and authors' sake, the GPL requires that modified versions be marked as changed, so that their problems will not be attributed erroneously to authors of previous versions.

Some devices are designed to deny users access to install or run modified versions of the software inside them, although the manufacturer can do so. This is fundamentally incompatible with the aim of protecting users' freedom to change the software. The systematic pattern of such abuse occurs in the area of products for individuals to use, which is precisely where it is most unacceptable. Therefore, we have designed this version of the GPL to prohibit the practice for those products. If such problems arise substantially in other domains, we stand ready to extend this provision to those domains in future versions of the GPL, as needed to protect the freedom of users.

Finally, every program is threatened constantly by software patents. States should not allow patents to restrict development and use of software on general-purpose computers,

but in those that do, we wish to avoid the special danger that patents applied to a free program could make it effectively proprietary. To prevent this, the GPL assures that patents cannot be used to render the program non-free.

The precise terms and conditions for copying, distribution and modification follow.

TERMS AND CONDITIONS

0. Definitions.

“This License” refers to version 3 of the GNU General Public License.

“Copyright” also means copyright-like laws that apply to other kinds of works, such as semiconductor masks.

“The Program” refers to any copyrightable work licensed under this License. Each licensee is addressed as “you”. “Licensees” and “recipients” may be individuals or organizations.

To “modify” a work means to copy from or adapt all or part of the work in a fashion requiring copyright permission, other than the making of an exact copy. The resulting work is called a “modified version” of the earlier work or a work “based on” the earlier work.

A “covered work” means either the unmodified Program or a work based on the Program.

To “propagate” a work means to do anything with it that, without permission, would make you directly or secondarily liable for infringement under applicable copyright law, except executing it on a computer or modifying a private copy. Propagation includes copying, distribution (with or without modification), making available to the public, and in some countries other activities as well.

To “convey” a work means any kind of propagation that enables other parties to make or receive copies. Mere interaction with a user through a computer network, with no transfer of a copy, is not conveying.

An interactive user interface displays “Appropriate Legal Notices” to the extent that it includes a convenient and prominently visible feature that (1) displays an appropriate copyright notice, and (2) tells the user that there is no warranty for the work (except to the extent that warranties are provided), that licensees may convey the work under this License, and how to view a copy of this License. If the interface presents a list of user commands or options, such as a menu, a prominent item in the list meets this criterion.

1. Source Code.

The “source code” for a work means the preferred form of the work for making modifications to it. “Object code” means any non-source form of a work.

A “Standard Interface” means an interface that either is an official standard defined by a recognized standards body, or, in the case of interfaces specified for a particular programming language, one that is widely used among developers working in that language.

The “System Libraries” of an executable work include anything, other than the work as a whole, that (a) is included in the normal form of packaging a Major Component,

but which is not part of that Major Component, and (b) serves only to enable use of the work with that Major Component, or to implement a Standard Interface for which an implementation is available to the public in source code form. A “Major Component”, in this context, means a major essential component (kernel, window system, and so on) of the specific operating system (if any) on which the executable work runs, or a compiler used to produce the work, or an object code interpreter used to run it.

The “Corresponding Source” for a work in object code form means all the source code needed to generate, install, and (for an executable work) run the object code and to modify the work, including scripts to control those activities. However, it does not include the work’s System Libraries, or general-purpose tools or generally available free programs which are used unmodified in performing those activities but which are not part of the work. For example, Corresponding Source includes interface definition files associated with source files for the work, and the source code for shared libraries and dynamically linked subprograms that the work is specifically designed to require, such as by intimate data communication or control flow between those subprograms and other parts of the work.

The Corresponding Source need not include anything that users can regenerate automatically from other parts of the Corresponding Source.

The Corresponding Source for a work in source code form is that same work.

2. Basic Permissions.

All rights granted under this License are granted for the term of copyright on the Program, and are irrevocable provided the stated conditions are met. This License explicitly affirms your unlimited permission to run the unmodified Program. The output from running a covered work is covered by this License only if the output, given its content, constitutes a covered work. This License acknowledges your rights of fair use or other equivalent, as provided by copyright law.

You may make, run and propagate covered works that you do not convey, without conditions so long as your license otherwise remains in force. You may convey covered works to others for the sole purpose of having them make modifications exclusively for you, or provide you with facilities for running those works, provided that you comply with the terms of this License in conveying all material for which you do not control copyright. Those thus making or running the covered works for you must do so exclusively on your behalf, under your direction and control, on terms that prohibit them from making any copies of your copyrighted material outside their relationship with you.

Conveying under any other circumstances is permitted solely under the conditions stated below. Sublicensing is not allowed; section 10 makes it unnecessary.

3. Protecting Users’ Legal Rights From Anti-Circumvention Law.

No covered work shall be deemed part of an effective technological measure under any applicable law fulfilling obligations under article 11 of the WIPO copyright treaty adopted on 20 December 1996, or similar laws prohibiting or restricting circumvention of such measures.

When you convey a covered work, you waive any legal power to forbid circumvention of technological measures to the extent such circumvention is effected by exercising rights under this License with respect to the covered work, and you disclaim any intention to limit operation or modification of the work as a means of enforcing, against the work's users, your or third parties' legal rights to forbid circumvention of technological measures.

4. Conveying Verbatim Copies.

You may convey verbatim copies of the Program's source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice; keep intact all notices stating that this License and any non-permissive terms added in accord with section 7 apply to the code; keep intact all notices of the absence of any warranty; and give all recipients a copy of this License along with the Program.

You may charge any price or no price for each copy that you convey, and you may offer support or warranty protection for a fee.

5. Conveying Modified Source Versions.

You may convey a work based on the Program, or the modifications to produce it from the Program, in the form of source code under the terms of section 4, provided that you also meet all of these conditions:

- (a) The work must carry prominent notices stating that you modified it, and giving a relevant date.
- (b) The work must carry prominent notices stating that it is released under this License and any conditions added under section 7. This requirement modifies the requirement in section 4 to "keep intact all notices".
- (c) You must license the entire work, as a whole, under this License to anyone who comes into possession of a copy. This License will therefore apply, along with any applicable section 7 additional terms, to the whole of the work, and all its parts, regardless of how they are packaged. This License gives no permission to license the work in any other way, but it does not invalidate such permission if you have separately received it.
- (d) If the work has interactive user interfaces, each must display Appropriate Legal Notices; however, if the Program has interactive interfaces that do not display Appropriate Legal Notices, your work need not make them do so.

A compilation of a covered work with other separate and independent works, which are not by their nature extensions of the covered work, and which are not combined with it such as to form a larger program, in or on a volume of a storage or distribution medium, is called an "aggregate" if the compilation and its resulting copyright are not used to limit the access or legal rights of the compilation's users beyond what the individual works permit. Inclusion of a covered work in an aggregate does not cause this License to apply to the other parts of the aggregate.

6. Conveying Non-Source Forms.

You may convey a covered work in object code form under the terms of sections 4 and 5, provided that you also convey the machine-readable Corresponding Source under the terms of this License, in one of these ways:

- (a) Convey the object code in, or embodied in, a physical product (including a physical distribution medium), accompanied by the Corresponding Source fixed on a durable physical medium customarily used for software interchange.
- (b) Convey the object code in, or embodied in, a physical product (including a physical distribution medium), accompanied by a written offer, valid for at least three years and valid for as long as you offer spare parts or customer support for that product model, to give anyone who possesses the object code either (1) a copy of the Corresponding Source for all the software in the product that is covered by this License, on a durable physical medium customarily used for software interchange, for a price no more than your reasonable cost of physically performing this conveying of source, or (2) access to copy the Corresponding Source from a network server at no charge.
- (c) Convey individual copies of the object code with a copy of the written offer to provide the Corresponding Source. This alternative is allowed only occasionally and noncommercially, and only if you received the object code with such an offer, in accord with subsection 6b.
- (d) Convey the object code by offering access from a designated place (gratis or for a charge), and offer equivalent access to the Corresponding Source in the same way through the same place at no further charge. You need not require recipients to copy the Corresponding Source along with the object code. If the place to copy the object code is a network server, the Corresponding Source may be on a different server (operated by you or a third party) that supports equivalent copying facilities, provided you maintain clear directions next to the object code saying where to find the Corresponding Source. Regardless of what server hosts the Corresponding Source, you remain obligated to ensure that it is available for as long as needed to satisfy these requirements.
- (e) Convey the object code using peer-to-peer transmission, provided you inform other peers where the object code and Corresponding Source of the work are being offered to the general public at no charge under subsection 6d.

A separable portion of the object code, whose source code is excluded from the Corresponding Source as a System Library, need not be included in conveying the object code work.

A “User Product” is either (1) a “consumer product”, which means any tangible personal property which is normally used for personal, family, or household purposes, or (2) anything designed or sold for incorporation into a dwelling. In determining whether a product is a consumer product, doubtful cases shall be resolved in favor of coverage. For a particular product received by a particular user, “normally used” refers to a typical or common use of that class of product, regardless of the status of the particular user or of the way in which the particular user actually uses, or expects or is expected to use, the product. A product is a consumer product regardless of whether the product has substantial commercial, industrial or non-consumer uses, unless such uses represent the only significant mode of use of the product.

“Installation Information” for a User Product means any methods, procedures, authorization keys, or other information required to install and execute modified versions of a covered work in that User Product from a modified version of its Corresponding Source. The information must suffice to ensure that the continued functioning of the modified object code is in no case prevented or interfered with solely because modification has been made.

If you convey an object code work under this section in, or with, or specifically for use in, a User Product, and the conveying occurs as part of a transaction in which the right of possession and use of the User Product is transferred to the recipient in perpetuity or for a fixed term (regardless of how the transaction is characterized), the Corresponding Source conveyed under this section must be accompanied by the Installation Information. But this requirement does not apply if neither you nor any third party retains the ability to install modified object code on the User Product (for example, the work has been installed in ROM).

The requirement to provide Installation Information does not include a requirement to continue to provide support service, warranty, or updates for a work that has been modified or installed by the recipient, or for the User Product in which it has been modified or installed. Access to a network may be denied when the modification itself materially and adversely affects the operation of the network or violates the rules and protocols for communication across the network.

Corresponding Source conveyed, and Installation Information provided, in accord with this section must be in a format that is publicly documented (and with an implementation available to the public in source code form), and must require no special password or key for unpacking, reading or copying.

7. Additional Terms.

“Additional permissions” are terms that supplement the terms of this License by making exceptions from one or more of its conditions. Additional permissions that are applicable to the entire Program shall be treated as though they were included in this License, to the extent that they are valid under applicable law. If additional permissions apply only to part of the Program, that part may be used separately under those permissions, but the entire Program remains governed by this License without regard to the additional permissions.

When you convey a copy of a covered work, you may at your option remove any additional permissions from that copy, or from any part of it. (Additional permissions may be written to require their own removal in certain cases when you modify the work.) You may place additional permissions on material, added by you to a covered work, for which you have or can give appropriate copyright permission.

Notwithstanding any other provision of this License, for material you add to a covered work, you may (if authorized by the copyright holders of that material) supplement the terms of this License with terms:

- (a) Disclaiming warranty or limiting liability differently from the terms of sections 15 and 16 of this License; or
- (b) Requiring preservation of specified reasonable legal notices or author attributions in that material or in the Appropriate Legal Notices displayed by works

- containing it; or
- (c) Prohibiting misrepresentation of the origin of that material, or requiring that modified versions of such material be marked in reasonable ways as different from the original version; or
- (d) Limiting the use for publicity purposes of names of licensors or authors of the material; or
- (e) Declining to grant rights under trademark law for use of some trade names, trademarks, or service marks; or
- (f) Requiring indemnification of licensors and authors of that material by anyone who conveys the material (or modified versions of it) with contractual assumptions of liability to the recipient, for any liability that these contractual assumptions directly impose on those licensors and authors.

All other non-permissive additional terms are considered “further restrictions” within the meaning of section 10. If the Program as you received it, or any part of it, contains a notice stating that it is governed by this License along with a term that is a further restriction, you may remove that term. If a license document contains a further restriction but permits relicensing or conveying under this License, you may add to a covered work material governed by the terms of that license document, provided that the further restriction does not survive such relicensing or conveying.

If you add terms to a covered work in accord with this section, you must place, in the relevant source files, a statement of the additional terms that apply to those files, or a notice indicating where to find the applicable terms.

Additional terms, permissive or non-permissive, may be stated in the form of a separately written license, or stated as exceptions; the above requirements apply either way.

8. Termination.

You may not propagate or modify a covered work except as expressly provided under this License. Any attempt otherwise to propagate or modify it is void, and will automatically terminate your rights under this License (including any patent licenses granted under the third paragraph of section 11).

However, if you cease all violation of this License, then your license from a particular copyright holder is reinstated (a) provisionally, unless and until the copyright holder explicitly and finally terminates your license, and (b) permanently, if the copyright holder fails to notify you of the violation by some reasonable means prior to 60 days after the cessation.

Moreover, your license from a particular copyright holder is reinstated permanently if the copyright holder notifies you of the violation by some reasonable means, this is the first time you have received notice of violation of this License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice.

Termination of your rights under this section does not terminate the licenses of parties who have received copies or rights from you under this License. If you

rights have been terminated and not permanently reinstated, you do not qualify to receive new licenses for the same material under section 10.

9. Acceptance Not Required for Having Copies.

You are not required to accept this License in order to receive or run a copy of the Program. Ancillary propagation of a covered work occurring solely as a consequence of using peer-to-peer transmission to receive a copy likewise does not require acceptance. However, nothing other than this License grants you permission to propagate or modify any covered work. These actions infringe copyright if you do not accept this License. Therefore, by modifying or propagating a covered work, you indicate your acceptance of this License to do so.

10. Automatic Licensing of Downstream Recipients.

Each time you convey a covered work, the recipient automatically receives a license from the original licensors, to run, modify and propagate that work, subject to this License. You are not responsible for enforcing compliance by third parties with this License.

An “entity transaction” is a transaction transferring control of an organization, or substantially all assets of one, or subdividing an organization, or merging organizations. If propagation of a covered work results from an entity transaction, each party to that transaction who receives a copy of the work also receives whatever licenses to the work the party’s predecessor in interest had or could give under the previous paragraph, plus a right to possession of the Corresponding Source of the work from the predecessor in interest, if the predecessor has it or can get it with reasonable efforts.

You may not impose any further restrictions on the exercise of the rights granted or affirmed under this License. For example, you may not impose a license fee, royalty, or other charge for exercise of rights granted under this License, and you may not initiate litigation (including a cross-claim or counterclaim in a lawsuit) alleging that any patent claim is infringed by making, using, selling, offering for sale, or importing the Program or any portion of it.

11. Patents.

A “contributor” is a copyright holder who authorizes use under this License of the Program or a work on which the Program is based. The work thus licensed is called the contributor’s “contributor version”.

A contributor’s “essential patent claims” are all patent claims owned or controlled by the contributor, whether already acquired or hereafter acquired, that would be infringed by some manner, permitted by this License, of making, using, or selling its contributor version, but do not include claims that would be infringed only as a consequence of further modification of the contributor version. For purposes of this definition, “control” includes the right to grant patent sublicenses in a manner consistent with the requirements of this License.

Each contributor grants you a non-exclusive, worldwide, royalty-free patent license under the contributor’s essential patent claims, to make, use, sell, offer for sale,

import and otherwise run, modify and propagate the contents of its contributor version.

In the following three paragraphs, a “patent license” is any express agreement or commitment, however denominated, not to enforce a patent (such as an express permission to practice a patent or covenant not to sue for patent infringement). To “grant” such a patent license to a party means to make such an agreement or commitment not to enforce a patent against the party.

If you convey a covered work, knowingly relying on a patent license, and the Corresponding Source of the work is not available for anyone to copy, free of charge and under the terms of this License, through a publicly available network server or other readily accessible means, then you must either (1) cause the Corresponding Source to be so available, or (2) arrange to deprive yourself of the benefit of the patent license for this particular work, or (3) arrange, in a manner consistent with the requirements of this License, to extend the patent license to downstream recipients. “Knowingly relying” means you have actual knowledge that, but for the patent license, your conveying the covered work in a country, or your recipient’s use of the covered work in a country, would infringe one or more identifiable patents in that country that you have reason to believe are valid.

If, pursuant to or in connection with a single transaction or arrangement, you convey, or propagate by procuring conveyance of, a covered work, and grant a patent license to some of the parties receiving the covered work authorizing them to use, propagate, modify or convey a specific copy of the covered work, then the patent license you grant is automatically extended to all recipients of the covered work and works based on it.

A patent license is “discriminatory” if it does not include within the scope of its coverage, prohibits the exercise of, or is conditioned on the non-exercise of one or more of the rights that are specifically granted under this License. You may not convey a covered work if you are a party to an arrangement with a third party that is in the business of distributing software, under which you make payment to the third party based on the extent of your activity of conveying the work, and under which the third party grants, to any of the parties who would receive the covered work from you, a discriminatory patent license (a) in connection with copies of the covered work conveyed by you (or copies made from those copies), or (b) primarily for and in connection with specific products or compilations that contain the covered work, unless you entered into that arrangement, or that patent license was granted, prior to 28 March 2007.

Nothing in this License shall be construed as excluding or limiting any implied license or other defenses to infringement that may otherwise be available to you under applicable patent law.

12. No Surrender of Others’ Freedom.

If conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot convey a covered work so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not convey it at all. For example, if you agree

to terms that obligate you to collect a royalty for further conveying from those to whom you convey the Program, the only way you could satisfy both those terms and this License would be to refrain entirely from conveying the Program.

13. Use with the GNU Affero General Public License.

Notwithstanding any other provision of this License, you have permission to link or combine any covered work with a work licensed under version 3 of the GNU Affero General Public License into a single combined work, and to convey the resulting work. The terms of this License will continue to apply to the part which is the covered work, but the special requirements of the GNU Affero General Public License, section 13, concerning interaction through a network will apply to the combination as such.

14. Revised Versions of this License.

The Free Software Foundation may publish revised and/or new versions of the GNU General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Program specifies that a certain numbered version of the GNU General Public License “or any later version” applies to it, you have the option of following the terms and conditions either of that numbered version or of any later version published by the Free Software Foundation. If the Program does not specify a version number of the GNU General Public License, you may choose any version ever published by the Free Software Foundation.

If the Program specifies that a proxy can decide which future versions of the GNU General Public License can be used, that proxy’s public statement of acceptance of a version permanently authorizes you to choose that version for the Program.

Later license versions may give you additional or different permissions. However, no additional obligations are imposed on any author or copyright holder as a result of your choosing to follow a later version.

15. Disclaimer of Warranty.

THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM “AS IS” WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

16. Limitation of Liability.

IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY

WHO MODIFIES AND/OR CONVEYS THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

17. Interpretation of Sections 15 and 16.

If the disclaimer of warranty and limitation of liability provided above cannot be given local legal effect according to their terms, reviewing courts shall apply local law that most closely approximates an absolute waiver of all civil liability in connection with the Program, unless a warranty or assumption of liability accompanies a copy of the Program in return for a fee.

END OF TERMS AND CONDITIONS

How to Apply These Terms to Your New Programs

If you develop a new program, and you want it to be of the greatest possible use to the public, the best way to achieve this is to make it free software which everyone can redistribute and change under these terms.

To do so, attach the following notices to the program. It is safest to attach them to the start of each source file to most effectively state the exclusion of warranty; and each file should have at least the “copyright” line and a pointer to where the full notice is found.

```
<one line to give the program's name and a brief idea of what it does.>
```

```
Copyright (C) <textyear> <name of author>
```

```
This program is free software: you can redistribute it and/or modify  
it under the terms of the GNU General Public License as published by  
the Free Software Foundation, either version 3 of the License, or  
(at your option) any later version.
```

```
This program is distributed in the hope that it will be useful,  
but WITHOUT ANY WARRANTY; without even the implied warranty of  
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the  
GNU General Public License for more details.
```

```
You should have received a copy of the GNU General Public License  
along with this program. If not, see <http://www.gnu.org/licenses/>.
```

Also add information on how to contact you by electronic and paper mail.

If the program does terminal interaction, make it output a short notice like this when it starts in an interactive mode:

```
<program> Copyright (C) <year> <name of author>
```

This program comes with ABSOLUTELY NO WARRANTY; for details type ‘show w’.
This is free software, and you are welcome to redistribute it
under certain conditions; type ‘show c’ for details.

The hypothetical commands **show w** and **show c** should show the appropriate parts of the General Public License. Of course, your program’s commands might be different; for a GUI interface, you would use an “about box”.

You should also get your employer (if you work as a programmer) or school, if any, to sign a “copyright disclaimer” for the program, if necessary. For more information on this, and how to apply and follow the GNU GPL, see <http://www.gnu.org/licenses/>.

The GNU General Public License does not permit incorporating your program into proprietary programs. If your program is a subroutine library, you may consider it more useful to permit linking proprietary applications with the library. If this is what you want to do, use the GNU Lesser General Public License instead of this License. But first, please read <http://www.gnu.org/philosophy/why-not-lgpl.html>.

B Anejo 2: Código de los algoritmos implementados

B.1 Algoritmo de cálculo de plantillas

```

1
2 QVector<QVector<complex<qreal> > * > * Templates::calcularTemplate
3 (Planta *planta, QVector<qreal> *omega){
4
5     QVector <qreal> * denominador = new QVector <qreal> ();
6     QVector <qreal> * numerador = new QVector <qreal> ();
7     QVector<QVector<qreal> * > * variables = new QVector <QVector <qreal> * >
8         ();
9
10    qint32 lonNume = planta->getNumerador()->size();
11    qint32 lonDeno = planta->getDenominador()->size();
12    qint32 contador[lonNume+lonDeno+1];
13
14    variables->reserve(lonNume + lonDeno);
15    numerador->reserve(lonNume);
16    denominador->reserve(lonDeno);
17
18    qint32 i = 0;
19
20    combinaciones = 1;
21
22    for (i = 0; i < lonNume; i++){
23        QVector <qreal> * vector = getVariables(planta->getNumerador()->at(i));
24        combinaciones *= vector->size();
25        variables->insert(i,vector);
26        numerador->insert(i,vector->at(0));
27        contador[i] = 0;
28    }
29
30    for (qint32 j = i, i = 0; i < lonDeno; i++,j++){
31        QVector <qreal> * vector = getVariables(planta->getDenominador()->at(i)
32            );
33        combinaciones *= vector->size();
34        variables->insert(j,vector);
35        denominador->insert(i,vector->at(0));
36        contador[j] = 0;
37    }
38
39    QVector <qreal> * k = getVariables(planta->getK());
40    QVector <qreal> * ret = getVariables(planta->getRet());
41
42    qint32 combT = combinaciones * k->size() * ret->size();
43    QVector<QVector<complex<qreal> > * > * temCompleto = new QVector <QVector<
44        complex<qreal> > * > ();
45    temCompleto->reserve(omega->size());

```



```

44
45     for (qint32 i = 0; i < omega->size(); i++){
46         QVector <complex <qreal>> * vector = new QVector <complex <qreal>> ();
47         vector->reserve(combT);
48         temCompleto->append(vector);
49     }
50
51     for (i = 0; i < combinaciones; i++){
52
53         foreach (const qreal &kT, *k) {
54             foreach (const qreal &retT, *ret) {
55                 QVector <complex <qreal>> * pResu = planta->getPunto(numerador,
56                     denominador, omega, kT, retT);
57                 for (qint32 j = 0; j < temCompleto->size(); j++){
58                     temCompleto->at(j)->append(pResu->at(j));
59                 }
60             }
61         }
62
63         contador[0]=contador[0]+1;
64
65         for (qint32 j = 0; j < lonNume+lonDeno; j++){
66
67             if (contador[j] >= (variables->at(j)->size())){
68                 contador[j] = 0;
69                 contador[j+1] = contador[j+1]+1;
70             }
71             if (j < lonNume){
72                 numerador->replace(j, variables->at(j)->at(contador[j]));
73
74             }else {
75                 denominador->replace(j-lonNume, variables->at(j)->at(contador[j])
76                     );
77             }
78         }
79     }
80
81     variables->clear();
82     numerador->clear();
83     denominador->clear();
84     return temCompleto;
85 }
86

```

B.2 Algoritmo *e-hull* para el cálculo de contornos de plantillas

```

1     QVector <complex <qreal> > * Templates::e_hull(QVector<complex<qreal> > *temp,
2         qreal epsilon){

```

```
3   qint32 numDatos = temp->size();
4   qint32 MAXP = 3 * numDatos;
5
6   qint32 b1 = 0;
7   qreal numDe = -numeric_limits<qreal>::infinity();
8
9   for(qint32 i = 0; i < numDatos ; i++){ //calculamos que punto está más a la
10                                          //derecha y ese será el primer
11                                          punto.
12
13       qreal realC = real(temp->at(i));
14       if (realC > numDe){
15           b1 = i;
16           numDe = realC;
17       }
18   }
19
20   qint32 b2 = buscarSegundo(b1, temp, epsilon); //buscamos el segundo punto.
21
22   if (b2 < 0)
23       return NULL;
24
25   QVector <qint32> * resultado = new QVector <qint32>();
26
27   qint32 punto_previo = b1;
28   qint32 punto_actual = b2;
29
30   resultado->append(b1);
31   resultado->append(b2);
32   //a partir de aquí buscamos los demás puntos
33   qreal punto_sig = buscarSiguiete(b1,b2,temp,epsilon);
34   if (punto_sig < 0)
35       return NULL;
36
37   qint32 contador = 2;
38
39   while (b1 != punto_actual  b2 != punto_sig){
40
41       resultado->append(punto_sig);
42       contador++;
43
44       if (contador > MAXP)
45           break;
46
47       punto_previo = punto_actual;
48       punto_actual = punto_sig;
49
50       punto_sig = buscarSiguiete(punto_previo, punto_actual, temp, epsilon);
51
52       if (punto_sig < 0)
53           return NULL;
```

```
52 }
53
54
55 QVector <qint32> * aux = new QVector <qint32> ();
56 aux->reserve(resultado->size());
57
58 bool isRepetido = false;
59
60 foreach (const qint32 var1, *resultado) { //Quitamos los valores repetidos
    del vector
61     foreach (const qint32 var2, *aux) {
62         if (var1 == var2){
63             isRepetido = true;
64         }
65     }
66     if (!isRepetido){
67         aux->append(var1);
68     }
69 }
70
71 QVector <complex <qreal> > * devolver = new QVector <complex <qreal> > ();
72 devolver->reserve(aux->size());
73
74 //como estábamos guardando los índices ahora retornamos
75 //los números complejos de verdad.
76 foreach (const qint32 var, *aux) {
77     devolver->append(temp->at(var));
78 }
79
80 resultado->clear();
81 aux->clear();
82
83
84 return devolver;
85 }
86
87 qint32 Templates::buscarSegundo(qint32 b1, QVector<complex<qreal> > *cv, qreal
    epsilon){
88
89     qreal dist = 0;
90     complex <qreal> primero = cv->at(b1);
91
92     qreal fmin = numeric_limits<qreal>::infinity();
93     qint32 pmin = -1;
94     qreal aco = 0;
95     qreal dmax = 0;
96
97     complex <qreal> cvActual;
98
99     qreal fas = 0;
```

```

100
101     for (qint32 i = 0; i < cv->size(); i++){ //recorremos todo el vector de
        puntos.
102
103         cvActual = cv->at(i);
104         dist = abs(primeros - cvActual); //calculamos el valor absoluto de la
            resta.
105
106         if (dist > 0 && dist <= epsilon){ //nos quedamos con los mas pequeños.
107
108             //calculamos la phase del ángulo de la resta de complejos.
109             fas = arg (cvActual - primeros);
110
111             if (fas < 0) //si dicha fase es menor que cero le sumamos 2*PI
112                 fas += 2*M_PI;
113
114             dist = abs(cvActual - primeros);
115
116             aco = qAcos(dist / epsilon); //calculamos el arcotangente del punto
                actual dividido por epsilon
117             fas -= aco; //a la fase actual le restamos el arcotangente.
118
119             if (fas < fmin){ //vamos cual es la fase mínima y la guardamos
120                 fmin = fas;
121                 pmin = i;
122                 dmax = dist;
123
124             }else if (fas == fmin && dist > dmax){ //si son iguales guardamos
                aquella que su distancia sea mayor.
125                 pmin = i;
126                 dmax = dist;
127             }
128         }
129     }
130
131     return pmin; //retornamos el mínimo
132 }
133
134 qint32 Templates::buscarSiguiete(qint32 punto_previo, qint32 punto_actual,
135                                 QVector<complex<qreal> > *cv, qreal epsilon){
136     complex <qreal> actual = cv->at(punto_actual);
137     complex <qreal> anterior = cv->at(punto_previo);
138
139     qreal aco2 = qAcos(abs(anterior-actual) / epsilon);
140
141     qreal fasActual = 0;
142
143     qreal aco1Actual = 0;
144     qreal dmax = 0;
145

```

```

146     qreal psiActual = 0;
147
148
149     qreal psiMinActual = numeric_limits<qreal>::infinity();
150     qint32 posPsiMin = -1;
151
152     complex <qreal> cvActual;
153     qreal absResta;
154
155     for (qint32 i = 0; i < cv->size(); i++){
156
157         cvActual = cv->at(i);
158         absResta = abs(cvActual - actual); //Calculamos el valor absoluto de la
159             distancia del punto al punto actual.
160
161         /Si la distancia es mayor que cero y menor que epsilon.
162         if (absResta > 0 && absResta <= epsilon && cvActual != actual &&
163             cvActual != anterior){ /
164
165             //-----
166
167             //calculamos la fase entre los dos puntos normalizada.
168             fasActual = arg((cvActual - actual) / (anterior - actual));
169
170             if(fasActual < 0) // si es menor que cero le sumamos 2*PI.
171                 fasActual = fasActual + 2*M_PI;
172
173             //-----
174
175             aco1Actual = qAcos(absResta / epsilon ); //calculamos la
176                 arcotangente de dicho valor absoluto
177
178             //-----
179
180             //Calculamos el psi de la fase,
181             //hay tres tipos de cálculos distintos
182             if(fasActual == 0){
183                 psiActual = 2*M_PI - aco1Actual - aco2;
184             }else if (fasActual < aco2){
185                 psiActual = fasActual + aco1Actual- aco2;
186             }else{
187                 psiActual = fasActual - aco1Actual - aco2;
188             }
189
190             if (psiActual < 0) // Si alguno es negativo se sumamos 2*PI
191                 psiActual += 2*M_PI;
192
193             //-----

```

```

193         if (psiActual < psiMinActual) { //Buscamos el psi mínimo
194             psiMinActual = psiActual; //como puede haber iguales nos
                quedamos con
195             posPsiMin = i; //la distancia mas grande.
196             dmax = absResta;
197         }else if (psiActual == psiMinActual &&
198                 (absResta > dmax)){
199             posPsiMin = i;
200             dmax = absResta;
201         }
202     }
203 }
204
205 return posPsiMin; //retornamos el mínimo
206
207 }

```

B.3 Algoritmo de cálculo de fronteras

```

1  QVector <QVector <QVector <qreal> * > * > * Boundaries::
2  bndEstabilidad(QVector<qreal> *omega, Planta *planta,
3      QVector <QVector <complex <qreal> > * > * templates,
4      ParVal * datosFas, qint32 puntosFas, ParVal * datosMag,
5      qint32 puntosMag, qreal infinito)
6  {
7      // Se genera la rejilla base del algoritmo.
8      QVector <qreal> * fases = linspace(datosFas->getX(),datosFas->getY(),
          puntosFas);
9      QVector <qreal> * mag = linspace(datosMag->getX(), datosMag->getY(),
          puntosMag);
10
11      //Se crean las variables básicas.
12      complex <qreal> p0;
13      QVector <complex <qreal> > * p;
14
15      QVector <ParVal * > * templateMovido;
16      qreal d = -numeric_limits<qreal>::infinity();
17      qreal dTemp = 0;
18      qreal inf;
19
20      QVector <QVector <QVector <qreal> * > * > * vectorSabanas = new QVector <
          QVector <QVector <qreal> * > * > ();
21      vectorSabanas->reserve(omega->size());
22
23      //Si el valor de infinito ha sido introducido por el usuario.
24      if (infinito < 0){
25          inf = numeric_limits<qreal>::infinity();
26      }else {
27          inf = infinito;
28      }
29

```

```

30 //Se recorren las frecuencias de diseño.
31 for (qint32 i = 0; i < omega->size(); i++){
32     //Se resuelve la planta con las frecuencias de diseño.
33     p0 = planta->getPunto(omega->at(i));
34     //Se obtiene cada plantilla.
35     p = templates->at(i);
36     //Una sábana por cada frecuencia de diseño.
37     QVector<QVector<qreal>*> sabana = new QVector <QVector<qreal>*>();
38     sabana->reserve(fases->size());
39
40     //Se recorre la rejilla
41     foreach (const qreal &f, *fases){
42         QVector <qreal> * sabanaVertical = new QVector <qreal> ();
43         sabanaVertical->reserve(mag->size());
44
45         foreach (const qreal &l, *mag){
46
47             //Se pasa la magnitud a lineal
48             qreal maglineal = pow(10,l/20);
49             //Se calcula el número complejo correspondiente a la posición
50             //de la rejilla y se pasa de Nichols a lineal.
51             complex<qreal> L = complex<qreal>(maglineal * cos
52                 (f * M_PI /180),
53                 maglineal * sin (f * M_PI /180));
54
55             templateMovido = new QVector <ParVal * > ();
56
57             qreal valorAnterior = 0;
58
59             //Se recorre la plantilla y se calcula la plantilla
60             //movida por el punto de la rejilla.
61             foreach (const complex <qreal> &temp, *p) {
62                 //Cálculo principal.
63                 complex <qreal> complejoMovido = temp * L / p0;
64                 //Fase del ángulo del número complejo en grados.
65                 qreal fase = arg(complejoMovido)* 180 / M_PI;
66                 //Se llama a la función unwrap.
67                 fase = unwrap(valorAnterior, fase);
68
69                 valorAnterior = fase;
70
71                 //Se guardan los puntos de la plantilla movida.
72                 ParVal *par = new ParVal(20*log10(abs(complejoMovido)),fase);
73                 templateMovido->append(par);
74             }
75             //Se comprueba si el punto crítico está dentro de
76             //la plantilla movida.
77             if (pnpoly(templateMovido, new ParVal(0, -180))
78                 pnpoly(templateMovido, new ParVal(0, 180))) {
79                 //Si el punto crítico está dentro

```

```

80         //Se pone este valor como infinito.
81         sabanaVertical->append(Inf);
82
83     }else{
84         //Sino se calcula el valor de la sábana.
85         d = -numeric_limits<qreal>::infinity();
86         foreach (const complex <qreal> &temp, *p) {
87             dTemp = 20 * log10 (abs (L / ((p0 / temp) + L)));
88             if (dTemp > d)
89                 d = dTemp;
90         }
91         templateMovido->clear();
92
93         sabanaVertical->append(d);
94     }
95 }
96 sabana->append(sabanaVertical);
97
98 }
99 vectorSabanas->append(sabana);
100 }
101
102 return vectorSabanas;
103 }
104
105 bool Boundaries::pnpoly(QVector <ParVal * > * vector, ParVal * punto)
106 {
107     qint32 i, j;
108     qint32 lon = vector->size();
109     bool c = false;
110     for (i = 0, j = lon-1; i < lon; j = i++) {
111         if (((vector->at(i)->getY() > punto->getY()) != (vector->at(j)->getY()
112             > punto->getY())) &&
113             (punto->getX() < (vector->at(j)->getX() - vector->at(i)->getX() *
114             (punto->getY()-vector->at(i)->getY()) / (vector->at(j)->getY()-
115             vector->at(i)->getY()) + vector->at(i)->getX() )
116             c = !c;
117     }
118
119     delete punto;
120     return c;
121 }
122
123 //Se normaliza a valores [-180,180):
124 qreal Boundaries::constrainAngle(qreal x){
125     x = fmod(x + M_PI,2 * M_PI);
126     if (x < 0)
127         x += 2 * M_PI;
128     return x - M_PI;

```



```
129 }
130
131 //Se convierte a valores [-360,360]
132 qreal Boundaries::angleConv(qreal angle){
133     return fmod(constrainAngle(angle),2 * M_PI);
134 }
135 qreal Boundaries::angleDiff(qreal a,qreal b){
136     qreal dif = fmod(b - a + M_PI,2 * M_PI);
137     if (dif < 0)
138         dif += 2 * M_PI;
139     return dif - M_PI;
140 }
141 qreal Boundaries::unwrap(qreal previousAngle,qreal newAngle){
142     //Hay que pasar el ángulo actual y el anterior.
143     return previousAngle - angleDiff(newAngle,angleConv(previousAngle));
144 }
```

C Anejo 3: Ejemplo de planta guardada en *XML*

En este ejemplo se va a mostrar el código resultante en *XML* de guardar la planta:

$$P(s) = k \frac{1}{(s + a)} \quad (18)$$

$$a \in [1, 5] \quad \text{Nominal} = 5$$

Como frecuencias de diseño (ω) solo va a ser una con valor 10, el fichero resultante es el siguiente:

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <QFT>
3    <Planta nombre="dd">
4      <tipo tipo="K-Ganancia">
5        <numerador size="0"/>
6        <denominador size="1">
7          <variable-0>
8            <nominal>5</nominal>
9            <variable>true</variable>
10           <nombre>a</nombre>
11           <rango>
12             <inicio>1</inicio>
13             <final>5</final>
14           </rango>
15         </variable-0>
16       </denominador>
17       <variable-k>
18         <nominal>1</nominal>
19         <variable>false</variable>
20       </variable-k>
21       <variable-ret>
22         <nominal>0</nominal>
23         <variable>false</variable>
24       </variable-ret>
25     </tipo>
26   </Planta>
27   <omega>
28     <inicio>0</inicio>
29     <final>0</final>
30     <nPuntos>1</nPuntos>
31     <tipo>2</tipo>
32     <valores>10 </valores>
33   </omega>
34   <templates>
35     <completo size="1">
36       <vector-0>-0.00990099 -0.0275229 -0.04 </vector-0>
37       <vector-0.0>-0.0990099 -0.0917431 -0.08 </vector-0.0>
38     </completo>
39     <contorno size="1">

```

```

40      <vector-0>-0.00990099 -0.04 -0.0275229 </vector-0>
41      <vector-0.0>-0.0990099 -0.08 -0.0917431 </vector-0.0>
42      </contorno>
43  </templates>
44  <boundaries>
45      <bound size="1">
46          <omega-0 size="2">
47              <vector-0>-258 -258 -257 ... -258 -258 -258 </vector-0>
48              <vector-1>4 3 2 2 1 ... 7 6 5 4 </vector-1>
49          </omega-0>
50      </bound>
51      sabana size="1">
52          <omega-0 size="361">
53              <vector-0>-59.0831 -58.0842 ... -0.00818312 -0.00729353 </vector-0>
54              <vector-1>-59.0831 -58.0842 ... -0.00812755 -0.00724399 </vector-1>
55                  .
56                  .
57                  .
58              <vector-359>-59.0832 -58.0843 ... -0.0082362 -0.00734085 </vector
                    -359>
59              <vector-360>-59.0831 -58.0842 ... -0.0081831 -0.00729353 </vector
                    -360>
60          </omega-0>
61      </sabana>
62  </boundaries>
63 </QFT>

```

Bibliografía

- [ANÓNIMO [2010]] ANÓNIMO. Aprende Qt4 desde hoy mismo. <http://es.scribd.com/doc/79497289/Aprenda-Qt4-Hoy-Mismo> [2010]. Último acceso (1/08/2013).
- [ARAMINI [1980]] M. J. ARAMINI. *Implementation of an Improved Contour Plotting Algorithm*. Tesis Doctoral Stevens Institute of Technology [1980]. Último acceso (2/08/2013).
- [BLANCHETTE Y SUMMERFIELD [2008]] J. BLANCHETTE Y M. SUMMERFIELD [2008]. *C++ gui programming with qt 4, second edition*. Prentice Hall Press, Upper Saddle River, NJ, USA second ed^{ón}.
- [BODE [1945]] H. W. BODE [1945]. *Network analysis and feedback amplifier design / H.W. Bode*. Princeton, N.J. : Van Nostrand.
- [BOEHM [1981]] B. W. BOEHM [1981]. *Software Engineering Economics*. Prentice Hall, Englewood Cliffs, NJ.
- [BORGHESANI ET AL. [2003]] C. BORGHESANI, Y. CHAIT Y O. YANIV [2003]. *The QFT Frequency Domain Control Design Toolbox For Use with MATLAB*. Terasoft, Inc. Último acceso (2/08/2013).
- [COMUNIDAD ET AL. [2000]] COMUNIDAD, COLLABNET, ELEGO, VISUALSVN Y WANDISCO. Subversion. <http://subversion.apache.org/> [2000]. Último acceso (01/08/2013).
- [DALHEIMER. [2002]] M. K. DALHEIMER. [2002]. Qt vs. Java: a comparison of Qt and Java for large-scale, industrial-strength GUI development.
- [DE HALLEUX [2002]] DE J. HALLEUX. A C++ implementation of an improved contour plotting algorithm. <http://www.codeproject.com/Articles/1727/A-C-implementation-of-an-improved-contour-plotting> [2002]. Último acceso (2/08/2013).
- [DEBIAN [2013]] DEBIAN. Benchmark. benchmarksgame.alioth.debian.org [2013]. Último acceso (31/07/2013).
- [DIGIA [1991]] DIGIA. Página web oficial proyecto Qt. <http://qt-project.org/> [1991]. Último acceso (1/08/2013).
- [DORMIDO ET AL. [2004]] S. DORMIDO, J. ARANDA Y J. DÍAZ [2004]. *An Interactive Software Tool for Learning Robust Control Design Using Quantitative Feedback Theory Methodology*. International Journal of Engineering Education.
- [EICHHAMMER [2012]] E. EICHHAMMER. QCustomPlot [Versión 0.9.0]. <http://www.qcustomplot.com/> [2012]. Último acceso (2/08/2013).
- [FRANKLIN [2003]] W. R. FRANKLIN. PNPOLY - Point Inclusion in Polygon Test. http://www.ecse.rpi.edu/~wrf/Research/Short_Notes/pnpoly.html [2003]. Último acceso (2/08/2013).

- [GAMMA ET AL. [1995]] E. GAMMA, R. HELM, R. JOHNSON Y J. VLISSIDES [1995]. *Design patterns: elements of reusable object-oriented software*. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA.
- [GARCIA-SANZ ET AL. [2009]] M. GARCIA-SANZ, A. MAUCH Y C. PHILIPPE [2009]. *QFT Control Toolbox: an Inter-active Object-Oriented Matlab CAD tool for Quantitative Feedback Theory*. 6th IFAC Symposium on Robust Control Design.
- [GERA Y HOROWITZ [1980]] A. GERA Y I. HOROWITZ [1980]. Optimization of the Loop Transfer Function. *Int. J. of Control* **31**.
- [GUTMAN [1996a]] P. GUTMAN [1996a]. *Qsyn - the Toolbox for Robust Control Systems Design for use with Matlab, Reference Guide*. El-Op Electro-Optics Industries Ltd, Rehovot, Israel.
- [GUTMAN [1996b]] P. GUTMAN [1996b]. *Qsyn - the Toolbox for Robust Control Systems Design for use with Matlab, User's Guide..* El-Op Electro-Optics Industries Ltd, Rehovot, Israel.
- [HAMANO Y TORVALDS [2012]] J. HAMANO Y L. TORVALDS. Git. <http://git-scm.com/> [2012]. Último acceso (01/08/2013).
- [HOROWITZ [1959]] I. M. HOROWITZ [1959]. *Fundamental theory of automatic linear feedback control systems*. Automatic Control, IRE Transactions on.
- [HOROWITZ [1963]] I. M. HOROWITZ [1963]. *Synthesis of feedback systems*. Academic Press, New York.
- [IBM [1996]] IBM. RUP. <http://www-01.ibm.com/software/rational/rup/> [1996]. Último acceso (1/08/2013).
- [MAGIC [2011]] N. MAGIC. Magic Draw 17. <http://www.nomagic.com/products/magicdraw.html> [2011]. Último acceso (25/08/2013).
- [MONTROYA [1998]] F. J. MONTROYA. *Diseño de sistemas de control no lineales mediante QFT: análisis computacional y desarrollo de una herramienta CACSD*. Tesis Doctoral [1998].
- [MORENO ET AL. [2006]] J. C. MORENO, A. BAÑOS Y M. BERENGUEL [2006]. Improvements on the computation of boundaries in QFT. *International Journal of Robust and Nonlinear Control* **16**, nº 12: 575–597.
- [NORDIN [1993]] M. NORDIN. *Uncertain systems with backlash: modeling, identification and synthesis*. Proyecto Fin de Carrera Royal Institute of Technology [1993].
- [PRECHELT [2000]] L. PRECHELT [2000]. An Empirical Comparison of Seven Programming Languages. *Computer* **33**, nº 10: 23–29.
- [VAN HEESCH [2013]] VAN D. HEESCH. Doxygen. <http://www.stack.nl/~dimitri/doxygen/> [2013]. Último acceso (27/08/2013).
- [W3C [1996]] W3C. eXtensible Markup Language. <http://www.w3.org/XML/> [1996]. Último acceso (1/08/2013).

-
- [WALSTON Y FELIX [1977]] C. E. WALSTON Y C. P. FELIX [1977]. A Method of Programming Measurement and Estimation. *IBM Systems Journal*.